



Hochschule München
Fakultät für Elektrotechnik und Informationstechnik

Masterstudiengang Elektrotechnik

Masterarbeit

von
Maximilian Metzler

**Umsetzung eines SLAM-basierten
Navigationsverfahrens in Bezug auf ein
automatisiert fahrendes Modellauto**

Implementation of a SLAM-based navigation method in relation to
an autonomous driving model car

Betreuer:	Prof. Dr. Arne Striegler
Bearbeitungsbeginn:	01.03.2022
Abgabetermin:	01.09.2022
lfd. Nr.:	764

Erklärungen des Bearbeiters:

Name: Metzler

Vorname: Maximilian

1. Ich erkläre hiermit, dass ich die vorliegende Masterarbeit selbständig verfasst und noch nicht anderweitig zu Prüfungszwecken vorgelegt habe.

Sämtliche benutzte Quellen und Hilfsmittel sind angegeben, wörtliche und sinngemäß Zitate sind als solche gekennzeichnet.

Erding, 06.08.2022

Ort, Datum



Unterschrift

2. Ich erkläre mein Einverständnis, dass die von mir erstellte Masterarbeit in die Bibliothek der Hochschule München eingestellt wird. Ich wurde darauf hingewiesen, dass die Hochschule in keiner Weise für die missbräuchliche Verwendung von Inhalten durch Dritte infolge der Lektüre der Arbeit haftet. Insbesondere ist mir bewusst, dass ich für die Anmeldung von Patenten, Warenzeichen oder Geschmacksmustern selbst verantwortlich bin und daraus resultierende Ansprüche selbst verfolgen muss.

Erding, 06.08.2022

Ort, Datum



Unterschrift

Kurzfassung

Menschliches Fehlverhalten zählt jedes Jahr zu den Hauptursachen für Verkehrsunfälle mit vielen Toten und Verletzten. Die Entwicklung moderner Fahrerassistenzsysteme hat bereits gezeigt, dass dadurch die Sicherheit von Fahrzeugen signifikant erhöht werden kann. Studien zeigen, dass der Ausbau hin zum komplett autonomen Automobil das Potenzial hat Unfälle komplett zu vermeiden. Aus diesem Grund wird in dieser Arbeit auf einer Miniatur-Modellautoplattform eine automatisierte Fahrunktion der Stufe vier umgesetzt. Ziel ist es anhand des Demonstrators einen Einblick in die Implementierung von automatisierter Navigation zu geben und für das Thema zu sensibilisieren. Als erstes wird mit einem SLAM-Algorithmus und einem vorher installierten 2D-LiDAR eine Karte der Büroumgebung erstellt. Im nächsten Schritt wird schließlich auf Grundlage der Karte und mithilfe von Odometriedaten und Laserscans eine automatisierte Navigation verwirklicht. Die Lokalisierung wird dabei über den AMCL-Algorithmus realisiert. Die Planung der globalen Route wird mit dem A*-Algorithmus erreicht und die Umsetzung der Steuersignale für das Motormanagement inklusive der Neuplanung der Trajektorie durch den TEB-Local-Planner umgesetzt. Fahrversuche haben gezeigt, dass eine vollautomatisierte Navigation im vorher definierten Raum sehr gut funktioniert. Außerdem wird auf statische und dynamisch bewegte Hindernisse mit einer Umplanung der Trajektorie adaptiv reagiert. In Fahrsituationen mit keiner validen Route wird vom Roboter erkannt, dass es keine Lösung gibt und der sichere Zustand eingenommen.

Abstract

Human error is one of the main causes of traffic accidents every year, resulting in many deaths and injuries. The development of modern driver assistance systems has already shown that this can significantly increase vehicle safety. Studies show that the development towards fully autonomous cars has the potential to avoid accidents completely. For this reason, a level four automated driving function is implemented on a miniature model car platform in this thesis. The aim is to use the demonstrator to provide an insight into the implementation of automated navigation and to raise awareness of the topic. First, a map of the office environment is created using a SLAM algorithm and a previously installed 2D-LiDAR. In the next step, automated navigation is finally realized based on the map and with the help of odometry data and laser scans. The localization is realized using the AMCL-algorithm. The planning of the global route is achieved with the A*-algorithm and the implementation of the control signals for the motor management including the replanning of the trajectory is realized by the TEB-Local-Planner. Driving tests have shown that fully automated navigation in the previously defined space works very well. Furthermore, static and dynamically moving obstacles are adaptively reacted to by replanning the trajectory. In driving situations with no valid route, the robot recognizes that there is no solution and assumes the safe state.

Inhaltsverzeichnis

Abbildungsverzeichnis	III
Tabellenverzeichnis	V
Abkürzungsverzeichnis	VI
Formelzeichen	VII
1. Einleitung.....	1
1.1 Motivation	4
1.2 Zielsetzung der Arbeit	5
1.3 Umfeld der Arbeit.....	6
1.4 Aufbau der Arbeit.....	6
2. Grundlagen und Stand der Technik	8
2.1 Autonomes Fahren	8
2.1.1 Die sechs Stufen des Autonomen Fahrens	9
2.1.2 Beschreibung ausgewählter Fahrfunktionen	11
2.2 Sensorik.....	13
2.2.1 Kamera.....	14
2.2.2 Radar.....	15
2.2.3 LiDAR.....	17
2.3 Kinematik.....	20
2.4 Partikel-Filter	24
2.5 Adaptive Monte Carlo Lokalisierung (AMCL)	25
2.6 Simultaneous Localization and Mapping (SLAM)	29
2.7 Globaler Pfadplaner	31
2.7.1 Dijkstra-Algorithmus.....	31
2.7.2 A*-Algorithmus.....	33
2.8 Lokaler Pfadplaner.....	34
2.8.1 Timed Elastic Band.....	34
2.8.2 Online Trajektorien-Optimierung	36

3.	Methodik und Vorgehen	39
3.1	Aufbau des Modellautos	39
3.2	Robot Operating System (ROS)	42
3.3	Inbetriebnahme des LiDARs.....	44
3.4	Kartenerstellung mit SLAM.....	49
3.4.1	Bereitstellung der Eingangsdaten	49
3.4.2	Durchführung der Kartenerstellung	52
3.5	Vollautomatisierte Navigation	54
3.5.1	Lokalisierung	55
3.5.2	Pfadplanung	57
3.5.3	Navigation	60
3.6	Erstellung von Routen	62
4.	Ergebnisse und Diskussion	64
4.1	Navigation ohne zusätzliche Hindernisse	64
4.2	Navigation mit zusätzlichen Hindernissen.....	66
4.2.1	Statische Hindernisse.....	66
4.2.2	Dynamisch bewegte Hindernisse	67
4.3	Einnahme des sicheren Zustands bei Blockierung	69
4.4	Grenzen der Navigation.....	70
4.4.1	Pfadfindung	70
4.4.2	Wendemanöver.....	71
4.4.3	Objekterkennung.....	72
5.	Fazit und Ausblick	73
	Literatur	75
	Anhang.....	80
1.	Installation und Konfiguration des Host Laptops	80
2.	Inbetriebnahme des LiDARs	82
3.	Installation und Inbetriebnahme von gmapping	82
4.	Umsetzung der vollautomatisierten Navigation mittels ROS-Navigation-Stack.....	92
5.	Programmierung eines GUI- Userinterfaces für die Routenerstellung	102

Abbildungsverzeichnis

Abbildung 1: Entwicklung der Zahl der Straßenverkehrstoten [2]	2
Abbildung 2: Arten des Fehlverhaltens des Fahrers bei Unfällen mit Personenschaden [2]	3
Abbildung 3: Begriffsabgrenzung "Autonomes Fahren" [13]	8
Abbildung 4: Stufen der Automatisierung bei Fahrsystemen [14]	10
Abbildung 5: Sensorverbund der Mercedes S-Klasse in Bezug auf den Level 3 Staupilot [20]	13
Abbildung 6: Funktionsweise einer Kamera [22, S. 352]	14
Abbildung 7: Schematischer Aufbau eines Radars [23, S. 330]	16
Abbildung 8: Funktionsweise der Frequenzmodulation FMCW [23, S. 331]	16
Abbildung 9: Funktionsprinzip eines LiDARs [22, S. 319]	17
Abbildung 10: LiDAR-Blockdiagramm zur Sende- und Empfangslogik [16, S. 141]	18
Abbildung 11: Detektion des Erfassungsbereichs vor dem Ego-Fahrzeug [23, S. 334]	19
Abbildung 12: Darstellung eines Systems mit zwei lenkbaren Achsen [25, S. 116]	20
Abbildung 13: Darstellung der Ackermann-Lenkung [25, S. 118]	22
Abbildung 14: Darstellung der Prädiktions- und Updatephase nach [29, S. 4]	27
Abbildung 15: Darstellung mehrerer Iterationen der Roboterlokalisierung [29, S. 5]	28
Abbildung 16: Ablauf des SLAM-Algorithmus mit Rao-Blackwellisiertem Partikel-Filter [31, S. 2]	30
Abbildung 17: Ablauf des Dijkstra-Algorithmus nach [33, S. 198]	31
Abbildung 18: Beispielhafter Graph zur Anwendung des Dijkstra-Algorithmus [34]	32
Abbildung 19: Ablauf des A*-Algorithmus nach [35, S. 534]	33
Abbildung 20: Schematische Darstellung des TEB-Planers [36, S. 74]	34
Abbildung 21: Darstellung von homologen Pfaden [38, S. 42]	37
Abbildung 22: Anwendungsbeispiel TEB mit Online Trajektorien-Optimierung [37, S. 152]	38
Abbildung 23: Abbildung des Modellautos	39
Abbildung 24: Darstellung aller Komponenten mit ihren Kommunikationswegen	40
Abbildung 25: Beispielhafte Kommunikation zwischen zwei ROS-Knoten [46, S. 151]	43
Abbildung 26: Darstellung des Hokuyo URG-04LX-UG01 [47]	46
Abbildung 27: Darstellung des Scan-Winkels des Hokuyo-LiDARs [48, S. 2]	46
Abbildung 28: Grafische Darstellung des Topics /scan in rviz	48

Abbildung 29: Darstellung des Input-/Output-Flusses von gmapping	49
Abbildung 30: Darstellung der Koordinatensysteme des Modellautos.....	51
Abbildung 31: Aufgezeichnete Karte eines Büroraumes mittels slam_gmapping	54
Abbildung 32: Darstellung der Lokalisierung mit AMCL [59]	55
Abbildung 33: Darstellung des initialen AMCL-Zustands	56
Abbildung 34: Darstellung der AMCL-Partikel nach mehreren Iterationen	57
Abbildung 35: Beispielhafte Pfadplanung ohne Hindernis.....	59
Abbildung 36: Beispielhafte Pfadplanung mit Hindernis.....	60
Abbildung 37: Übersicht des Navigationsablaufs [61]	60
Abbildung 38: Darstellung der GUI zur automatisierten Navigation.....	62
Abbildung 39: Darstellung einer Route ohne zusätzliche Hindernisse	65
Abbildung 40: Abbildung der Trajektorie um drei statische Hindernisse.....	66
Abbildung 41: Trajektorienplanung um ein bewegtes Objekt von der Seite.....	67
Abbildung 42: Trajektorienplanung um ein bewegtes Objekt von vorne	68
Abbildung 43: Einnahme des sicheren Zustands bei Blockierung	69
Abbildung 44: Planung eines nicht optimalen Pfades durch Hindernisse hindurch	71
Abbildung 45: Darstellung eines geplanten Wendemanövers	72

Tabellenverzeichnis

Tabelle 1: Aufbau der Nachricht sensor_msgs/LaserScan nach [50].....	47
Tabelle 2: Aufbau der Nachricht nav_msgs/Odometry Message nach [53]	50
Tabelle 3: Aufbau der Nachricht tf2_msgs/TFMessage nach [54]	51
Tabelle 4: Aufbau der Nachricht nav_msgs/OccupancyGrid Message nach [55].....	52
Tabelle 5: Aufbau der Nachricht nav_msgs/MapMetaData nach [56]	53
Tabelle 6: Auflistung ausgewählter Pfadplanungsparameter	58
Tabelle 7: Aufbau der Nachricht AckermannDriveStamped Message nach [62]	61

Abkürzungsverzeichnis

Abkürzung	Beschreibung
ACC	Adaptive Cruise Control
ADAS	Advanced Driver Assistance Systems
AMCL	Adaptive Monte Carlo Localization
CACC	Cooperative Adaptive Cruise Control
CMOS	Complementary Metal Oxide Semiconductor
DGPS	Differential Global Positioning System
DSRC	Dedicated Short-Range Communication
EKF	Extended Kalman Filter
ESC	Electronic Speed Controller
FFT	Fast Fourier Transformation
FOV	Field of View
IMU	Inertial Measurement Unit
LiDAR	Light Imaging, Detection and Ranging
LSA	Lenk- und Spurführungsassistent
NHA	Nothalteassistent
PDF	Probability Density Function
Radar	Radio Detection and Ranging
ROS	Robot Operating System
SLAM	Simultaneous Localization and Mapping
TEB	Timed Elastic Band

Formelzeichen

<i>Formelzeichen</i>	<i>Beschreibung</i>
A	Fläche
B	Elastisches Band (TEB)
c	Lichtgeschwindigkeit
d	Abstand
f	Frequenz
$h(x)$	Heuristik
$H(\tau)$	H-Signatur eines Pfades τ
I	Impulse eines Inkrementalwertgebers
m	Karte
$p(x)$	Wahrscheinlichkeitsdichtefunktion
t	Zeit
v	Geschwindigkeit
w	Gewicht
x_t	Zustandsvektor zum Zeitpunkt t
y_t	Messung zum Zeitpunkt t
S_t	Partikelmenge zum Zeitpunkt t
λ	Wellenlänge
σ	Radarquerschnitt
δ	Radwinkel
β	Schwimmwinkel
τ	Pfad
Ω	Umfang
$\Delta\theta$	Gierrate
θ	Impulse pro Radumdrehung

1. Einleitung

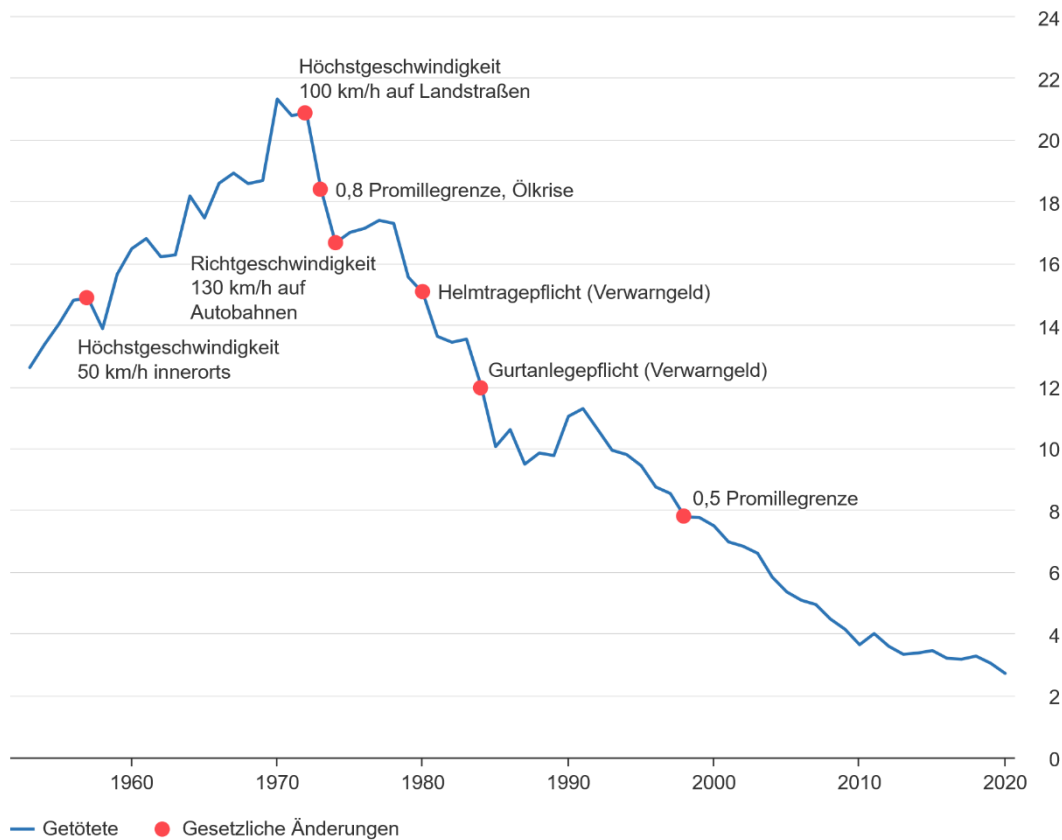
Nach einem Bericht der WHO (World Health Organization) sterben weltweit jährlich ca. 1.35 Millionen Menschen im Straßenverkehr [1, S. 3]. In Deutschland sind laut Zahlen des Statistischen Bundesamtes im Jahr 2020 2719 Menschen gestorben und 327 550 wurden verletzt [2]. In den USA sind unter Berücksichtigung der Bevölkerungsanzahl sogar noch mehr Menschen gestorben. Dort beläuft sich die Zahl der Verkehrstoten im Jahr 2020 auf 38 680 [3]. Die bereinigte Todeszahl ist somit um das Vierfache höher als in Deutschland. Die Association for Safe International Road Travel (ASIRT) gibt sogar die Verletzungen im Straßenverkehr als häufigste Todesursache bei jungen Menschen im Alter von 5-29 Jahren an [4].

All diese Zahlen belegen, dass der Verkehrssektor nach wie vor eine zentrale Rolle im Hinblick auf den Schutz vulnerabler Gruppen und dem Retten von Menschenleben einnimmt. Abbildung 1 zeigt die Entwicklung der Zahl der Straßenverkehrstoten in Deutschland. Dieser Verlauf kann stellvertretend für ein Industrieland angenommen werden. Die Zahl der Getöteten steigt bis zum Maximum im Jahr 1970 mit ca. 22 000 Toten an. Danach nimmt sie durch die Einführung bestimmter Gegenmaßnahmen wie z.B. der Höchstgeschwindigkeit 100 km/h auf Landstraßen oder der Gurtanlegepflicht schrittweise ab. Seit den 2000er Jahren kann man beobachten, dass keine weiteren Beschränkungen erlassen wurden, die Kurve jedoch immer weiter abfällt. Dies ist vor allem auf die voranschreitende technische Entwicklung in Fahrzeugen zurückzuführen.

Die Fahrzeugsicherheit ist zunehmend entscheidend für die Vermeidung von Unfällen und trägt maßgeblich dazu bei, die Zahl der Toten und Verletzten im Straßenverkehr zu reduzieren [1, S. 57-58]. Als Beispiele können hier zum einen Elemente der passiven Sicherheit wie Airbags, oder energieabsorbierende Bauteile genannt werden. Zum anderen gibt es Funktionen der aktiven Sicherheit, die explizit einen Unfall verhindern können. Dazu zählen ESP, ABS und seit wenigen Jahren Regelsysteme, die in die Längs- und Querverführung des Fahrzeugs eingreifen [5, S. 15-16]. Beispiele dafür sind der Nothalteassistent (NHA), Lenk- und Spurführungsassistent (LSA) oder Adaptive Cruise Control (ACC). Ausgewählte Funktionen werden für ein besseres Verständnis des Themas in Kapitel 2.1.2 ausführlicher beschrieben.

Entwicklung der Zahl der im Straßenverkehr Getöteten

in Tausend



© Statistisches Bundesamt (Destatis), 2021

Abbildung 1: Entwicklung der Zahl der Straßenverkehrstoten [2]

Um die Zahlen der Verkehrstoten weiter reduzieren zu können, müssen die Hauptursachen näher beleuchtet werden. Das Statistische Bundesamt gibt mit 88.5 % menschliches Fehlverhalten des Fahrzeugführers als mit Abstand häufigste Ursache an. Andere Ursachen wie schlechte Sicht oder Glätte waren nur zu lediglich 7.5 % der ausschlaggebende Faktor und Fehlverhalten von Fußgängern nur zu 2.9 % [6, S. 49]. Abbildung 2 zeigt eine tiefere Aufschlüsselung der Arten des menschlichen Fehlverhaltens, das zu Unfällen mit Personenschaden geführt hat. Die Grafik zeigt sehr deutlich, dass die Nichteinhaltung der Verkehrsregeln wie zum Beispiel Missachtung der Vorfahrt, falsche Straßenbenutzung (Einbahnstraße), zu geringes Abstandhalten oder überhöhte Geschwindigkeit die Hauptursachen darstellen. Darüber hinaus stellt auch Drogenkonsum wie Alkoholeinfluss ein ernstzunehmendes Problem dar. Somit geht vom Menschen die größte Gefahr aus und weniger von äußeren Umweltfaktoren. Schlussfolgernd lässt sich sagen, dass ein weiterer Ausbau und die Fortentwicklung von Fahrerassistenzsystemen

das größte Potenzial haben, um Unfälle zu vermeiden und Menschenleben zu retten. Durch diese technischen Systeme kann der Fahrer aktiv unterstützt werden oder im Extremfall die Fahrzeugführung komplett übernommen werden. Ein autonomes Fahrzeug der Stufe 5 (s. Kapitel 2.1.1) benötigt keinen Fahrer mehr und erledigt alle Fahraufgaben komplett selbstständig. Folglich ist das Risiko eines menschlichen Fehlverhaltens praktisch bei null. Unfälle sind zwar in der Zukunft nach wie vor nicht komplett ausgeschlossen, aber sie lassen sich erheblich reduzieren, da die Technologie viel zuverlässiger und sicherer funktioniert als ein menschlicher Fahrer [7].

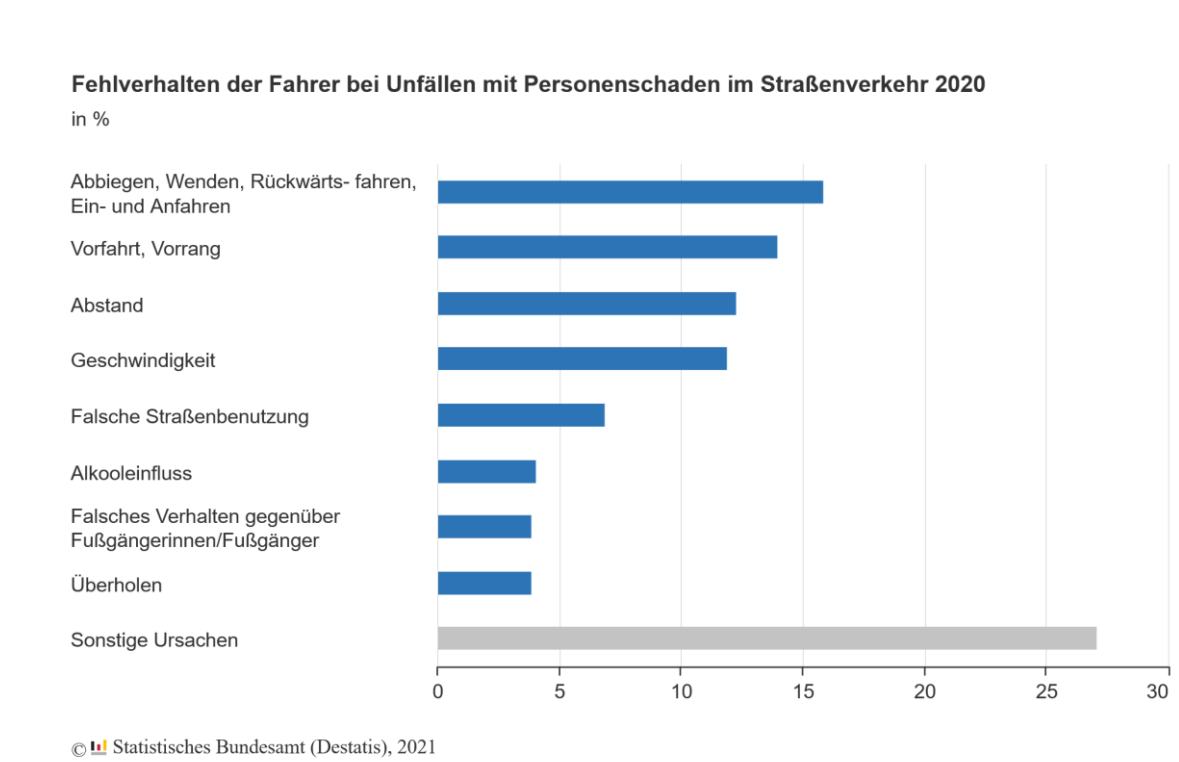


Abbildung 2: Arten des Fehlverhaltens des Fahrers bei Unfällen mit Personenschaden [2]

1.1 Motivation

Das größte Potenzial, das von ADAS-Funktionen (Advanced Driver Assistance Systems) ausgeht, ist die Erhöhung der Verkehrssicherheit und somit der Vermeidung von Unfalltoten [8, S. 4]. Wie bereits im vorherigen Kapitel beschrieben, sinkt die Zahl der jährlichen Verkehrstoten vor allem durch den Einsatz moderner Assistenzsysteme stetig, jedoch stellen Verkehrsunfälle noch immer die häufigste Todesursache bei jungen Menschen dar [4]. Es gibt bereits Prognosemodelle wonach im Jahr 2070 durch autonome Fahrzeuge Unfälle komplett vermieden werden können [8, S. 367]. Damit könnte die Zahl der Verletzten und Unfalltoten auf ein Minimum reduziert werden.

Die Motivation des autonomen und automatisierten Fahrens ist jedoch noch weit vielfältiger. Ein weiterer Vorteil ist die gezielte Steuerung des Verkehrsflusses durch CACC (Cooperative Adaptive Cruise Control). Das bedeutet es wird ein neuer Kommunikationskanal mittels DSRC (Dedicated Short-Range Communication) verwendet, um Beschleunigungs- und Verzögerungsinformationen an die folgenden Autos zu senden. Damit ist es möglich Staus, die durch unvorhersehbares Beschleunigen oder Abbremsen verursacht werden, zu verhindern [9, S. 240-241]. Dadurch würde sogar zusätzlich die Effizienz und Umweltfreundlichkeit der Fahrzeuge steigen. Falls ein Stau unvermeidbar ist, werden aktuell Staupiloten entwickelt, die der Fahrstufe 3 (s. Kapitel 2.1.1) entsprechen. Seit Juni 2021 sind dafür auch die rechtlichen Grundlagen geschaffen. Daimler hat im Herbst 2021 als erster deutscher OEM einen Staupiloten in Serie gebracht, der auf Autobahnen bis 60 km/h das Fahren übernimmt [10]. Diese Funktion ermöglicht es dem Fahrer nicht mehr permanent aufmerksam sein zu müssen. Folglich entstehen im Auto völlig neue Nutzungsräume. Der Fahrer kann in der Zwischenzeit ein Buch oder Mails lesen, sich mit den Passagieren aktiv beschäftigen oder einen Film schauen. Das bedeutet Zeit, die ansonsten sinnlos im Stau vergeudet wird, kann proaktiv genutzt werden.

Diese Masterarbeit entsteht bei ARRK Engineering am BMW Autonomous Driving Campus. ARRK fungiert an diesem Standort als Entwicklungsbetreuer für ADAS-Funktionen von BMW. Ziel der Arbeit ist es an einem Miniatur-Modellauto eine vollautomatisierte Navigation zu implementieren. Dieses Modellauto dient dazu Fahrerassistenzsysteme praktisch umzusetzen und deren Funktionsweise kennenzulernen. Das Auto erfüllt also den Zweck eines Demonstrators, um für interne Mitarbeiter des Standortes oder auch externe Partner das Thema des autonomen Fahrens näher zu veranschaulichen. Die umgesetzten Funktionen besitzen dabei nicht den Anspruch oder das Ziel in realen Serienfahrzeugen zum Einsatz zu kommen, da sie dafür viel komplexer abgesichert werden müssten.

1.2 Zielsetzung der Arbeit

In diesem Kapitel werden die Zielsetzung und die einzelnen Teilschritte der Arbeit detaillierter erklärt. Wie bereits im vorherigen Abschnitt beschrieben, ist es das Ziel der Arbeit an einem Miniatur-Modellauto eine vollautomatisierte Navigation der Stufe 4 umzusetzen. Das bedeutet das Fahrzeug erledigt in einem vorher definierten Anwendungsfall alle Fahraufgaben komplett selbstständig und der menschliche Fahrer kann die Kontrolle vollständig abgeben, ohne vom Auto aufgefordert zu werden das Steuer zu übernehmen. Falls das System mit einer Situation nicht mehr umgehen kann, wird die Systemgrenze rechtzeitig erkannt und der sichere Zustand hergestellt [5, S. 11].

Das vorher festgelegte Anwendungsgebiet ist in diesem Fall ein vorher definierter Büroraum. Innerhalb dieses Raumes soll ein vorgegebener Zielpunkt automatisiert erreicht werden. Der Startpunkt muss dem Fahrzeug bei Fahrtbeginn bekannt sein. Während der Fahrt sollen alle Hindernisse erkannt und wenn nötig umfahren werden. Das Modellauto soll dabei immer den kürzesten Pfad zum Ziel wählen. Um dies zu erreichen, wird im ersten Schritt ein LiDAR auf das Modellauto montiert und in Betrieb genommen. Damit wird es möglich alle Hindernisse zu erkennen. Als nächstes wird mit einem SLAM (Simultaneous Localization and Mapping) – Verfahren und dem LiDAR-Sensor eine 2D-Karte der Umwelt erstellt. Dafür werden Odometriedaten und die erkannten Markierungspunkte des Raumes über einen Partikel-Filter so fusioniert, dass der Roboter immer weiß, wo er sich befindet. Für die Kartenerstellung muss das Modellauto noch manuell mit einer Joysticksteuerung durch die Umgebung bewegt werden. Die erstellte Karte kann mit den Navigationskarten von realen Serienfahrzeugen verglichen werden, die auch bereits vor Fahrtbeginn verfügbar sein müssen. Diese sind zwar noch weitaus komplexer, die Grundinformationen wo sich Hindernisse befinden und welche Routen möglich sind, sind jedoch identisch.

Ist die Karte erstellt und dem Roboter seine Startposition auf der Karte bekannt, wird über einen globalen Pfadplaner der kürzeste Weg zum Ziel berechnet. Der Weg wird anschließend über einen lokalen Pfadplaner umgesetzt. Dieser generiert für die Motorsteuerung die passenden Signale und führt bei Hinderniserkennung eine Neuplanung des Pfades aus. Auf dem Weg zum vorgegebenen Ziel wird die Lokalisierung über den AMCL-Algorithmus (Adaptive Monte Carlo Localization) durchgeführt. Die Umsetzung der kompletten Navigationsfunktion erfolgt innerhalb von ROS (Robotic Operating System) und mithilfe entsprechender ROS-Toolboxen. Abschließend wird eine Python-GUI erstellt, mit der feste Routen abgefahren werden können. Das heißt das Programm soll die Signale für den Navigation-Stack liefern, um dann beispielsweise eine Route vom Startpunkt zum Kühlschrank (Getränke holen) oder Büroeingang (Post abholen) und anschließend zu

einem bestimmten Arbeitsplatz abzufahren. Dies entspricht dann dem Use-Case eines Büroroboters, der Transporttätigkeiten übernehmen kann. Zum Schluss soll es möglich sein eine Aussage darüber zu treffen, wie gut die automatisierte Navigation innerhalb eines Raumes funktioniert und wo die Grenzen des Systems liegen.

1.3 Umfeld der Arbeit

Diese Masterarbeit wird bei der ARRK Engineering GmbH am Standort Unterschleißheim angefertigt. Gegründet wurde die Firma im Jahr 1948 in Osaka. Im Laufe der Zeit ist ARRK stetig gewachsen und besteht mittlerweile aus 20 Unternehmen, die über die ganze Welt verteilt sind. In Europa sind die vier Bereiche Engineering, Prototyping, Tooling und Low Volume Production beheimatet. In diesen Domänen sind mehr als 2000 Personen beschäftigt und es wird ein jährlicher Umsatz von ca. 225 Mio. Euro generiert [11]. Die Kompetenzen von ARRK liegen in der Planung bis hin zur Entwicklung und Fertigung eines neuen Produkts. Zahlreiche Kunden vor allem aus der Automobilbranche nehmen bereits das breite Portfolio von ARRK in Anspruch [12]. Am Standort München und Unterschleißheim stellt die BMW Group den Hauptkunden dar. Am Standort Unterschleißheim (BMW Autonomous Driving Campus) unterstützt ARRK bei der Entwicklung der neuesten Fahrerassistenzsysteme mit dem langfristigen Ziel Autonomes Fahren zu realisieren. Hierbei werden Tests und Absicherungen der Fahrsysteme, sowie die Funktionsweiterentwicklung durchgeführt. Dabei sind vor allem Kenntnisse aus der Welt der autonomen Systeme und Softwareentwicklung gefragt.

1.4 Aufbau der Arbeit

Zunächst werden in Kapitel 2 die technischen Grundlagen nähergebracht. Dabei wird zuerst der Begriff des autonomen Fahrens genauer erläutert und schließlich die Untergliederung der fünf autonomen Fahrstufen dargestellt. Anschließend werden ausgewählte Fahrerassistenzsysteme und die dafür benötigte Sensorik vorgestellt, um einen tieferen Einblick in die Thematik zu geben. Danach werden alle technischen Bestandteile erklärt, die benötigt werden, um eine vollautomatisierte Navigation in Bezug auf das Modellauto umzusetzen. Dazu gehört die Roboterkinematik, Funktionsweise eines Partikel-Filters, der AMCL-Algorithmus zur Lokalisierung auf einer Karte, das SLAM-Verfahren zur Kartenerstellung, globaler und lokaler Pfadplaner.

In Kapitel 3 geht es um die konkrete Umsetzung der automatisierten Navigation. Es wird als erstes der Aufbau des Modellautos beschrieben und schließlich die Implementierungsschritte der einzelnen Komponenten von der Inbetriebnahme des LiDARs, der Kartenerstellung bis hin zur eigenständigen Pfadplanung und Lokalisierung.

In Kapitel 4 wird die Navigation des Autos ausgewertet und es werden die erzielten Ergebnisse präsentiert. Außerdem wird auf mögliche Grenzen des Systems eingegangen. Kapitel 5 bildet den Abschluss. Darin wird ein Fazit gezogen und ein Ausblick auf zukünftige Weiterentwicklungen des Modellautos gegeben.

2. Grundlagen und Stand der Technik

Dieses Kapitel gibt einen Überblick zu den technischen Grundlagen, die zum automatisierten Fahren benötigt werden. Der Fokus liegt dabei auf dem SLAM-basierten Navigationsverfahren.

2.1 Autonomes Fahren

Zuerst macht es Sinn sich den Begriff „Autonomes Fahren“ genauer anzusehen. Heutzutage wird dieser Ausdruck oft stellvertretend für alle möglichen verschiedenen Fahrstufen und Funktionen gebraucht. Die Bundesanstalt für Straßenwesen liefert zu diesem Thema eine gute Begriffsabgrenzung (s. Abbildung 3).

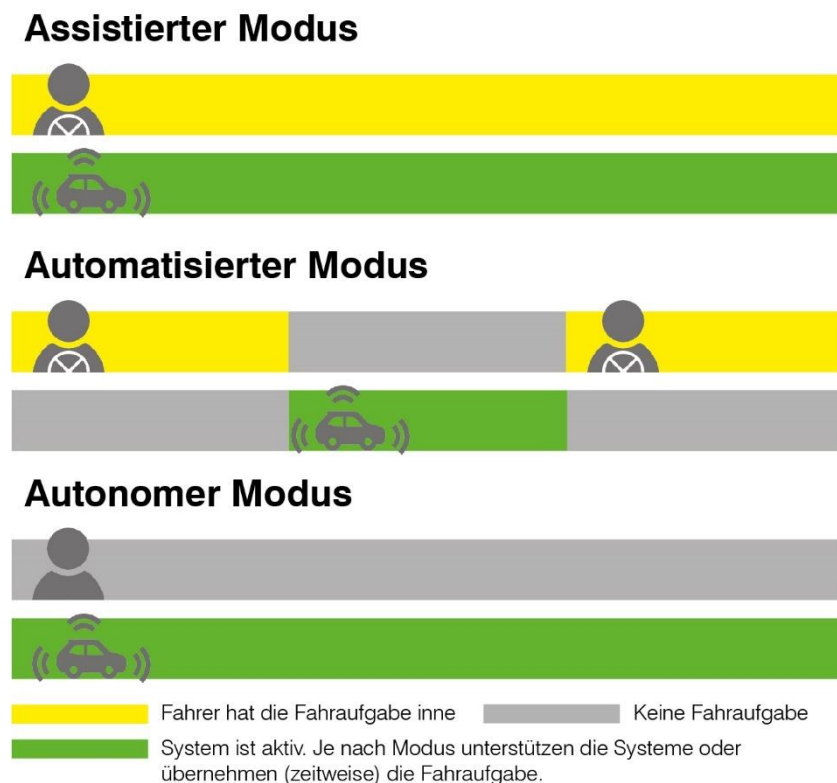


Abbildung 3: Begriffsabgrenzung "Autonomes Fahren" [13]

Wie in Abbildung 3 zu sehen ist, verwendet das Modell drei verschiedene Begrifflichkeiten. Der Assistierte Modus unterstützt den Fahrer zeitweise. Dabei muss der Fahrer permanent aufmerksam sein und gegebenenfalls die Kontrolle übernehmen. Ein Beispiel hierfür wäre

ACC und der Lenk- und Spurführungsassistent. Diese Funktionen greifen aktiv in die Längs- und Querführung ein, können aber nur zeitweise die Fahraufgabe ausüben. Dies entspricht nach SAE-Standard J3016 (s. Kapitel 2.1.1) allen Fahrstufen von 0 bis einschließlich 2. Beim Automatisierten Modus muss der Fahrzeugführer hingegen nicht mehr aufmerksam sein und kann sich fremden Tätigkeiten widmen. Das bedeutet das Fahrerassistenzsystem kann in einer vorher definierten Situation (z.B. Autobahnfahrt) alle Aufgaben automatisiert übernehmen. Falls eine Systemgrenze erreicht wird, wird der Fahrer mit ausreichend Zeitabstand informiert und muss übernehmen. Dieses Szenario wird mit Stufe 3 erreicht. Ein Vertreter dafür ist, wie in [10] beschrieben, der Staupilot. Die Bundesanstalt für Straßenwesen ordnet schließlich Systeme der Stufen 4 und 5 dem Autonomen Modus zu. In der Regel ist in der Literatur bei Stufe 4 vom vollautomatisierten Fahren die Rede. Da es jedoch Zweifel gibt, ob das autonome Fahren nach Stufe 5 überhaupt in der Zukunft erreichbar sein wird, wird hier bereits bei Stufe 4 von autonomem Fahren gesprochen. Entscheidend ist, dass alle Menschen im Fahrzeug nur noch Passagiere sind. Der Mensch muss nicht aufmerksam sein und auch nicht in das Fahrgeschehen eingreifen können [13].

2.1.1 Die sechs Stufen des Autonomen Fahrens

Wie bereits angesprochen lassen sich Fahrerassistenzsysteme in sechs Fahrstufen einteilen. Diese Einteilung geht auf den Standard J3016 der SAE (Society of Automotive Engineers) zurück. Dieser stellt die meistzitierte Referenz in der Automobilbranche für die Fähigkeiten automatisierter Fahrzeuge dar [14]. Die Stufen der unterschiedlichen Automatisierungsgrade sind in Abbildung 4 zu sehen.

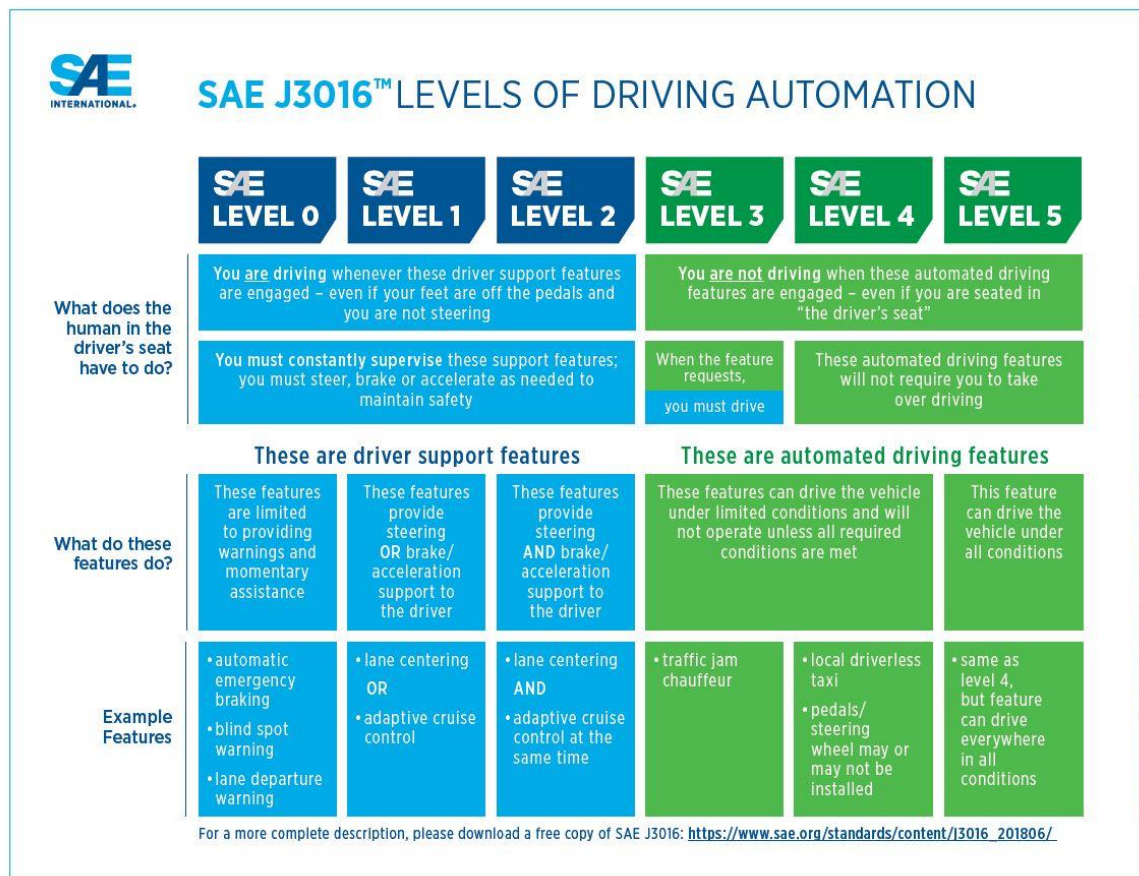


Abbildung 4: Stufen der Automatisierung bei Fahrsystemen [14]

Stufe 0: Stufe 0 bedeutet, dass der Fahrer komplett selbstständig fährt, auch wenn unterstützende Systeme vorhanden wären [15, S. 59]. Assistenzsysteme der Stufe 0 sind somit meistens Warnsysteme (z.B. Spurverlassenswarnung) oder Funktionen, die nur in einem kurzen Moment aktiv sind wie der automatische Notbremsassistent [14].

Stufe 1: Bei der folgenden Fahrstufe wird für einen beschränkten Zeitraum die Längs- oder Querführung übernommen. Wichtig ist, dass der Fahrer stets aufmerksam ist, die Hände am Lenkrad positioniert hat und zu jedem Moment die Kontrolle übernehmen kann. Ein Beispiel dafür ist Adaptive Cruise Control (ACC, s. Kapitel 2.1.2) [5, S. 10-11].

Stufe 2: Diese Stufe wird auch als Teilautomatisiertes Fahren bezeichnet. Der Unterschied zur Stufe 1 ist, dass sowohl Längsführung als auch Querführung für einen bestimmten Zeitraum übernommen werden. Der Fahrer muss nach wie vor aufmerksam und stets eingriffsbereit sein. Repräsentiert wird diese Stufe durch die parallele Aktivierung von ACC und LSA (Lenk- und Spurhalteassistent) [5, S. 10-11].

Stufe 3: Stufe 3 stellt das Hochautomatisierte Fahren dar. Hier wird in einer definierten Situation (z.B. Autobahnfahrt) ebenfalls die Längs- und Querführung übernommen. Jedoch

muss der Fahrer nicht mehr permanent aufmerksam sein und kann sich somit anderen Tätigkeiten widmen. Wird eine Systemgrenze erreicht, muss der Fahrer unter einer ausreichenden Zeitreserve die Kontrolle übernehmen. Ein Beispiel hierfür ist der Staupilot [5, S. 10-11].

Stufe 4: Beim Vollautomatisierten Fahren werden in einem definierten Anwendungsfall alle Fahraufgaben vom System übernommen. Alle Mitfahrer werden zu Passagieren [5, S. 10-11]. Erfordert es die Situation kann auch hier das Fahrsystem den Fahrer zur Übernahme auffordern. Der entscheidende Unterschied zur vorherigen Stufe ist jedoch, dass der Fahrer dieser Aufforderung nicht nachkommen muss. In diesem Fall nimmt das System den sicheren Zustand ein. Das kann bedeuten, dass zum Beispiel der nächste Parkplatz angesteuert. Ein Vertreter dieser Stufe ist das Valet Parking (s. Kapitel 2.1.2) [15, S. 59-60].

Stufe 5: Erst bei Stufe 5 wird vom Autonomen Fahren gesprochen. Hierbei werden alle Fahrfunktionen nicht nur in einem definierten Anwendungsfall übernommen, sondern vollständig vom Start bis zum Ziel der Fahrt [5, S. 10-11]. Dies stellt eine besondere Herausforderung dar, da die verwendete Sensorik meistens Grenzen hat (z.B. schlechtes Wetter, kein GPS-Empfang). Insofern ist es bei dieser Fahrstufe unabdingbar eine Vernetzung unter den Fahrzeugen oder auch mit den Verkehrsleitsystemen wie Ampeln zu gewährleisten [15, S. 58-59].

2.1.2 Beschreibung ausgewählter Fahrfunktionen

In diesem Abschnitt werden exemplarisch die Funktionen ACC, LSA, Staupilot und Valet Parking vorgestellt, da sie stellvertretend für die Fahrstufen eins bis vier stehen.

ACC: ACC steht für Adaptive Cruise Control und wird der Stufe 1 zugeordnet. Dabei wird zu Beginn auf eine vom Fahrer eingestellte Geschwindigkeit geregelt. Wird auf ein langsames Fahrzeug getroffen oder schert ein anderes Fahrzeug ein, wird der Abstand zum vorausfahrenden Fahrzeug entsprechend den vorher getätigten Einstellungen angepasst. Somit stellt diese Funktion eine adaptive Geschwindigkeitsregelung dar, die sowohl in den Antriebsverbund als auch in die Bremse eingreift [16, S. 158]. Dafür wird ein Radar-Sensor benötigt. Dieser detektiert Objekte (in neueren Versionen auch im Verbund mit einer Kamera). Um den Abstand und die Relativgeschwindigkeit von Objekten erkennen zu können, werden Radiowellen im Frequenzbereich von 76 bis 77 GHz mit einer Wellenlänge von ca. 4 mm verwendet. Die größte Herausforderung besteht darin die Objekte korrekt in den eigenen Fahrschlauch abzubilden. Nur so kann das richtige Zielobjekt ausgewählt werden [16, S. 159-161].

LSA: LSA steht für Lenk- und Spurführungsassistent. Im Sprachgebrauch werden für die gleiche Funktion oft auch Spurhalteassistent oder aktive Lenkunterstützung verwendet. Ziel des Systems ist es das Fahrzeug in der Fahrspur zu halten. Aufgrund von Ablenkung oder Müdigkeit kommt es nämlich häufig zum unbeabsichtigten Verlassen der Spur und letztendlich zum Unfall [17]. Wesentlich für diese Funktion ist das Erkennen der Fahrbahnmarkierungen. Diese werden verwendet, um die optimierte Fahrlinie zu berechnen. Ist der Abstand zur Fahrbahnbegrenzung zu gering, wird eingegriffen [18, S. 2]. Die Erkennung der Fahrlinien wird mit Multifunktionskameras durchgeführt. Diese können durch einen breiten Öffnungswinkel und eine große Reichweite von >150 m zum einen Objekte und zum anderen auch Linien erkennen [19]. Wird die LSA-Funktion zusammen mit ACC kombiniert entspricht dies dem teilautomatisierten Fahren (Level 2).

Staupilot: Der Staupilot zählt zu den Fahrfunktionen der Stufe 3 wie er seit Herbst 2021 in der Mercedes S-Klasse zu finden ist. Der Unterschied zu bisherigen Stauassistenten ist, dass der Fahrer nicht mehr aufmerksam sein muss und die Übernahme nicht sofort, sondern mit einem gewissen Zeitspiel erfolgen kann. Des Weiteren wird vom Hersteller auch die volle Haftung für das System übernommen. Die Funktion ist nur auf der Autobahn, bei gutem Wetter und bei Geschwindigkeiten bis zu 60 km/h aktivierbar. Das System regelt die Längs- und Querführung des Fahrzeugs und kann sogar Ausweichmanöver durchführen. Die technischen Voraussetzungen für das Assistenzsystem bilden eine Reihe von verschiedenen Sensoren. Zum einen erfolgt die Fahrspurerkennung über eine Stereokamera. Zum anderen wird die komplette Umgebung durch weitere Kameras, Radar- und Ultraschallsensoren abgebildet. Komplett neu ist zudem der Einsatz eines LiDARs, der wie der Frontradar am Kühlergrill installiert ist. Darüber hinaus ist es bei der Absicherung von Level 3-Funktionen sehr wichtig die aktuelle Position des Fahrzeugs zentimetergenau mittels Differential GPS (DGPS) zu bestimmen [20]. Da die Positionsbestimmung von herkömmlichem GPS zu ungenau ist, werden ortsfeste Referenzstationen verwendet, um die Position zu korrigieren [21, S. 1619-1620].

Valet Parking: Da es im Moment noch keine Level 4-Funktionen in Serienfahrzeugen gibt, wird hier ein kurzer Ausblick zum Valet Parking gegeben. Dieses System wird aktuell beispielsweise von Daimler und Bosch zusammen erforscht und erprobt. Dabei wird das Auto in einer „Drop Off Area“ eines Parkhauses abgestellt. Anschließend sucht das Fahrzeug selbstständig und ohne Passagiere einen Parkplatz und führt ein Parkmanöver durch. Wird das Auto wieder benötigt, kann es mittels Smartphone-Befehl angefordert werden. Das Fahrzeug parkt dann wieder aus und begibt sich zu einer „Pick Up Area“ [15, S. 61-62].

2.2 Sensorik

In den bisherigen Kapiteln wurde recht deutlich erkennbar, dass automatisiertes oder autonomes Fahren nur mit entsprechender Sensorik möglich ist. Je komplexer die Fahrfunktion ist, desto mehr verschiedene Sensoren werden meistens verbaut, um die Sicherheit zu erhöhen. Exemplarisch ist in Abbildung 5 der Sensorverbund der Mercedes S-Klasse in Bezug auf den Staupilot der Stufe 3 zu sehen.

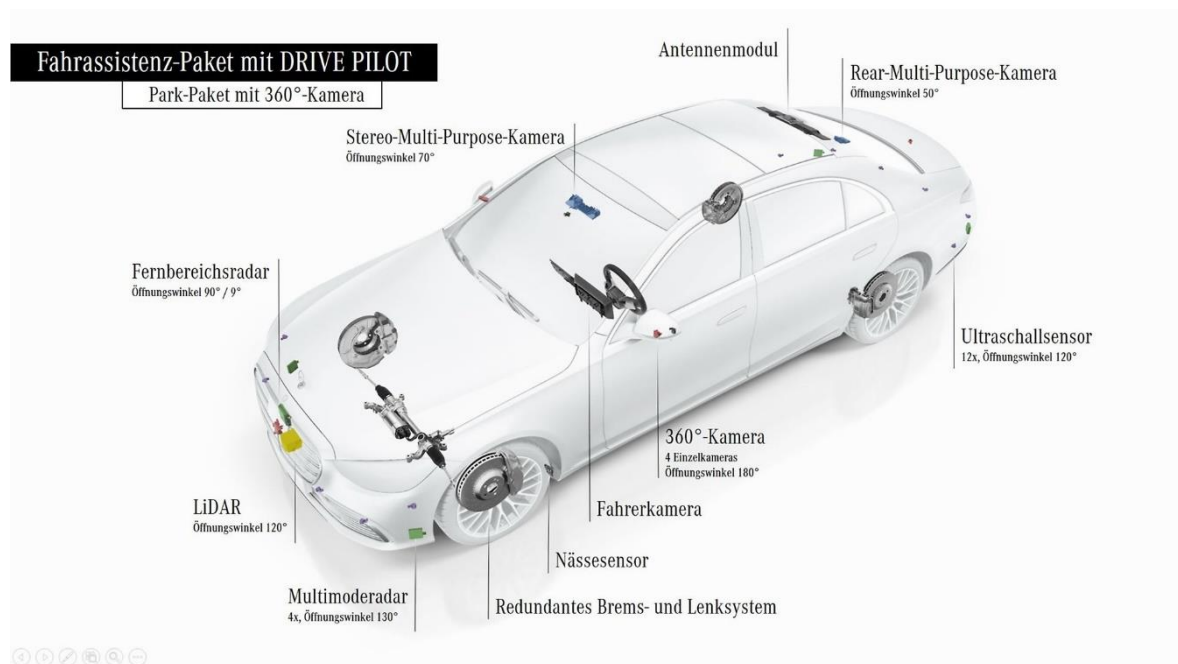


Abbildung 5: Sensorverbund der Mercedes S-Klasse in Bezug auf den Level 3 Staupilot [20]

Wie in Abbildung 5 zu sehen ist, wird das Assistenzsystem durch die Kombination von mehreren Sensoren realisiert. An der Front des Fahrzeugs befinden sich Radar, LiDAR und eine Stereokamera. Damit kann zum einen die Fahrspur erkannt und somit eine Trajektorie berechnet werden. Zum anderen werden damit Objekte vor dem Fahrzeug detektiert. Darüber hinaus befinden sich an der Fahrzeugseite beziehungsweise am Heck diverse Ultraschallsensoren und weitere Kameras. Diese werden benötigt, um seitliche Objekte zu identifizieren, die bei einem Ausweichmanöver oder Spurwechsel eine Rolle spielen. Außerdem werden ein leistungsfähiges Antennenmodul und ein Nässesensor verwendet. Dadurch wird sichergestellt, dass das System nur bei trockenem Wetter und auf der Autobahn aktivierbar ist. Zuletzt ist noch erwähnenswert, dass der Fahrer permanent durch eine Fahrerkamera beobachtet wird. Der Fahrer muss zwar bei aktivierter Funktion nicht

mehr aufmerksam sein, er muss sich aber trotzdem auf dem Fahrersitz befinden, um, falls nötig, innerhalb einer bestimmten Zeit das Steuer wieder übernehmen zu können.

2.2.1 Kamera

Kameras finden im Automobil in vielfältiger Hinsicht Anwendung. Sie werden für die Fahrer- und Innenraumüberwachung, für Handgestenerkennung, Objektdetektion oder Umwelterfassung verwendet [22, S. 348-349]. In diesem Abschnitt wird nur auf Kameras eingegangen, die in der Objekterkennung Anwendung finden, da dies im Hinblick auf automatisierte Navigation die größte Bedeutung hat. Heutige Kamerasysteme dieser Art arbeiten meistens im sichtbaren Spektralbereich. Dadurch können die gleichen Verkehrsmerkmale, die vom menschlichen Auge erfasst werden, am besten verarbeitet werden [22, S. 350-351]. Die generelle Funktionsweise einer Kamera ist in Abbildung 6 dargestellt.

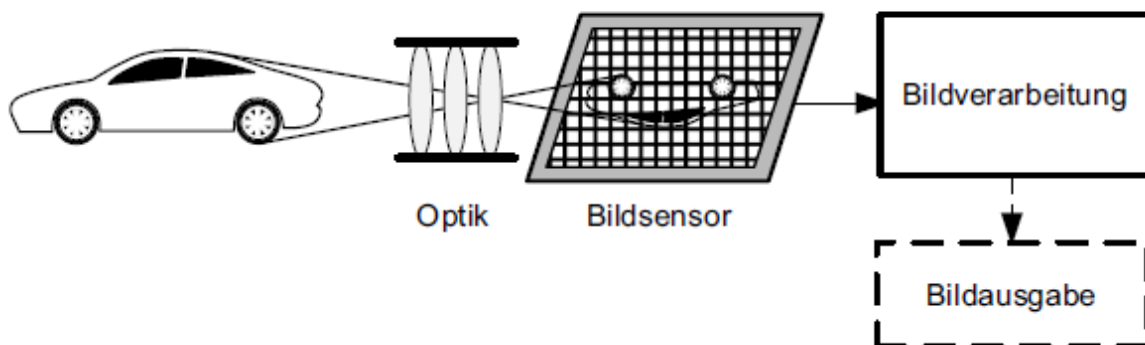


Abbildung 6: Funktionsweise einer Kamera [22, S. 352]

Mit einer Optik wird eine Szene auf den Bildsensor projiziert. Durch den fotoelektrischen Effekt werden Elektronen frei, die mit einer Ausleseelektronik verarbeitet werden. Die heutzutage am häufigsten verwendete Bildsensortechnik ist CMOS (Complementary Metal Oxide Semiconductor). Bei diesem System sind alle Bildpunkte in einer Matrix angeordnet und können einzeln von außen adressiert werden. Der Vorteil von CMOS-Chips ist die geringe Baugröße und eine hohe Bildwiederholrate [23, S. 337-338]. Danach wird das Rohbild weiterverarbeitet und ausgegeben. Die Anforderungen an Kamerasysteme sind zum einen eine hohe Auflösung ($> 15\text{Pixel/}^\circ$), um beispielsweise Verkehrsschilder optimal erkennen zu können. Zum anderen muss eine kurze Belichtungszeit von $< 30\text{ ms}$ gegeben sein, da sich Fahrzeuge teilweise mit hohen Geschwindigkeiten bewegen. Des Weiteren

spielt das Blickfeld der Kamera eine wichtige Rolle (FOV-Field of View). Dabei wird horizontales (HFOV) und vertikales (VFOV) Blickfeld unterschieden. Besonders wichtig ist ein großer Öffnungswinkel in horizontaler Richtung, da somit bei engen Kurvenradien die Fahrspurmarkierungen immer noch erkannt werden können oder einscherende Fahrzeuge rechtzeitig detektiert werden können [22, S. 353]. Ein Beispiel für ein modernes Kamerasystem ist die Multifunktionskamera von Bosch mit einem horizontalen Öffnungswinkel von $\pm 50^\circ$ und einem vertikalen Öffnungswinkel von $+27^\circ / -21^\circ$. Zudem ist eine Reichweite von ca. 150 m möglich [19].

2.2.2 Radar

Im Jahr 1998 wurde das erste Mal in Europa ein Radar in einem ACC-System verwendet. Dort ist es der primäre Sensor für die Abstandsregelung und Kollisionsvermeidung [23, S. 329]. Die Funktionsweise eines Radars basiert auf dem Prinzip, dass zunächst Wellenpakete ausgesendet werden. Diese werden an reflektierenden Oberflächen zurückgeworfen und können dann am Empfangsteil des Radars ausgewertet werden. Durch die Laufzeit der elektromagnetischen Wellen, kann der Abstand zu einem Objekt bestimmt werden und mit dem Dopplereffekt die Relativgeschwindigkeit [16, S. 133]. Im Straßenverkehr wird meistens das Frequenzband von 76-77 GHz angewendet. Daraus ergibt sich eine Wellenlänge von ca. 4 mm. Um das Reflektionsvermögen eines Gegenstands zu bewerten, wird der so genannte Radarquerschnitt σ (Radar Cross Section RCS) angegeben. Die Einheit von σ ist eine Fläche. Je größer diese Fläche, desto höher ist die Entdeckungswahrscheinlichkeit durch den Radar [22, S. 261-262]. Der Radarquerschnitt einer Metallplatte kann beispielsweise folgendermaßen nach [22, S. 261-262] berechnet werden.

$$\sigma_{\text{Platte}} = 4\pi \frac{A^2}{\lambda^2} \quad (2.1)$$

Der schematische Aufbau eines Radars ist in Abbildung 7 dargestellt. Im Hochfrequenzteil erzeugt ein Microcontroller die Signale für die Ansteuerung der Frequenzgenerierung. Die Frequenz wird mit einem Oszillator erzeugt und durch eine Antenne abgestrahlt. Die Empfangsantenne empfängt das reflektierte Signal und mischt es mit dem Sendesignal, um die Dopplerfrequenz zu bestimmen. Anschließend wird es verstärkt und eine Fourier Transformation (FFT Fast Fourier Transformation) durchgeführt. Die Auswertung erfolgt im HF-Microcontroller, der schließlich die Informationen an den NF-Teil weiterleitet. Der Niederfrequenzteil übernimmt die Kommunikation mit den anderen Fahrzeugkomponenten [23, S. 330].

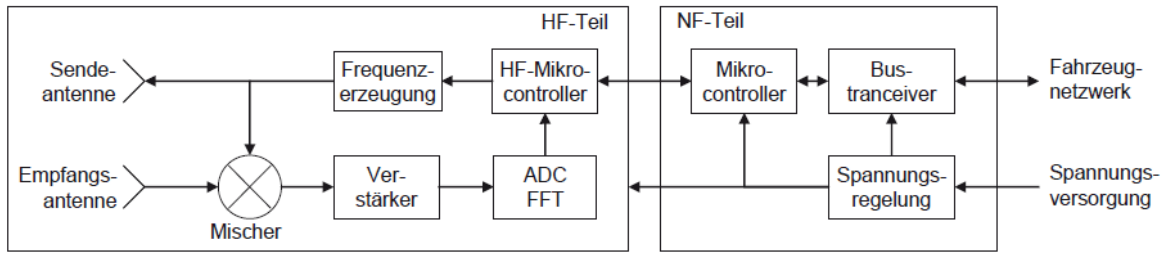


Abbildung 7: Schematischer Aufbau eines Radars [23, S. 330]

Wie bereits erwähnt, wird für die Abstandsbestimmung zu einem Objekt die Laufzeitmessung verwendet. Diese kann direkt oder indirekt erfolgen. Bei der direkten Messung wird die Pulsmodulation verwendet. Dabei werden kurze Wellenpakete mit konstanter Frequenz ausgesendet. Wenn die Laufzeit zwischen ausgesendetem und empfangenem Signal gemessen wurde, kann mit folgender Formel der Abstand berechnet werden [16, S. 133].

$$d = \frac{t_{diff} \cdot c}{2} \quad (2.2)$$

Bei der indirekten Laufzeitmessung kommt hingegen die Frequenzmodulation FMCW (Frequency Modulated Continuous Wave) zum Einsatz. Die Funktionsweise ist in Abbildung 8 zu sehen.

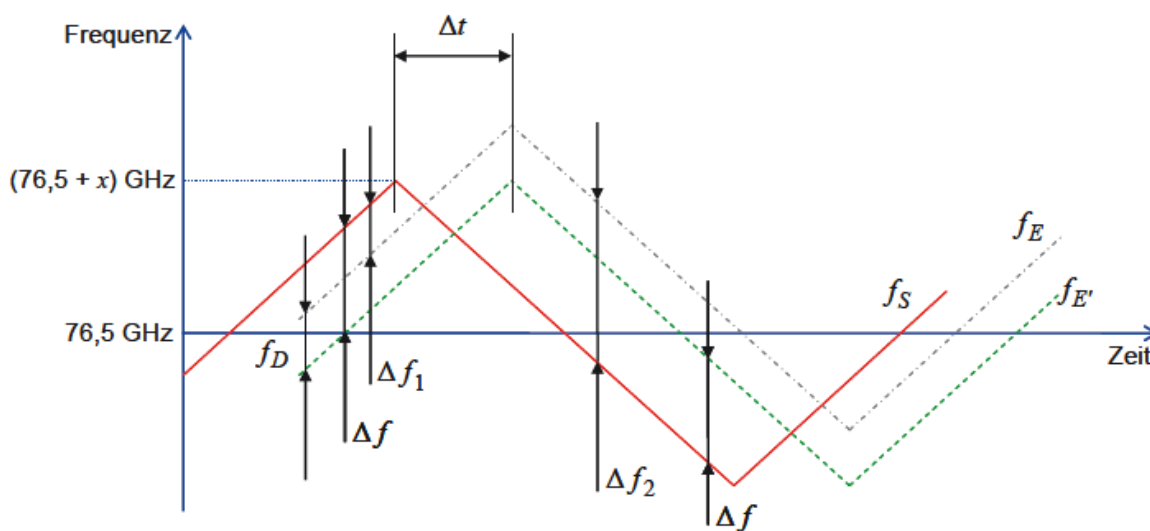


Abbildung 8: Funktionsweise der Frequenzmodulation FMCW [23, S. 331]

Die Sendefrequenz f_s ändert sich linear mit der Zeit. Die Frequenz f_E des Empfangssignals ist um die Laufzeit Δt verschoben. Die Frequenzdifferenz Δf kann dann als direktes Maß für den Abstand verwendet werden [23, S. 331]. Wenn zusätzlich noch eine Relativgeschwindigkeit besteht ($\rightarrow f_E$ Frequenz des Empfangssignals mit Relativgeschwindigkeit), müssen die beiden Frequenzdifferenzen Δf_1 und Δf_2 bestimmt werden. Wegen des Dopplereffekts ist die Empfangsfrequenz f_E jeweils um den Betrag f_D erhöht. Die Addition von Δf_1 und Δf_2 ist schließlich ein Maß für den Abstand zwischen Sender und Empfänger und die Differenz für die Dopplerfrequenz und somit die Relativgeschwindigkeit. Der Zusammenhang von Relativgeschwindigkeit und Dopplerfrequenz ist folgendermaßen gegeben [16, S. 134]:

$$f_D = -2f_c \cdot \frac{v_{rel}}{c} \quad (2.3)$$

f_c steht hier für die Trägerfrequenz des Sendesignals und c für die Lichtgeschwindigkeit.

2.2.3 LiDAR

LiDAR steht für Light Detection and Ranging. Er arbeitet ähnlich wie Radarsensoren, da auch elektromagnetische Wellen verwendet werden, jedoch mit einer höheren Frequenz. Der nutzbare Wellenlängenbereich liegt zwischen 800 und 1000 nm. Der Nachteil zum Radar ist die große Dämpfung der Lichtstrahlen bei Nebel und Gischt [16, S. 141]. Andererseits besitzen LiDAR-Sensoren eine höhere Auflösung und haben eine größere Reichweite [24]. Abbildung 9 zeigt das generelle Funktionsprinzip eines LiDARs.

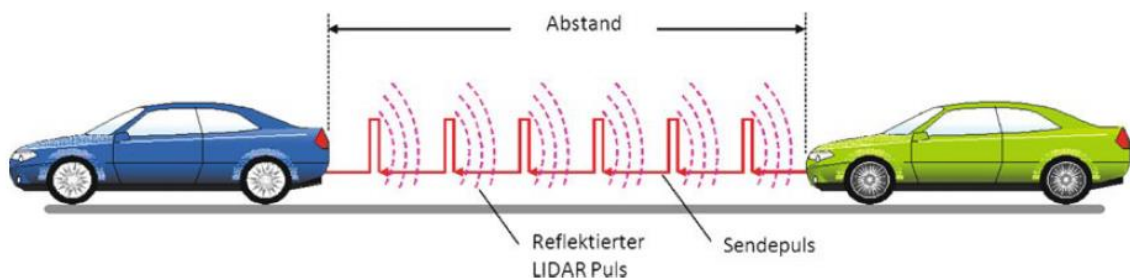


Abbildung 9: Funktionsprinzip eines LiDARs [22, S. 319]

Zuerst werden gepulste Lichtstrahlen ausgesendet. Diese werden an einem Objekt reflektiert und schließlich vom Empfänger ausgewertet. Die Laufzeit des Lichtsignals ist

dabei proportional zum Abstand des Objekts. Demnach kann auch mit Formel (2.2) die Entfernung zum vorausfahrenden Fahrzeug berechnet werden [22, S. 318]. Die Empfangs- und Sendelogik ist in Abbildung 10 dargestellt.

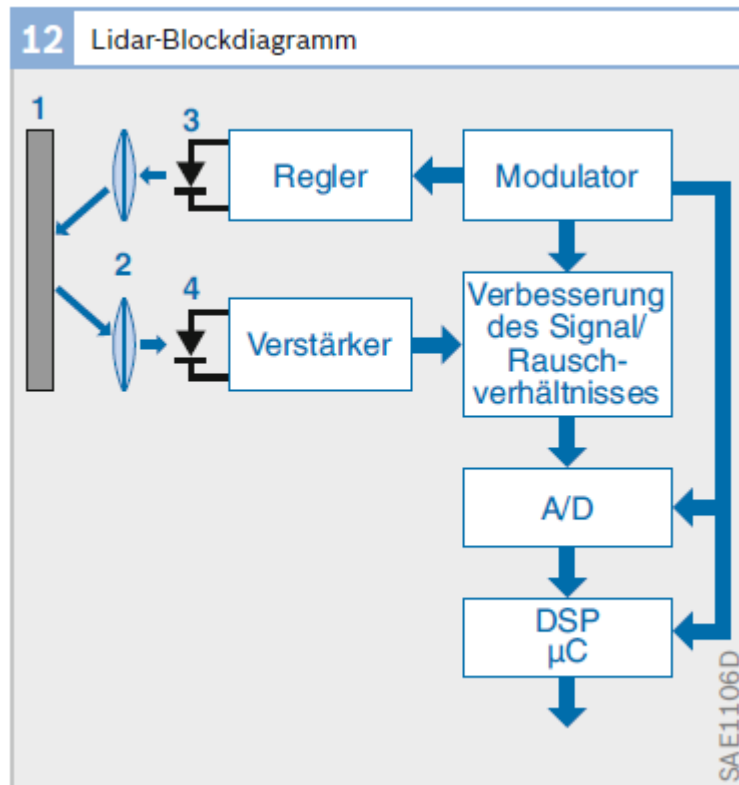


Abbildung 10: LiDAR-Blockdiagramm zur Sende- und Empfangslogik [16, S. 141]

Dem Bild ist zu entnehmen, dass ein Modulator das benötigte Signal erzeugt. Dieses wird dann von einer Diode emittiert. Häufigstes Modulationsverfahren ist in der Praxis die Pulsmodulation, da dadurch ein gutes Signal/Rauschverhältnis erreicht wird. Die Dauer der Pulse liegt typischerweise im Nanosekundenbereich. Damit liegen die Pulslängen im Bereich von einem Meter. Die reflektierten Lichtstrahlen werden von einer Empfängerdiode detektiert. Über einen Verstärker und einen A/D-Wandler erfolgt anschließend die Auswertung in einem DSP. Mit modernen Signalverarbeitungsverfahren sind Messgenauigkeiten im cm-Bereich möglich [16, S. 141]. Zusätzlich zur Abstandsmessung ist auch die Bestimmung der Relativgeschwindigkeit von anderen Fahrzeugen notwendig. Bei einem LiDAR kann dies theoretisch auch mithilfe der Dopplerfrequenz wie bei Radaren erfolgen. Jedoch ist das Messen der Dopplerfrequenz im Lichtspektrum zu teuer und zu aufwändig. Deshalb wird die Relativgeschwindigkeit über die Differentiation zweier aufeinanderfolgenden Abstandsmessungen ermittelt. Dies ist in Formel (2.4) dargestellt [22, S. 324-325].

$$\vec{v}_{rel} = \frac{d\vec{R}}{dt} = \lim_{\Delta t \rightarrow 0} \frac{\Delta \vec{R}}{\Delta t} \quad (2.4)$$

R steht hierbei für den Abstand.

Zuletzt wird noch auf die Detektion des Erfassungsbereichs vor dem Fahrzeug eingegangen. Abbildung 11 zeigt die verschiedenen Möglichkeiten.

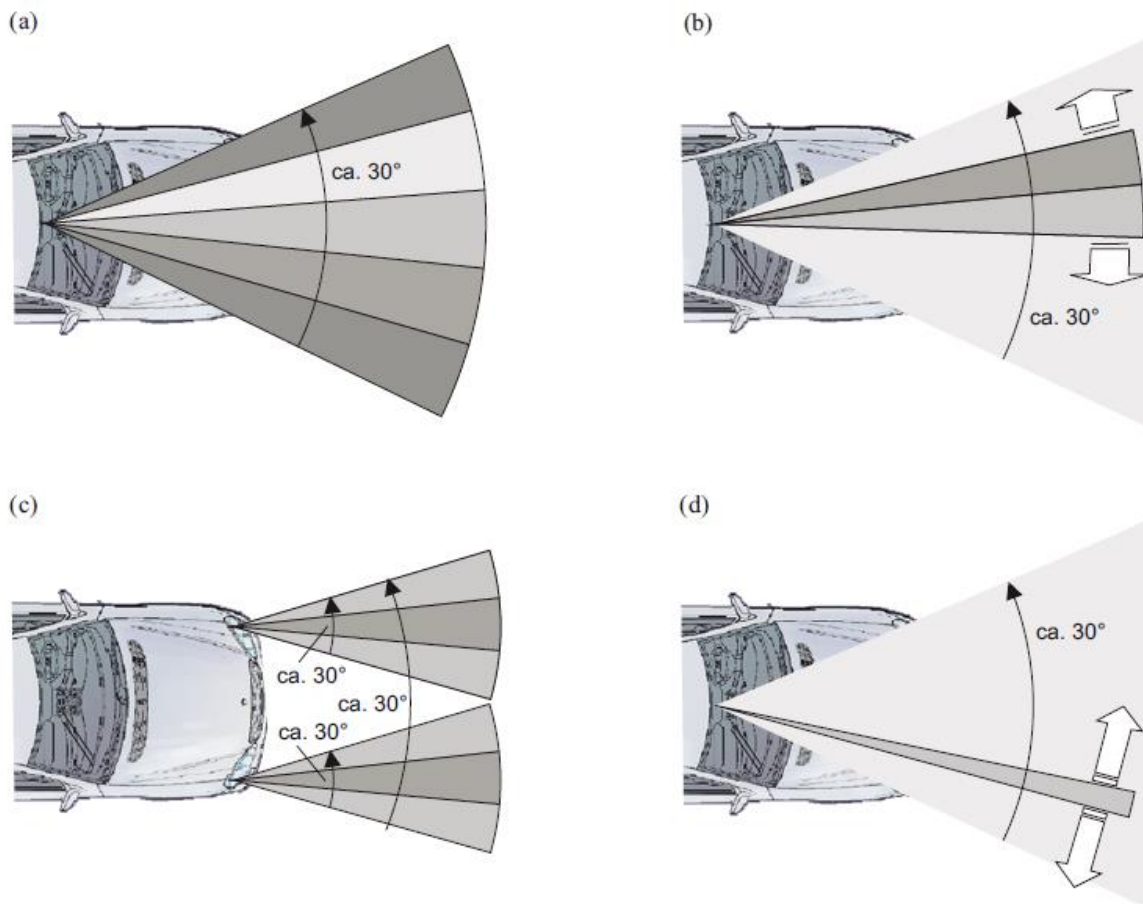


Abbildung 11: Detektion des Erfassungsbereichs vor dem Ego-Fahrzeug [23, S. 334]

Bei Variante a) sind mehrere Strahlen starr angeordnet. Bei b) werden die Strahlen hingegen langsam geschwenkt. Bei c) sind jeweils mehrere Strahlen verteilt und starr positioniert und bei der letzten Variante d) wird nur ein Strahl verwendet, der permanent schnell geschwenkt wird. Bei den starren Anordnungen wird immer nur die Blickrichtung des Ego-Fahrzeugs aufgenommen. Dies hat den Vorteil, dass für die Auswertung nur geringer Rechenaufwand benötigt wird. Allerdings werden Objekte, die sich von der Seite nähern erst spät erkannt. Bei den anderen Konfigurationen mit schwenkbarem Lichtstrahl wird ein größeres Gebiet erfasst. Deswegen muss hierbei mithilfe der Fahrtrajektorie erst

der relevante Bereich ausgemacht werden. Insofern ist hierbei mehr Rechenaufwand nötig, es werden aber alle Objekte erfasst [23, S. 334-335].

2.3 Kinematik

Die Vorwärtskinematik stellt eine wichtige Grundlage für die folgenden Kapitel dar. Es geht darum die Pose eines Roboters inkrementell zu ermitteln. Das heißt beginnend bei einer Startpose, wird jeweils die Änderung in diskreten Zeitschritten bestimmt und integriert. Wird die Trajektorie eines bewegten Objekts durch Messung der Orientierung und Geschwindigkeit über der Zeit durchgeführt, spricht man auch von Koppelnavigation. Wird ferner die Geschwindigkeit über die Raddrehzahlen berechnet, ist von Odometrie die Rede [25, S. 110-111]. Die Bestimmung der Bewegung eines mobilen Systems ist immer von der verwendeten Antriebs- bzw. Lenkungstechnik abhängig. Bei allen realen Straßenfahrzeugen wird in der Regel ein Antriebssystem mit einer oder zwei lenkbaren Achsen verwendet. Der allgemeine Fall mit zwei Achsen ist in Abbildung 12 zu sehen.

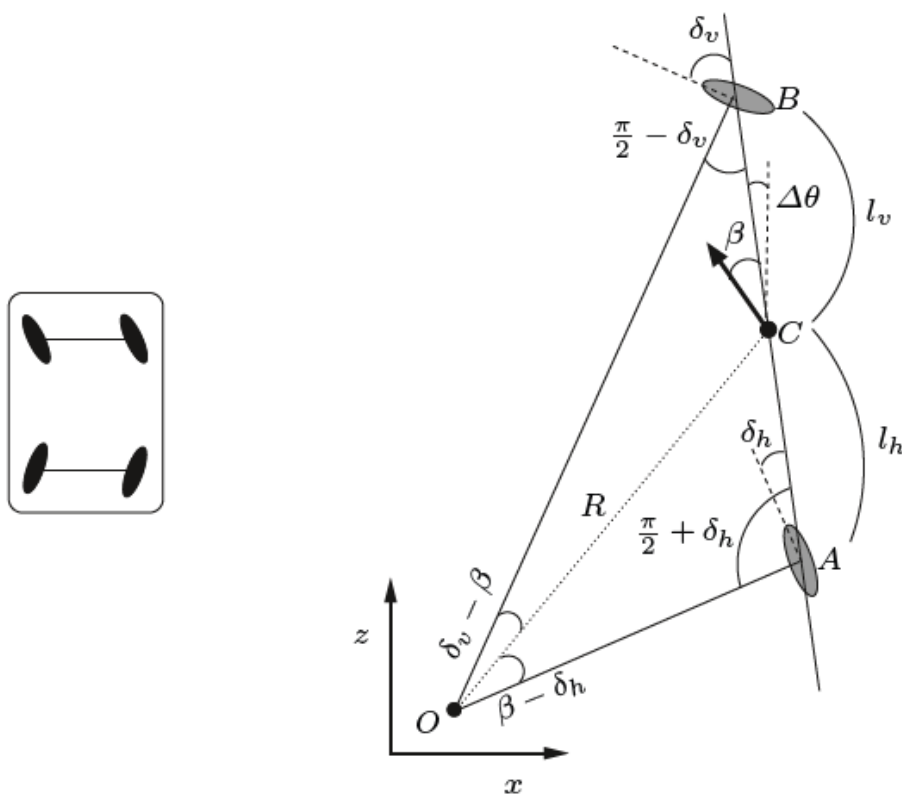


Abbildung 12: Darstellung eines Systems mit zwei lenkbaren Achsen [25, S. 116]

Das dargestellte Modell zeigt die Reduktion auf eine Mittelachse, bei der die beiden vorderen und hinteren Räder zu jeweils einem Rad zusammengefügt wurden [25, S. 114]. Die Geschwindigkeit des Gesamtfahrzeugs lässt sich nach [25, S. 117] folgendermaßen berechnen:

$$v = \frac{v_v \cos \delta_v + v_h \cos \delta_h}{2 \cos \beta} \quad (2.5)$$

Die Formelzeichen v_v und v_h stehen für die Geschwindigkeit am vorderen bzw. hinteren Rad. Außerdem bezeichnen δ_v und δ_h die Winkelstellungen der jeweiligen Räder. Der Schwimmwinkel β gibt den Winkel zwischen Längsachse und tatsächlicher Bewegungsrichtung an. Des Weiteren lassen sich die Geschwindigkeiten an den Rädern über Messung der Raddrehzahlen mittels Inkrementalgebern bestimmen. Diese liefern die Impulse („Ticks“) pro Umdrehung. Ist der Radumfang $\Omega = 2r\pi$ bekannt, kann mit der Gesamtanzahl an Impulsen I und den Impulsen pro Radumdrehung θ die zurückgelegte Strecke d berechnet werden [25, S. 114]:

$$d = \frac{I \cdot \Omega}{\theta} \quad (2.6)$$

Wird die Strecke d nun nach der Zeit abgeleitet, erhält man die Geschwindigkeit für jedes Rad. Der genannte Parameter β , der auch für die Geschwindigkeitsberechnung benötigt wird, lässt sich nach Formel (2.7) berechnen [25, S. 117]:

$$\beta = \tan^{-1} \left(\frac{l_v \tan \delta_h + l_h \tan \delta_v}{l_v + l_h} \right) \quad (2.7)$$

Die Angaben l_v und l_h beziehen sich auf die Abstände zwischen Vorderrad B bzw. Hinterrad A und Mittelpunkt C der Achse (siehe Abbildung 12). Somit ergibt sich der Gesamtabstand zwischen Vorder- und Hinterachse zu $L = l_v + l_h$. Abschließend wird noch der Parameter $\Delta\theta$ benötigt, der die Winkelgeschwindigkeit relativ zur z-Achse (auch Gierrate genannt) beschreibt. Damit wird also die Orientierung des Roboters angegeben. Die Gierrate wird schließlich folgendermaßen berechnet [25, S. 116]:

$$\Delta\theta = \frac{v \cos \beta}{l_v + l_h} (\tan \delta_v - \tan \delta_h) \quad (2.8)$$

Mit den ganzen präsentierten Gleichungen lässt sich final das Odometrie-Bewegungsmodell darstellen [25, S. 117]:

$$P_n = \begin{pmatrix} x_n \\ z_n \\ \theta_n \end{pmatrix} = \begin{pmatrix} x_{n-1} \\ z_{n-1} \\ \theta_{n-1} \end{pmatrix} + \begin{pmatrix} v \cdot \Delta t \cdot \sin(\theta_{n-1} + \beta + \Delta\theta \Delta t) \\ v \cdot \Delta t \cdot \cos(\theta_{n-1} + \beta + \Delta\theta \Delta t) \\ \Delta\theta \cdot \Delta t \end{pmatrix} \quad (2.9)$$

Die Pose des Roboters wird in Gleichung (2.9) durch die Position in x- und z-Richtung und den Gierwinkel ausgedrückt. Es wird das Koordinatensystem aus Abbildung 12 zugrunde gelegt. Die Berechnung der Pose zum Schritt n erfolgt dann mit der Pose zum vorherigen Zeitschritt $n - 1$ und der relativen Änderung dazu. Um die relative Änderung pro Zeiteinheit zu bestimmen, wird die Geschwindigkeit v über die Zeit Δt in x- und z-Richtung integriert. Darüber hinaus wird ebenfalls die Gierrate $\Delta\theta$ integriert. Folglich kann iterativ mit diesem Bewegungsmodell immer der neue Zustand des Roboters bestimmt werden.

Ein Spezialfall des Lenkungssystems nach Abbildung 12 ist die Ackermann-Lenkung. Hierbei ist die Hinterachse fest und nur die Vorderachse lenkbar. Diese Technik kommt heutzutage in den meisten PKW zum Einsatz. Abbildung 13 zeigt den schematischen Aufbau.

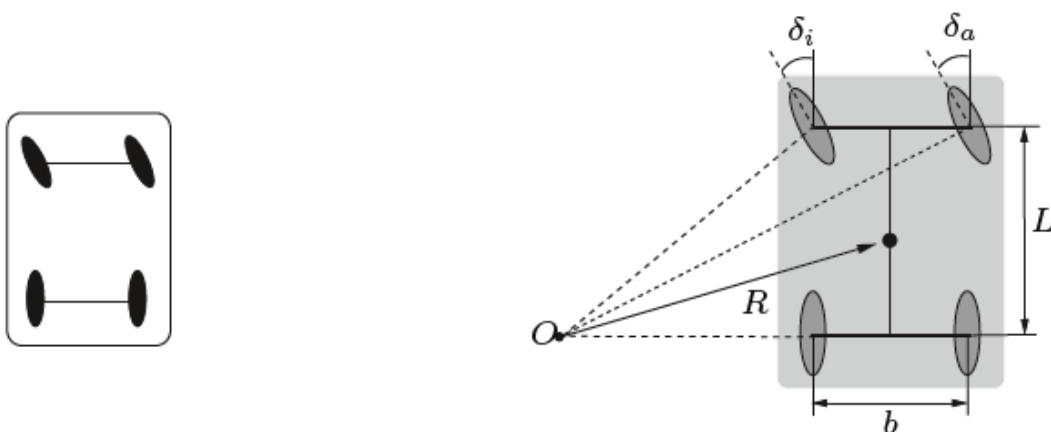


Abbildung 13: Darstellung der Ackermann-Lenkung [25, S. 118]

Der entscheidende Unterschied bei der Ackermann-Lenkung ist, dass die Hinterachse nicht lenkbar ist. Dadurch ergibt sich der hintere Radwinkel zu $\delta_h = 0$. Der innere und äußere Radwinkel der lenkbaren Vorderachse ist nach Gleichung (2.10) bestimmbar [25, S. 118]:

$$\delta_a = \frac{L}{R + \frac{b}{2}} \quad \delta_i = \frac{L}{R - \frac{b}{2}} \quad (2.10)$$

Daraus ist ersichtlich, dass der Winkel des inneren Rads bei einem Lenkmanöver immer einen größeren Winkel besitzt. L bezeichnet dabei den Abstand der beiden Achsen und b den Abstand der Räder auf einer Achse. Außerdem bezieht sich R auf die Strecke vom Mittelpunkt der Mittelachse zum Drehpunkt O . Darüber hinaus gibt es noch einen Zusammenhang zwischen innerem und äußerem Radwinkel [26, S. 284]:

$$\cot \delta_a = \cot \delta_i + \frac{b}{L} \quad (2.11)$$

Der daraus berechenbare Differenzwinkel zwischen δ_i und δ_a wird auch Spurdifferenzwinkel genannt.

Abschließend bleibt noch anzumerken, dass das Bewegungsmodell nach Gleichung (2.9) auch auf Systeme mit Ackermann-Lenkung anwendbar ist, da dieses Modell vom allgemeinen Fall mit zwei lenkbaren Achsen abgeleitet wurde. Der einzige Unterschied ist, dass der hintere Radwinkel δ_h null gesetzt wird. Damit ergeben sich schließlich andere Werte für Gierrate, Schwimmwinkel und Gesamtgeschwindigkeit. Mit diesen Werten kann aber die Pose P_n nach Gleichung (2.9) auch berechnet werden [25, S. 117-118].

Dieses Kapitel hat nun gezeigt, dass die Pose eines Roboters nur über die Odometrie bestimmbar ist. Das Problem dabei ist, dass Geschwindigkeiten über der Zeit integriert werden. Damit wird auch das Rauschen aufsummiert. Daraus ergibt sich die Tatsache, dass bereits nach wenigen Zeitschritten die Bestimmung der Roboterpose stark verfälscht wird. Der folgende Abschnitt beschäftigt sich genau mit dieser Thematik, wie die Lokalisierung verbessert werden kann.

2.4 Partikel-Filter

Im Bereich der Robotik ist es von großer Bedeutung die internen Zustände eines dynamischen Systems zu schätzen. Der interne Zustand kann zum Beispiel die Pose, also die Position und die Orientierung, eines Roboters sein. Um diese Zustände zu schätzen, werden Partikel-Filter eingesetzt. Ein anderer weit verbreiteter Name dafür ist auch Monte Carlo Filter. Der Vorteil ist, dass dieses Modell auf Systeme angewendet werden kann, die weder normalverteilte Zustandsgrößen, noch Rauschgrößen mit Normalverteilung besitzen [27, S. 1-2]. Ein nichtlineares, nicht Gauss'sches Zustandsmodell kann folgendermaßen nach [27, S. 2] formuliert werden:

$$x_t = F(x_{t-1}, v_t) \quad (2.12)$$

$$y_t = H(x_t, w_t) \quad (2.13)$$

Gleichung (2.12) wird als Zustandsgleichung des Systems bezeichnet. x_t steht für einen k -dimensionalen Zustandsvektor zum Zeitpunkt t . Dieser wird iterativ aus dem vorherigen Zustand x_{t-1} und dem Systemrauschen v_t ermittelt. F steht dabei für eine nichtlineare Funktion. Die zweite Gleichung (2.13) stellt die Beobachtungs- oder Messgleichung dar. Sie beschreibt die Messungen y_t , die zum Zeitpunkt t gemacht werden. w_t steht für das Messrauschen und H ebenfalls für eine nichtlineare Funktion. Ziel ist es die Dichtefunktion von x_t unter Einbeziehung der Beobachtungen $Y_t = \{y_1, \dots, y_N\}$ zu bestimmen. Dies entspricht der bedingten Dichte $p(x_t|Y_t)$ [27, S. 2-3]. Die Umsetzung erfolgt über zwei Schritte. Als erstes wird in der Prädiktionsphase der aktuelle Zustand des Systems in Form der Verteilungsfunktion $p(x_t|Y_{t-1})$ vorhergesagt. Dann wird in der Updatephase mithilfe von neuen Informationen, also den durchgeführten Messungen, die Dichtefunktion des Zustandsvektors aktualisiert. Dies entspricht der posteriori PDF (Probability Distribution Function) $p(x_t|Y_t)$. Die Verteilungsdichtefunktionen werden jeweils durch eine Anzahl von N gewichteten Partikeln angenähert [28, S. 1032-1034]. Um das Vorgehen näher zu beleuchten, wird im nächsten Kapitel auf ein konkretes Anwendungsbeispiel eines Partikelfilters eingegangen.

2.5 Adaptive Monte Carlo Lokalisierung (AMCL)

Die Adaptive Monte Carlo Lokalisierung ist ein Verfahren, das einen mobilen Roboter auf einer bekannten Karte lokalisieren kann, ohne den Startpunkt zu kennen. Grundlage dafür bildet ein Partikel-Filter, der den Systemzustand schätzt. Da in diesem Anwendungsfall die Pose eines autonomen Systems geschätzt wird, wird der interne Zustand durch den Vektor $x_t = [x, y, \theta]^T$ abgebildet. Die Parameter x und y geben die Koordinaten auf einer 2D-Karte an und θ bezieht sich auf den Gierwinkel des Roboters. Ziel ist es nun, wie bereits im letzten Kapitel beschrieben, die posteriori Dichteverteilung $p(x_t|Y_t)$ des Zustands zum Zeitpunkt t , unter der Berücksichtigung externer Messungen Y_t , zu bestimmen. Die Messungen können beispielsweise von einem LiDAR generiert werden, der Abstände zu Hindernissen misst. Um die Dichteverteilung zu erhalten, wird im ersten Schritt, der Prädiktionsphase, die aktuelle Position des Roboters nur unter Verwendung des Bewegungsmodells prädiziert. Das bedeutet, die Position ausgedrückt durch die priori Dichteverteilung $p(x_t|Y_{t-1})$, wird mithilfe der Odometrie bestimmt. Die Odometrie liefert eine Geschwindigkeit und einen Gierwinkel. Diese Werte werden als Control Input u_{t-1} bezeichnet. Mit diesen Daten und dem vorherigen Systemzustand x_{t-1} wird schließlich die Dichtefunktion des aktuellen Zustands berechnet. Das Bewegungsmodell lässt sich durch die bedingte Dichte $p(x_t|x_{t-1}, u_{t-1})$ ausdrücken. Die prädizierte Wahrscheinlichkeitsdichte des Zustands x_t lässt sich dann über Integration ermitteln [29, S. 1-2].

$$p(x_t|Y_{t-1}) = \int p(x_t|x_{t-1}, u_{t-1}) p(x_{t-1}|Y_{t-1}) dx_{t-1} \quad (2.14)$$

Des Weiteren werden Partikel verwendet, um die prädizierte Dichte $p(x_t|Y_{t-1})$ anzunähern. Dafür werden die Partikel S_{t-1} , die im vorherigen Schritt berechnet wurden als Ausgangspunkt verwendet. Initial wird eine zufällige Partikelmenge S_0 gewählt. Auf jedes dieser Partikel s_{t-1}^i wird das Bewegungsmodell angewendet, indem von der Dichte $p(x_t|s_{t-1}^i, u_{t-1})$ Samples gezogen werden. Dadurch entsteht die neue Partikelmenge S'_t , die die priori Dichte $p(x_t|Y_{t-1})$ annähert [29, S. 3]. Es ist zu beachten, dass zu diesem Zeitpunkt noch keine Sensormessungen in das Modell eingeflossen sind. Da die Verwendung des Bewegungsmodells allein zu ungenau wäre, müssen externe Beobachtungen miteinbezogen werden. Durchdrehende Räder oder die Integration von verrauschter Geschwindigkeit verfälschen beispielsweise den prädizierten Zustand. Um das Modell zu verbessern, werden nun in der Updatephase Messungen herangezogen, um schließlich die posteriori Dichtefunktion $p(x_t|Y_t)$ des internen Zustandsvektors x_t zu schätzen. Dies ist eine

bedingte Wahrscheinlichkeit. Das heißt gesucht ist die Wahrscheinlichkeit für eine bestimmte Pose unter der Bedingung der Messung Y_t . Diese Dichte kann mit dem Bayes-Theorem folgendermaßen berechnet werden.

$$p(x_t|Y_t) = \frac{p(y_t|x_t)p(x_t|Y_{t-1})}{p(y_t|Y_{t-1})} \quad (2.15)$$

Die Annäherung der posteriori Dichte erfolgt wieder über Partikel. Zu jedem Element der zuvor bestimmten Partikelmenge S'_t wird ein Gewicht w_t^i ermittelt. Das Gewicht lässt sich folgendermaßen berechnen [29, S. 3]:

$$w_t^i = p(y_t|s_t'^i) \quad (2.16)$$

Das bedeutet ein Gewicht ist die Wahrscheinlichkeit für eine Messung y_t unter der Bedingung eines gegebenen Partikels $s_t'^i$. Liefert eine Messung also einen unplausiblen Wert für ein bestimmtes Partikel im Raum, dann erhält es ein geringeres Gewicht. Ist ein Partikel beispielsweise in einer Türöffnung platziert, die LiDAR-Messung liefert aber hingegen Abstandspunkte einer Wand, so ist die Wahrscheinlichkeit, dass dieses Partikel die korrekte Pose des Roboters widerspiegelt eher gering. Schließlich erhält man die Partikelmenge S_t , indem von der gewichteten Menge $\{s_t'^i, w_t^i\}$ Samples gezogen werden. Dabei werden Samples mit einem hohen Gewicht mit einer größeren Wahrscheinlichkeit gezogen. Die neue Partikelmenge S_t ist letztendlich eine Annäherung für die posteriori Wahrscheinlichkeitsdichte $p(x_t|Y_t)$ des Systemzustands, also der Pose des Roboters [29, S. 3].

Um den Prozess der Prädiktions- und Updatephase noch einmal zu verdeutlichen, ist in Abbildung 14 dieser Vorgang graphisch dargestellt. Die obere Zeile zeigt immer die genaue Wahrscheinlichkeitsdichte und die untere Zeile die durch die Partikel angenäherte Dichte. Bei Schritt A wird mit einer Partikelwolke S_{t-1} begonnen, die die Unsicherheit der Roboterposition beschreibt. Als nächstes wird der Roboter in Bild B genau einen Meter bewegt. Deswegen muss sich das System im Radius von einem Meter aufhalten. Dies entspricht der Prädiktionsphase des Partikelfilters. Die Partikelmenge S'_t wurde also basierend auf den Odometriedaten (Control Input) berechnet. Als nächstes wird in Schritt C durch eine Sensormessung eine Orientierungsmarke einen halben Meter entfernt in der rechten oberen Ecke gemessen. Im oberen Bild ist die bedingte Wahrscheinlichkeit $p(y_t|x_t)$

für die Messung gegeben. Basierend darauf werden pro Partikel die Gewichte berechnet. Folglich ergibt sich die gewichtete Partikelmenge S'_t , gezeigt durch das untere Bild. Darin ist zu sehen, dass die Partikel rechts oben eine höhere Wahrscheinlichkeit besitzen als der Rest, da zu diesen Partikeln die gemessenen Punkte am besten passen. Im letzten Schritt D werden schließlich aus der alten Partikelmenge S'_t die Partikel gezogen, die die größten Gewichte besitzen. Dies entspricht der Updatephase. Somit entsteht die neue Partikelmenge S_t . Diese repräsentiert nun die Dichte für die Position des Roboters unter Berücksichtigung einer externen Observation. Im weiteren Verlauf würden diese Schritte nun iterativ wiederholt werden [29, S. 3-4].

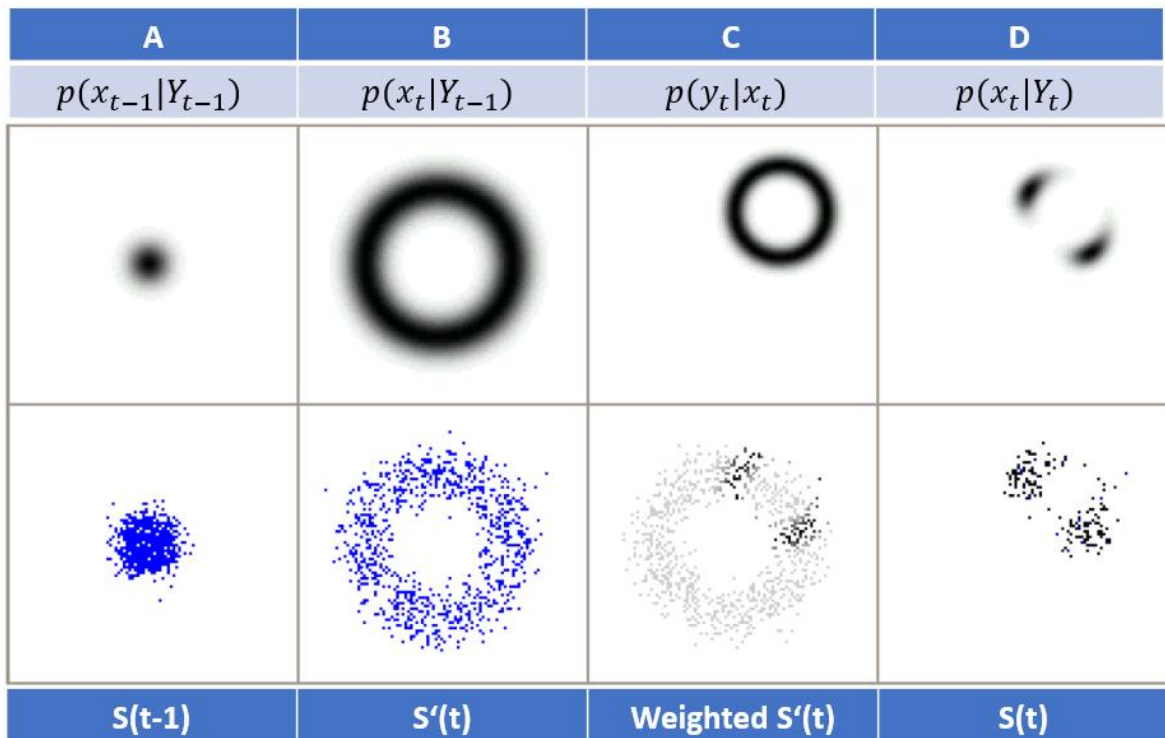
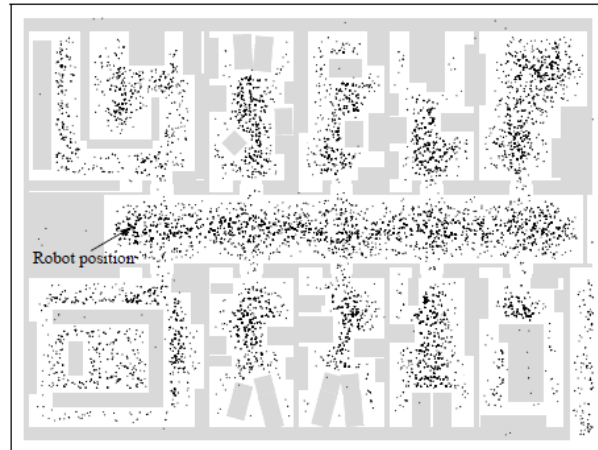


Abbildung 14: Darstellung der Prädiktions- und Updatephase nach [29, S. 4]

Abschließend sind in Abbildung 15 mehrere Iterationen einer Lokalisierung dargestellt. Zu Beginn bei a) wird mit 20 000 Partikeln gestartet, die gleichmäßig über den Raum verteilt sind. Dem Roboter ist seine Umgebung in Form einer Karte bekannt, weswegen die Partikel nur an Positionen gesetzt werden, an denen sich keine Hindernisse befinden. Nach der ersten Iteration sind zwei Positionen am wahrscheinlichsten für den Roboter. Dies ist am mittleren Bild der Abbildung 15 zu sehen. Hier konzentrieren sich die Partikel vor allem an zwei Stellen im Raum. Dabei ist die wahre Position des Roboters bereits enthalten. Die Stelle auf der rechten Seite wird auch in Betracht gezogen, weil der Korridor symmetrisch aussieht. Wenige Iterationen später in c) kann der Roboter seine Position endgültig bestimmen. Die Wahrscheinlichkeitsdichte der Pose beschränkt sich auf einen engen

Raum. Das heißt alle Partikel haben sich mit einer kleinen Streuung am gleichen Ort positioniert.

a)



b)



c)

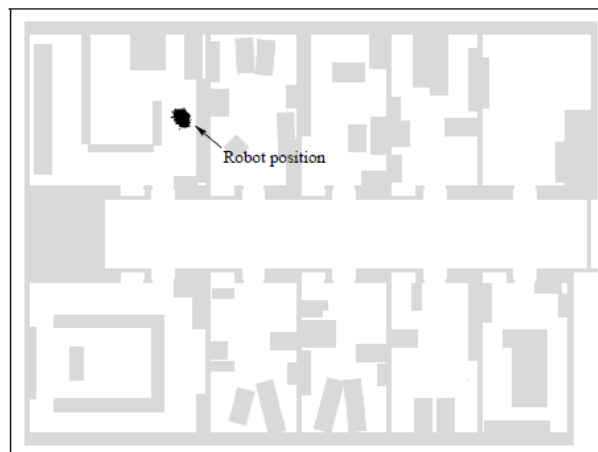


Abbildung 15: Darstellung mehrerer Iterationen der Roboterlokalisierung [29, S. 5]

2.6 Simultaneous Localization and Mapping (SLAM)

SLAM steht für Simultaneous Localization and Mapping. Es geht dabei um ein Verfahren in der Robotik, bei dem das autonome System eine Karte seiner Umgebung erstellt und gleichzeitig seine Position in dieser Karte schätzt. Dieses Szenario findet häufig Anwendung in Situationen, in denen die Umgebung nicht bekannt ist und kein GPS zur Positionsbestimmung verfügbar ist. Dies ist vor allem innerhalb von Gebäuden der Fall. Eine alleinige Navigation mittels Odometriedaten wäre bereits nach kurzer Zeit zu ungenau, da das Sensorrauschen sich über die Zeit addiert. Werden jedoch Odometriedaten mit optischen Sensoren wie LiDAR oder Kamera und einer Karte kombiniert, ist eine relativ genaue Lokalisierung möglich [30, S. 1309]. In der Praxis wird ein SLAM-Algorithmus häufig mit einem Rao-Blackwellisiertem Partikel-Filter implementiert. Der Gedanke dahinter ist zunächst ähnlich wie bei dem im letzten Kapitel behandelten Filter zur Positionsbestimmung (AMCL). Die Partikel sollen hier ebenfalls die Wahrscheinlichkeitsdichte des Roboterzustands, gegeben durch Position und Orientierung, annähern. Jedoch werden die Partikel jetzt noch durch die Schätzung einer Karte erweitert. Das heißt jedes Partikel besitzt die Informationen einer eigenen Karte und ein Gewicht, das beschreibt, wie gut ein Partikel den realen Zustand unter Einbeziehung der Mess- und Odometriedaten wiedergibt [31, S. 1-2]. Mathematisch ausgedrückt geht es also darum die posteriori PDF $p(x_t, m|y_t, u_{t-1})$ des Roboterzustands x_t und der Karte m unter Berücksichtigung der Messpunkte y_t eines LiDARs und der Odometriewerte u_{t-1} im vorherigen Zeitschritt zu erhalten. Der Ausdruck der bedingten Wahrscheinlichkeit kann dabei noch folgendermaßen zerlegt werden [32, S. 34-35]:

$$p(x_t, m|y_t, u_{t-1}) = p(m|x_t, y_t) \cdot p(x_t|y_t, u_{t-1}) \quad (2.17)$$

Gleichung (2.17) zeigt die Zerlegung des Problems in zwei Schritte. Zuerst wird nur die Pose des Roboters x_t mit den Messwerten y_t und den Odometriewerten u_{t-1} geschätzt. Dies entspricht der gleichen Vorgehensweise wie bei dem AMCL-Algorithmus. Im zweiten Schritt wird dann die Karte m mithilfe der Pose und den gegebenen Messungen berechnet. Somit entsteht am Schluss eine simultane Lokalisierung und Kartenerstellung [32, S. 35].

Das schrittweise Vorgehen des SLAM-Algorithmus mit Rao-Blackwellisiertem Partikel-Filter ist in Abbildung 16 dargestellt.

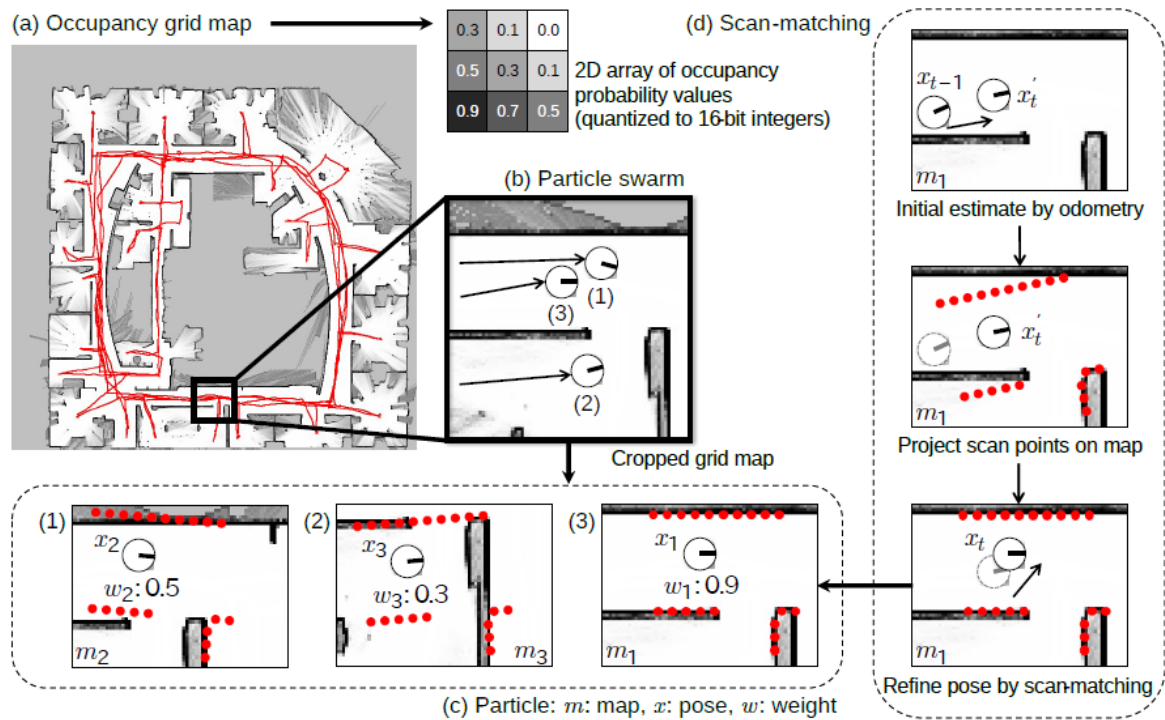


Abbildung 16: Ablauf des SLAM-Algorithmus mit Rao-Blackwellisiertem Partikel-Filter [31, S. 2]

Als erstes wird in Abbildung 16 d) mit einem Bewegungsmodell die Pose initial geschätzt. Dies entspricht der Prädiktionsphase des Partikelfilters. Als nächstes werden den Partikeln basierend auf den Messpunkten eines LiDARs Gewichte zugeordnet. Zum Schluss wird ein Resampling der Partikel proportional zu ihrem zugewiesenen Gewicht durchgeführt. Dies entspricht schließlich der Updatephase. Der beschriebene Vorgang wird auch Scan-Matching genannt und entspricht der Lokalisierung nach AMCL, also der Schätzung der Pose durch $p(x_t|y_t, u_{t-1})$, des Roboters [32, S. 35]. Die Lokalisierung wird dabei immer auf Basis der bereits teilerstellten Karte durchgeführt. Des Weiteren wird die Karte durch eine so genannte Occupancy Grid Map (siehe Abbildung 16 a)) repräsentiert. Dies ist eine zweidimensionale Matrix, welche die Umwelt repräsentiert, indem pro Zelle die Wahrscheinlichkeit für ein Hindernis abgespeichert wird [31, S. 3]. Nachdem die Pose geschätzt wurde, wird die Dichte $p(m|x_t, y_t)$ der Karte mithilfe der Pose und den LiDAR-Messungen ermittelt. Dies ist schematisch in Abbildung 16 c) zu sehen. Dort sind beispielhaft drei Partikel mit unterschiedlichen Posen und Karten dargestellt. Wird die Pose mit den roten Messpunkten kombiniert, fällt auf, dass das Partikel mit Zustand x_1 und Karte m_1 die Realität am besten approximiert. Die Messwerte passen nicht zu den Karten und Posen der anderen Partikel. Folglich besitzt das Partikel (3) mit $w_1 = 0.9$ das höchste Gewicht und wird im weiteren Verlauf stärker berücksichtigt [32, S. 35].

2.7 Globaler Pfadplaner

Ist eine Karte der Umgebung vorhanden, wird ein Pfadplaner benötigt, um den kürzesten Weg von einem Startpunkt zum Ziel zu finden. Der Suchraum wird durch den Graphen $G(V, E)$, der die Kanten E und Knoten V besitzt, ausgedrückt. Diese Art der Graphsuche wird beispielsweise auch von jedem Navigationsgerät im Fahrzeug durchgeführt. Zwei prominente Vertreter für Suchalgorithmen sind der Dijkstra- und der A*-Algorithmus. Beide sind immer optimal. Das bedeutet, sie finden in jedem Fall den kürzesten Weg vom Start zum Ziel. Beide werden im Folgenden näher vorgestellt [25, S. 272].

2.7.1 Dijkstra-Algorithmus

Initial muss ein Graph $G(V, E)$ mit seinen Knoten und Kanten gegeben sein. Für jede Kante wird ein Gewicht durch die Kostenfunktion c festgelegt. Die Funktion d drückt die Distanz eines Knotens zum Startknoten s aus. Ferner steht u für den aktuell besuchten Knoten. Der Algorithmus ist in Abbildung 17 basierend auf [33, S. 198] dargestellt.

Dijkstra-Algorithmus

Input: s (start node)

Output: $d(v_{goal})$ (distance from s to v_{goal})

Parameters: $G(V, E), v_{goal}$

mark s as unvisited and set $d(s) = 0$

mark other nodes v as unvisited with $d(v) = \infty$

$u = s$ (set start as current node)

while v_{goal} is unvisited:

foreach edge $e = (u, v) \in E$ and v is unvisited:

if $d(u) + c(e) < d(v)$:

$d(v) = d(u) + c(e)$

 mark u as visited

$u = \text{unvisited } v \in V \text{ with minimum } d(v)$

Abbildung 17: Ablauf des Dijkstra-Algorithmus nach [33, S. 198]

Um den Ablauf besser erklären zu können, ist in Abbildung 18 ein beispielhafter Graph mit den Knotendistanzen nach Beenden des Algorithmus dargestellt.

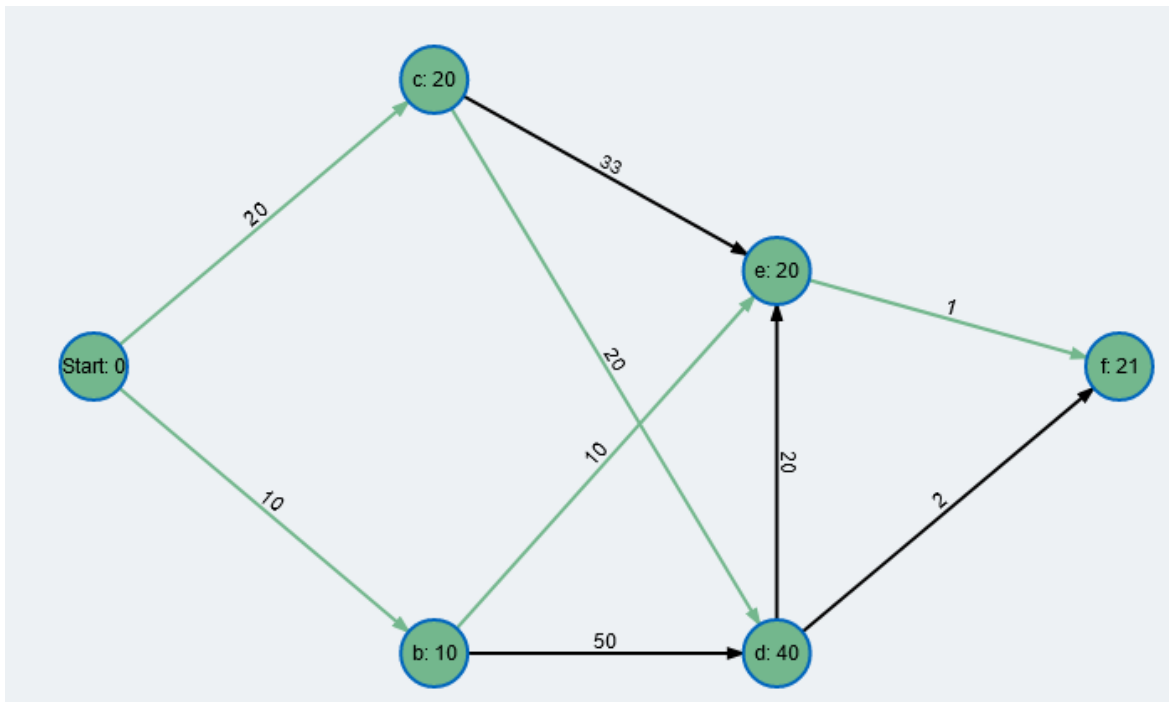


Abbildung 18: Beispielhafter Graph zur Anwendung des Dijkstra-Algorithmus [34]

Zu Beginn wird der Startknoten s festgelegt (siehe Abbildung 18, Knoten a). Dieser wird als unbesucht markiert und mit der Distanz $d(s) = 0$ initialisiert. Alle anderen Knoten werden ebenfalls als unbesucht markiert und bekommen die Distanz $d(v) = \infty$ zugewiesen. Der Algorithmus läuft nun so lange bis der Zielknoten f erreicht ist. Dafür werden alle Kanten des aktuellen Knotens betrachtet, die zu einem noch unbesuchten Knoten führen. Für den Startknoten a sind das die Nachbarknoten b und c . Falls die in einem Nachbarknoten gespeicherte Distanz $d(v)$ größer ist als die Summe aus Kantengewicht $c(e)$ und Distanz des aktuellen Knotens $d(u)$, dann erhält der Knoten v die neue Distanz $d(v) = d(u) + c(e)$. Dadurch hat Knoten c die neue Distanz 20 erhalten und Knoten b die Distanz 10. Sind alle unbesuchten Nachbarknoten des aktuellen Knotens u abgearbeitet, wird der Knoten u als besucht markiert. Danach wird der Knoten mit kleinster Distanz $d(v)$ zum Startpunkt als nächster aktueller Knoten u ausgewählt. Im gezeigten Graph nach Abbildung 18 ist dies der Knoten b , da seine Distanz mit 10 geringer ist als die des Knotens c . Ist schließlich der Zielknoten f mit der Distanz $d(v_f) = 21$ erreicht, bricht der Algorithmus ab. Der kürzeste Pfad kann dann über Rückwärtsrechnung bestimmt werden. Das heißt, für die in den Knoten gespeicherten Distanzen gibt es immer nur eine passende Kante, die zur Distanz eines vorherigen Knotens führt. Beginnend beim Zielknoten f führt nur die Kante (f, e) zu einem korrekten Knotendistanzwert, nämlich dem Knoten e mit $d(v_e) = 20$. Folgt man diesem Muster vom Zielknoten f zum Startknoten a ergibt sich der kürzeste Pfad bestehend aus den Kanten $\{(a, b), (b, e), (e, f)\}$.

2.7.2 A*-Algorithmus

Eine Verbesserung des Dijkstra-Algorithmus ist der A*-Algorithmus. Dieser optimiert den Suchraum dahingehend, dass mehr in direkter Richtung zum Ziel gesucht wird. Der Dijkstra-Algorithmus tendiert hingegen dazu den Suchraum gleich in alle Richtungen abzusuchen. Somit ist der A*-Algorithmus schneller und effizienter, da weniger Knoten besucht werden müssen, um einen optimalen Pfad von Start zu Ziel zu finden. Experimente haben sogar gezeigt, dass der A*-Algorithmus teilweise nur 10 % der besuchten Knoten benötigt, die der Dijkstra-Algorithmus für den gleichen optimalen Pfad gebraucht hätte [35, S. 532-535].

Der A*-Algorithmus ist in Abbildung 19 basierend auf [35, S. 534] zu sehen. Die gerichtete Suche zum Ziel wird über eine Heuristik $h(v)$ umgesetzt. Die Heuristik stellt dabei eine Schätzung der Distanz von jedem Knoten zum Ziel dar. Oft wird für diese Heuristik die Euklidische Distanz (Luftlinie) verwendet. Durch diese Vorausschau zum Ziel wird sichergestellt, dass Knoten, die sich zu weit vom Ziel weg befinden, nicht besucht werden. In Abbildung 19 ist die Verwendung der Heuristik in der letzten Zeile zu sehen. Hier wird der nächste Knoten ausgewählt, der besucht werden soll. Dabei muss nun jedoch die Summe aus Distanz zum Startknoten und der Heuristik minimal sein. Befindet sich ein unbesuchter Knoten zu weit weg vom Ziel, besitzt also eine Heuristik mit einem großen Wert, so wird er nicht als nächster Knoten in Betracht gezogen [35, S. 534-535].

A*-Algorithmus

Input: s (start node)

Output: $d(v_{goal})$ (distance from s to v_{goal})

Parameters: $G(V, E), v_{goal}$

mark s as unvisited and set $d(s) = 0$

mark other nodes v as unvisited with $d(v) = \infty$

$u = s$ (set start as current node)

while v_{goal} is unvisited:

foreach edge $e = (u, v) \in E$ and v is unvisited:

if $d(u) + c(e) < d(v)$:

$d(v) = d(u) + c(e)$

mark u as visited

$u = \text{unvisited } v \in V \text{ with minimum } d(v) + h(v)$

Abbildung 19: Ablauf des A*-Algorithmus nach [35, S. 534]

2.8 Lokaler Pfadplaner

Wenn ein initialer Pfad durch einen globalen Pfadplaner gefunden wurde, muss die Roboterbewegung in unmittelbarer Umgebung geplant und umgesetzt werden. Diese Aufgabe übernimmt der lokale Pfadplaner. Er bestimmt die beste Trajektorie und generiert die dafür notwendigen Steuerkommandos für den Antrieb und die Lenkung. In einer Welt mit dynamischen Objekten und fehlerhaften oder unvollständigen Karten ist es unabdingbar in Echtzeit auf Hindernisse reagieren zu können. Nur so sind Kollisionen vermeidbar [36, S. 74].

2.8.1 Timed Elastic Band

Der in dieser Arbeit verwendete Ansatz für den lokalen Pfadplaner heißt Timed Elastic Band (TEB). Die Kernidee besteht darin einen initialen Pfad, den man sich als elastisches Gummiband zwischen Start und Ziel vorstellen kann, so zu verformen, dass dynamische Hindernisse umfahren werden. Die Wegpunkte des globalen Pfades werden dabei in eine Trajektorie mit einer Zeitabhängigkeit transformiert. Das heißt es können auch Geschwindigkeit und Beschleunigung des Roboters angepasst werden [36, S. 74]. Abbildung 20 zeigt den schematischen Ablauf des TEB-Verfahrens.

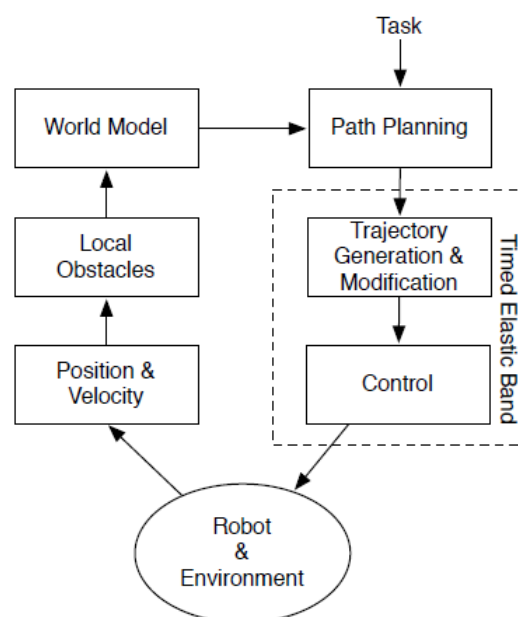


Abbildung 20: Schematische Darstellung des TEB-Planers [36, S. 74]

Ein Roboter interagiert mit seiner Umwelt. Über die Odometrie wird die Position durch Integration der Geschwindigkeit ermittelt. Lokale Hindernisse können mithilfe externer Sensorik wie einem LiDAR oder einer Kamera erfasst werden. Aus diesen Daten wird ein Modell der Umgebung erstellt. Der Pfadplaner verarbeitet diese Eingangsdaten und generiert daraus eine Trajektorie. Alternativ kann auch eine bestehende Trajektorie modifiziert werden, wenn beispielsweise dynamische Hindernisse miteinbezogen werden müssen. Um die gewünschte Trajektorie umsetzen zu können, werden Steuerkommandos zur Regelung der Geschwindigkeit und des Lenkwinkels an die Antriebssteuerung des Roboters weitergeleitet.

Der Algorithmus verwendet den Vektor $x_i = (x, y, \theta)^T$ als Systemzustand. Die Koordinaten x und y markieren die Position des Roboters im zweidimensionalen Raum und θ steht für die Orientierung. Eine Trajektorie setzt sich aus einer Sequenz an Posen zusammen [36, S. 75]:

$$Q = \{x_i\}_{i=0,\dots,n} \quad n \in \mathbb{N} \quad (2.17)$$

Darüber hinaus gibt es zwischen zwei Posen die Zeitdifferenz ΔT_i . Insgesamt bei n Posen, die eine Trajektorie formen, gibt es folglich $n - 1$ Zeitdifferenzen [36, S. 75]:

$$\tau = \{\Delta T_i\}_{i=0,\dots,n-1} \quad (2.18)$$

Das elastische Band (TEB) wird als Tupel beider Sequenzen definiert: $B := (Q, \tau)$. Die Schlüsselidee ist nun B hinsichtlich der Posen und Zeitintervalle zu optimieren. Dies geschieht mit der Kostenfunktion $f(B)$. Sie besteht aus einer gewichteten Summe von k Teilkomponenten, die die gewünschten Eigenschaften repräsentieren. Die Gleichungen (2.19) und (2.20) stellen diesen Zusammenhang dar [36, S. 75].

$$f(B) = \sum_k \gamma_k f_k(B) \quad (2.19)$$

$$B^* = \operatorname{argmin}_B f(B) \quad (2.20)$$

B^* steht schließlich für die optimierte Trajektorie, die gefunden ist, wenn die Funktion $f(B)$ minimiert ist. Die Komponenten $f_k(B)$ der Kostenfunktion $f(B)$ setzen sich zum einen aus den Geschwindigkeits- und Beschleunigungslimits des Antriebs zusammen. Folglich würde die Kostenfunktion einen hohen Wert annehmen, wenn ein Limit erreicht ist. Zum anderen gehen auch Aspekte bezüglich Hindernisse und Wegpunkte mit ein. Das bedeutet, Wegpunkte ziehen das elastische Band an und Hindernisse stoßen es ab. Der minimale Wert der Kostenfunktion ergibt sich also dann, wenn sich die Trajektorie möglichst nahe an den Wegpunkten des ursprünglichen globalen Pfades befindet und gleichzeitig keine Hindernisse enthält. Als letztes geht die Zeitdifferenz zwischen zwei Posen in die Kostenfunktion ein. Das Ziel ist hierbei nicht nur den kürzesten, sondern auch den schnellsten Pfad zu finden. Dies kann durch Anpassung der Beschleunigung bzw. der Geschwindigkeit erfolgen. Mit dem Faktor γ_k werden die verschiedenen Teilkomponenten, je nach Relevanz, schließlich noch gewichtet [36, S. 75-76].

Die Minimierung der Kostenfunktion kann beispielsweise durch das Gradientenabstiegsverfahren realisiert werden.

2.8.2 Online Trajektorien-Optimierung

Das Problem des ursprünglichen TEB-Algorithmus ist, dass er bei dynamisch bewegten Objekten in lokalen Minima der Kostenfunktion stecken bleibt. Das bedeutet es wird keine optimale Trajektorie gefunden und es kann zur Kollision kommen. Außerdem nimmt die Anzahl der lokalen Minima mit der Anzahl an Hindernissen zu. Aus diesem Grund stellt die Online Trajektorien-Optimierung eine Erweiterung des TEB-Ansatzes dar. Dabei werden fortlaufend während der Laufzeit, neben der TEB-Trajektorie, noch weitere alternative Trajektorien berechnet. Aus diesen Kandidaten wird schließlich die beste lokale Trajektorie bestimmt [37, S. 142-143]. Die Ermittlung der alternativen Trajektorien erfolgt über Homologie-Klassen. Zur Erklärung des Homologie-Begriffs wird Abbildung 21 herangezogen. Zwei Pfade τ_1 und τ_2 , die den gleichen Start- und Zielpunkt besitzen, sind zueinander homolog, wenn die disjunkte Vereinigung $\tau_1 \sqcup -\tau_2$ eine zweidimensionale Fläche ergibt, die keine Hindernisse beinhaltet. Da die geformte Fläche A keines der drei Objekte $\{O_1, O_2, O_3\}$ umschließt, besitzen die Pfade τ_1 und τ_2 die gleiche Homologie-Klasse. Würde hingegen die disjunkte Vereinigung $\tau_1 \sqcup -\tau_3$ oder $\tau_2 \sqcup -\tau_3$ gebildet werden, wäre in der entstandenen Fläche immer das Objekt O_2 enthalten. Deswegen gehört Pfad τ_3 zu einer zweiten Homologie-Klasse [38, S. 41-43].

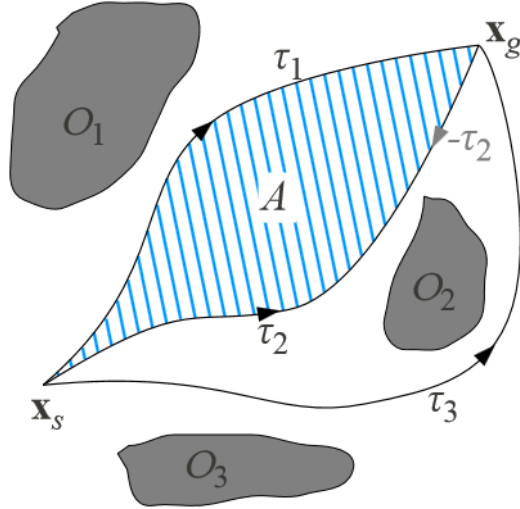


Abbildung 21: Darstellung von homologen Pfaden [38, S. 42]

Um die Homologie-Klasse eines Pfades τ mathematisch bestimmen zu können, wird jeweils die so genannte H-Signatur nach [37, S. 147] berechnet:

$$H(\tau) = \sum_{k=1}^{N-1} H_s(z_k, z_{k+1}) \quad (2.21)$$

Um die gesamte H-Signatur eines Pfades zu erhalten, werden die Signaturen H_s aller Segmente aufaddiert. Die H-Signatur eines Segments $H_s(z_k, z_{k+1})$ berechnet sich nach Gleichung (2.22) [37, S. 147]:

$$H_s(z_k, z_{k+1}) = \sum_{l=1}^R A_l (\ln(z_{k+1} - \xi_l) - \ln(z_k - \xi_l)) \quad (2.22)$$

Die Anzahl aller relevanten Hindernisse wird mit R bezeichnet. Pro Objekt wird ein repräsentativer Punkt ξ_l ausgewählt. Der Faktor A_l ergibt sich zu $A_l = f_0(\xi_l) [\prod_{j=1, j \neq l}^R (\xi_l - \xi_j)]^{-1}$ mit $f_0(z) = ab(z - BL)(z - TR)$ und $a = \text{ceil}(\frac{R}{2})$, $b = R - a$, $BL \in \mathbb{C}$, $TR \in \mathbb{C}$. Wenn die H-Signatur mehrerer Pfade identisch ist, dann gehören sie zur gleichen Homologie-Klasse. Zum Schluss werden alle Pfade pro Klasse so gefiltert, dass nur noch ein Pfad übrigbleibt, der dann jeweils eine Homologie-Klasse repräsentiert [37, S.

147]. Ein Anwendungsbeispiel des TEB mit Online Trajektorien-Optimierung ist in Abbildung 22 dargestellt.

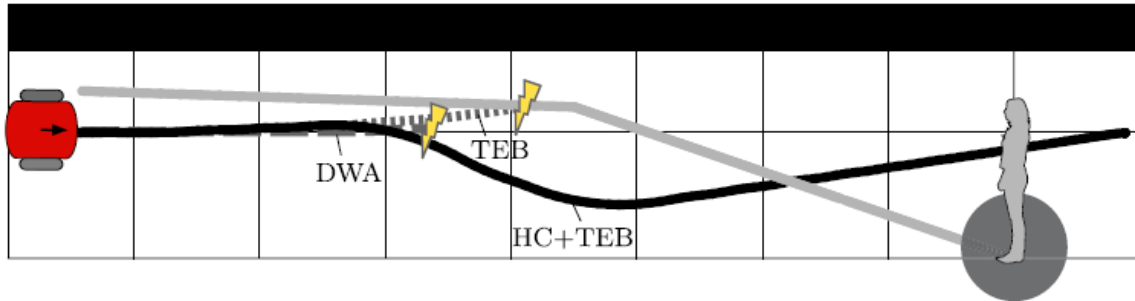


Abbildung 22: Anwendungsbeispiel TEB mit Online Trajektorien-Optimierung [37, S. 152]

Die graue Linie stellt die Trajektorie eines bewegten Hindernisses dar. Die gestrichelten Linien beziehen sich auf die Trajektorien, die durch den gewöhnlichen TEB-Algorithmus oder durch die DWA-Methode (in dieser Arbeit nicht behandelt) erzeugt wurden. Es ist zu sehen, dass nach dem gewöhnlichen TEB-Ansatz ohne Online Trajektorien-Optimierung eine Kollision mit dem Fußgänger entsteht. Verwendet man hingegen TEB mit Online Trajektorien-Optimierung, fällt auf, dass das Hindernis rechtzeitig umfahren wird und keine Kollision verursacht wird. Diese Trajektorie wird durch die schwarze Linie abgebildet. Entscheidend für das bessere Abschneiden ist die konstante Suche nach neuen Homologie-Klassen. Dadurch ist bereits relativ früh die optimale Trajektorie um das Hindernis herum bekannt und muss schließlich nur noch zum richtigen Zeitpunkt als beste Variante ausgewählt werden. Das originale TEB-Verfahren berücksichtigt hingegen von Anfang an keine alternativen Pfade. Somit bleibt es im lokalen Minimum stecken und kann nicht schnell genug reagieren, um eine neue Trajektorie zu planen. Dieses Beispiel zeigt somit eindeutig die Vorteile von TEB mit der Verwendung von Homologie-Klassen. Allerdings muss man auch berücksichtigen, dass dieser Ansatz rechenaufwändiger ist, da permanent alternative Pfade berechnet werden müssen.

3. Methodik und Vorgehen

In diesem Kapitel wird das Vorgehen und die ausgewählte Methodik präsentiert, um eine automatisierte Navigation, angelehnt an der Stufe 4 des autonomen Fahrens, umzusetzen. Zuerst wird auf den Aufbau des Modellautos eingegangen. Danach werden die wichtigsten Komponenten und Mechanismen des Robot Operating Systems (ROS) erklärt. Als nächstes wird auf die LiDAR-Inbetriebnahme eingegangen und wie zusammen mit einem SLAM-Algorithmus eine Karte der Umgebung erstellt werden kann. Wenn eine Karte bekannt ist, wird die automatisierte Navigation implementiert. Dafür werden die verwendeten Pfadplaner und die Lokalisierungsmethode beschrieben. Zum Schluss wird auf die vollautomatisierte Navigation mit den zuvor umgesetzten Komponenten eingegangen. Dazu gehört auch die Entwicklung einer GUI, mit der dann vordefinierte Routen angesteuert werden können.

3.1 Aufbau des Modellautos

Das Modellauto, das für die Umsetzung der automatisierten Fahrfunktion verwendet wird, ist in Abbildung 23 dargestellt.



Abbildung 23: Abbildung des Modellautos

Die Abbildung zeigt bereits alle fertig montierten Komponenten inklusive des LiDARs, der für diese Arbeit beschafft wurde. Grundlage für das Fahrzeug bildet das Chassis Traxxas Slash 4x4 Platinum Edition 6804R im Maßstab 1:10. Es besitzt Allradantrieb mit einem Radstand von 324 mm und den Maßen 294 mm x 568 mm [39]. Im Chassis sind zwei Elektromotoren verbaut. Zum einen der Sensorless-Brushless-Motor TRX335R1 Velineon, der das Fahrzeug antreibt. Und zum anderen ein Servomotor an der Vorderachse, der die Lenkung ansteuert. Beide Motoren werden über einen Electronic Speed Controller (ESC) geregelt. Hierbei kommt das Modell ESK8 1.2 zum Einsatz, das ursprünglich für die Verwendung bei E-Skateboards konzipiert wurde [40]. Softwareseitig nutzt der ESC das VESC Tool (Vedder Electronic Speed Controller). Damit lassen sich eine Vielzahl an Parametern konfigurieren, um die Elektromotoren optimal anzusteuern [41]. Die letzte Komponente des Antriebsverbunds ist der Lithium-Polymer-Akku. Dieser besitzt eine Kapazität von 5000 mAh und besteht aus drei seriell verschalteten Zellen [39]. Der Akku ist direkt mit dem ESC verbunden. Der ESC generiert schließlich zum einen die Spannung für den Servomotor und zum anderen mit einem Umrichter den dreiphasigen Strom für den Antriebsmotor.

Wie in Abbildung 23 zu sehen ist, wird eine Holzkonstruktion verwendet, um alle weiteren Komponenten zu montieren. Um darüber einen besseren Überblick zu erhalten, sind in Abbildung 24 alle Teile mit ihren Kommunikationswegen dargestellt.

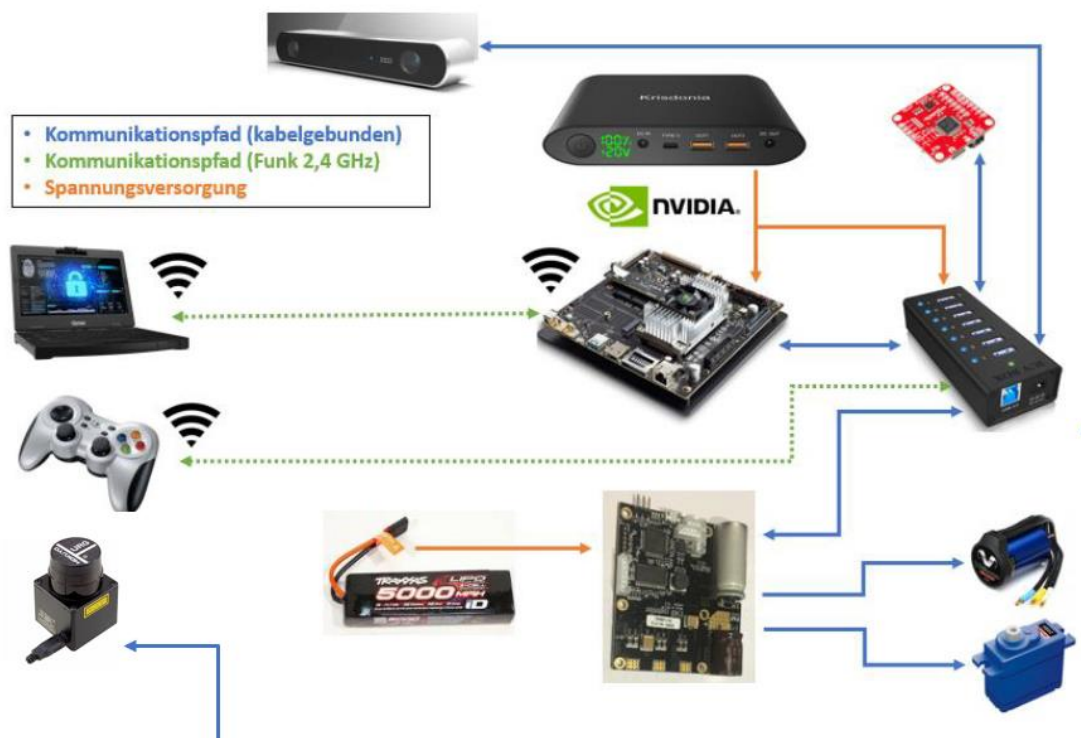


Abbildung 24: Darstellung aller Komponenten mit ihren Kommunikationswegen

In der rechten unteren Ecke befinden sich alle bereits beschriebenen Komponenten des Antriebs. Für die Umsetzung der automatisierten Fahrfunktionen bildet der NVIDIA Jetson TX2 das Herzstück. Dies ist ein eingebetteter Hochleistungscomputer auf einem Modul. Er besitzt eine NVIDIA Denver Dual Core CPU und einen ARM Cortex-A57 Prozessorkomplex mit vier Kernen. Des Weiteren ist ein NVIDIA Pascal Grafikprozessor mit 256 NVIDIA CUDA-Recheneinheiten verbaut. Dieser ermöglicht vor allem bei KI-Anwendungen eine hohe Rechenleistung. Der 8 GB Arbeitsspeicher wird über eine Speicherbandbreite von 59.7 GB/s angesprochen. Insgesamt beläuft sich der Leistungsverbrauch je nach Auslastung auf 7.5 W bis 15 W [42]. Alle diese Faktoren machen den Jetson TX2 zu einer sehr performanten und gleichzeitig effizienten Recheneinheit, die perfekt für robotische Anwendungen geeignet ist. Durch seine kompakte Bauform kann er zusätzlich leicht auf der Holzkonstruktion montiert werden. Die Kommunikation mit dem Motorsteuergerät findet über einen USB-Hub statt. Außerdem wird die Spannungsversorgung des Jetson TX2 und des USB-Hubs mit einer Krisdonia Powerbank mit 25 000 mAh Kapazität sichergestellt. Diese besitzt verschiedene Spannungsstufen, die eingestellt werden können. In der gegenwärtigen Anwendung wird eine Spannung von 16 V verwendet.

Für alle weiteren Robotikanwendungen wird diverse Sensorik benötigt. Eine zentrale Komponente zur Umgebungswahrnehmung ist die ZED Stereokamera. Diese besitzt zwei Linsen mit einem FOV (Field of View) von 90° (H) x 60° (V) x 100° (D) und einer Auflösung von 4 MP. Die Bildfrequenz beträgt maximal 100 fps. Durch die Verwendung dieser Kamera ist es möglich dreidimensionale Bilder des Raumes mit einer maximalen Tiefe von 20 m zu erstellen [43]. In vorangegangenen studentischen Abschlussarbeiten wurde dieser Sensor bereits für eine Verkehrsschildererkennung und eine ACC-Regelung verwendet.

Ein weiterer Sensor ist die IMU (Inertial Measurement Unit) SparkFun SEN-14001 9DoF Razor IMU M0. Dieser kombiniert einen SAMD21-Mikroprozessor mit einem MPU-9250 9DoF-Sensor. 9DoF steht hier für neun Freiheitsgrade. Diese ergeben sich aus der Messung der Beschleunigung, Winkelgeschwindigkeit und des Magnetfelds entlang der drei Koordinatenachsen eines Raums [44, S. 5-6]. Die IMU kommt in diesem Projekt nicht zum Einsatz, da die Bestimmung der Roboterpose durch doppelte Integration der Beschleunigung zu ungenau wäre. Stattdessen wird die Pose nach der Beschreibung in Kapitel 2.3 berechnet.

Der letzte wichtige Sensor zur Interaktion mit der Umgebung ist der LiDAR Hokuyo URG-04LX-UG01. Dieser wurde speziell für die Umsetzung der SLAM-basierten Navigation in dieser Arbeit beschafft. Die Montage des Laserscanners erfolgt in einer Höhe von 45 cm auf einer Holzplattform, da in dieser Höhe die meisten Objekte erfasst werden können. Wie alle anderen Sensoren wird dieser auch mit USB angesteuert (s. Abbildung 24). Der LiDAR

wird in diesem Kapitel nicht weiter beschrieben, da in Kapitel 3.3 eine detaillierte Beschreibung folgt.

Zuletzt wird noch auf die Komponenten zur externen Steuerung des Modellautos eingegangen. Diese sind jeweils über Wifi mit dem 2.4 GHz Frequenzband an den NVIDIA Jetson TX2 angebunden. Der Logitech Joypad F710 Controller ermöglicht eine manuelle Steuerung des Fahrzeugs. Dabei kann mit dem linken Joystick die Geschwindigkeit geregelt werden und mit dem rechten der Lenkwinkel. Die Umsetzung der Steuerbefehle erfolgt mittels ROS-Knoten, die mit dem ESC kommunizieren. Darüber hinaus wird noch ein Host-Laptop benötigt auf dem ebenfalls ROS installiert sein muss. Damit können Konfigurationen und die Überwachung aller Funktionen direkt auf dem Laptop durchgeführt werden. Somit muss nicht jedes Mal eine Verbindung mittels externen Bildschirms und Tastatur/Maus zum Jetson TX2 hergestellt werden. Dies ist vor allem im aktiven Fahrbetrieb des Roboters hilfreich.

3.2 Robot Operating System (ROS)

Wie bereits beschrieben bildet der NVIDIA Jetson TX2 die Schaltzentrale des mobilen Roboters. Alle Fahraufgaben werden mithilfe von ROS umgesetzt. ROS steht dabei für Robot Operating System. Dabei ist es weniger als klassisches Betriebssystem wie Linux oder Windows zu verstehen, sondern mehr als Framework, um Robotikapplikationen zu entwickeln. ROS wird also auf ein bestehendes Betriebssystem installiert [45, S. 79]. In dieser Arbeit wird als Basisbetriebssystem Ubuntu 18.04 mit ROS Melodic verwendet.

ROS sorgt als Roboterkontrollarchitektur dafür, dass Prozesse miteinander kommunizieren können. Es implementiert somit einen Standard für Nachrichtentypen, Kommunikationswege und auch ein Buildsystem für die Erstellung neuer Knoten [25, S. 317-318]. Abbildung 25 zeigt eine beispielhafte Kommunikation zwischen zwei Programmen, die bei ROS „Nodes“ genannt werden. „Node 1“ könnte hier ein Programm sein, das die Sensordaten eines LiDARs zur Verfügung stellt. „Node 2“ hingegen benötigt die Sensordaten als Input und könnte einen Aktor (z.B. einen Motor) ansteuern. Es wird immer ein ROS-Master benötigt, der den Basisknoten *roscore* ausführt. Dieser Knoten steuert die komplette Kommunikation im Hintergrund. Eine Nachricht wird dabei über so genannte *Topics* oder *Services* versendet. Diese stellen also die Kommunikationskanäle dar. Ein Topic kann von einem oder mehreren Knoten abonniert (*subscribe*) werden. Wenn ein Sender auf dem Topic Nachrichten veröffentlicht (*publish*), erhalten alle Knoten, die zuvor abonniert haben, diese Information. Bei Services ist die Kommunikation hingegen

ereignisgesteuert. Das heißt ein Client muss bei einem Dienst explizit einen Wert anfragen, bevor dieser schließlich ausgeliefert wird [46, S. 150-152].

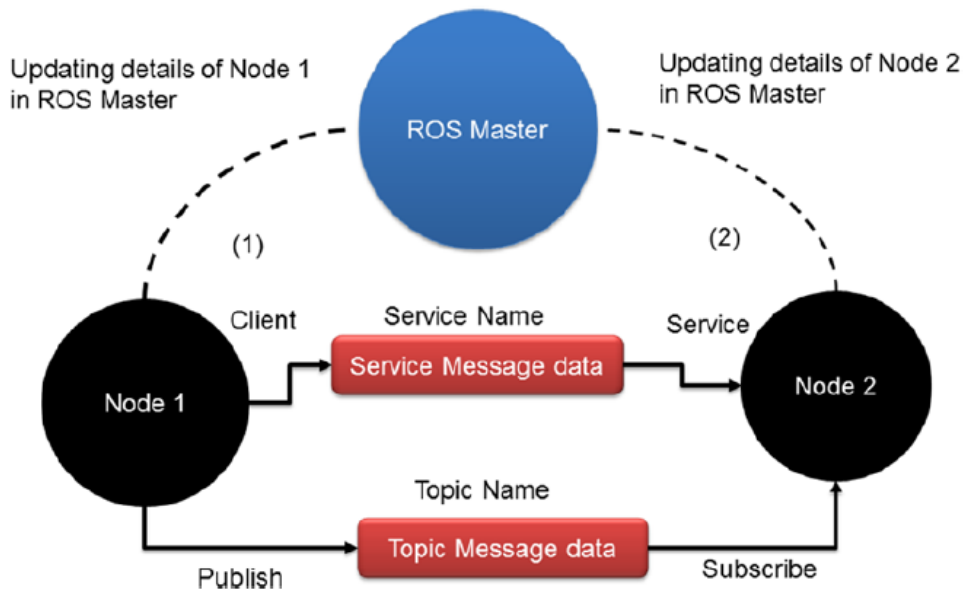


Abbildung 25: Beispielhafte Kommunikation zwischen zwei ROS-Knoten [46, S. 151]

Im Folgenden werden einige ROS-Grundbegriffe nach [46, S. 154-155] genauer erläutert.

ROS-Node: Ein ROS-Knoten stellt ein Programm dar, das auf ROS-APIs zugreift und Berechnungen durchführt. Ein Roboter besteht in der Regel aus sehr vielen solcher Softwaremodule, die untereinander kommunizieren.

ROS-Master: Ein Zwischenprogramm, das ROS-Knoten verbindet.

ROS-Parameter-Server: Ein Programm, das normalerweise zusammen mit dem ROS-Master läuft. Der Benutzer kann verschiedene Parameter oder Werte auf diesem Server speichern und alle Knoten können darauf zugreifen. Der Benutzer kann die Privatsphäre der Parameter auch einstellen. Wenn es sich um einen öffentlichen Parameter handelt, haben alle Knoten Zugriff; ist er privat, kann nur ein bestimmter Knoten auf den Parameter zugreifen. Durch die Parameter lassen sich gewisse Funktionalitäten von Knoten konfigurieren. Ein exemplarisches Anwendungsbeispiel wäre der Scanwinkel eines LiDARs.

ROS-Topic: ROS-Topics stehen für Busse, auf denen die ROS-Knoten eine Nachricht senden können. Ein Knoten kann eine beliebige Anzahl von Themen veröffentlichen oder abonnieren.

ROS-Message: Knoten versenden ROS-Messages auf den Topics. Es gibt zum einen Nachrichten, die auf primitiven Datentypen (int, float, bool, etc.) basieren, zum anderen können Benutzer auch ihre eigenen Nachrichtendefinitionen schreiben.

ROS-Service: Im Gegensatz zu ROS-Topics, die über einen Mechanismus zum Veröffentlichen und Abonnieren verfügen, besitzt der ROS Service ein Request/Reply-Konzept. Ein Dienstauftrag ist eine Funktion, die immer dann aufgerufen wird, wenn ein Client-Knoten eine Anfrage sendet. Der Knoten, der den Dienst bereitstellt, wird Server-Knoten genannt und derjenige, der den Dienst aufruft, heißt Client-Knoten.

ROS-Bag: Ein ROS-Bag stellt eine nützliche Methode zum Speichern und Wiedergeben von ROS-Topics dar. Dies ist vor allem hilfreich, um die Daten eines Roboters zu protokollieren und um sie später zu analysieren.

Zuletzt bleibt noch zu erwähnen, dass ROS sein eigenes Buildsystem besitzt, um Software zu kompilieren. Innerhalb eines Workspaces sind in der Regel alle ROS-Pakete abgelegt, die den Quellcode für die jeweilige Roboterapplikation enthalten. Das Buildsystem in ROS heißt *catkin*. Es kombiniert CMake Makros und Pythonskripte, um noch mehr Funktionalität bereitzustellen. Mit dem Befehl *catkin_make* wird die Buildkette gestartet und die Software des Workspaces gebaut [45, S. 84].

3.3 Inbetriebnahme des LiDARs

Bevor der LiDAR in Betrieb genommen werden kann, muss der Host-Laptop installiert werden, um einen externen Zugriff auf das Modellauto zu gewährleisten. Dafür wird ein Dell Latitude E6320 verwendet. Um keine Kompatibilitätsprobleme zu haben, wird die gleiche ROS- und Ubuntuversion verwendet, die der Jetson TX2 verwendet, nämlich ROS Melodic und Ubuntu 18.04. Die genauen Befehle zur Installation und dem Erstellen eines Workspaces sind im Anhang unter Abschnitt 1 zu finden.

Als nächstes muss eine Wifi-Verbindung vom Host-Laptop zum Jetson TX2 aufgebaut werden. Dafür könnte man einerseits einen WLAN-Router oder Access-Point verwenden. Das Problem ist jedoch hierbei, dass dieser immer in Reichweite sein muss. Wenn das Modellauto in einen anderen Raum bewegt wird, kann die Verbindung abreißen oder der Router muss immer mitgenommen werden. Deswegen bietet es sich an ein Adhoc-Netzwerk aufzubauen. Dies kann zwischen zwei Wifi-Chips ohne zusätzliche Komponente realisiert werden. Es müssen lediglich manuell statische IP-Adressen für die beiden Geräte vergeben werden. Der Jetson erhält die Adresse 10.42.0.1 mit der Subnetzmaske

255.255.255.0 und der Host-Laptop die 10.42.0.2 mit identischer Subnetzmaske. Somit kann schließlich eine Verbindung via SSH oder einem VNC-Client aufgebaut werden.

Nachdem nun eine Verbindung zwischen den Geräten steht, muss festgelegt werden, wer die Rolle des ROS-Masters übernimmt. Ziel ist es hierbei, dass der Host-Laptop alle veröffentlichten Topics des Jetson TX2 mitlesen kann und auch in der Lage ist auf diesen Topics Nachrichten zu veröffentlichen. Dadurch findet zum einen eine Überwachung des Modellautos statt, zum anderen können später aber auch Navigationsziele von dem Laptop aus vorgegeben werden. Dadurch werden keine zusätzlichen I/O-Geräte für den Jetson benötigt. Es ist geschickt den Jetson als ROS-Master zu wählen. Auf diesem wird dann der Basisknoten *roscore* ausgeführt, der die komplette Kommunikation steuert. Folglich kann der Jetson auch allein betrieben werden. Würde hingegen der Host-Laptop als Master gewählt werden, müsste dieser immer in Betrieb sein, um mit dem Modellauto fahren zu können. Die Festlegung des Masterstatus geschieht über zwei Variablen, die im `bashrc`-File des Jetson und des Laptops gesetzt werden. Eine detaillierte Beschreibung ist dazu im Anhang unter Abschnitt 1 zu finden.

Zum Schluss muss noch sichergestellt werden, dass beide Geräte die gleiche Uhrzeit verwenden. Ansonsten kann es möglicherweise im weiteren Verlauf zu Problemen kommen. Da beide Geräte miteinander über ein Adhoc-Netzwerk verbunden sind, besteht kein Internetzugang. Es kann also kein externer Zeitserver zur Synchronisation verwendet werden. Stattdessen fungiert der Host-Laptop als NTP-Server. Der Laptop bekommt die aktuelle Uhrzeit über kurzzeitiges Verbinden mit dem Internet. Da er im Gegensatz zum Jetson TX2 eine BIOS-Batterie besitzt, wird die Uhrzeit nach einem Neustart beibehalten. Schließlich muss noch die IP-Adresse des Laptops in der Datei `/etc/systemd/timesyncd.conf` des Jetson eingetragen werden. Dadurch wird der NTP-Server bekannt gemacht. Eine genaue Beschreibung zur Installation des NTP-Servers ist im Anhang unter Abschnitt 1 zu finden.

Nun kann der LiDAR in Betrieb genommen werden. Das für diese Arbeit beschaffte Modell ist der 2D-Laserscanner Hokuyo URG-04LX-UG01. Dieser ist in Abbildung 26 zu sehen.



Abbildung 26: Darstellung des Hokuyo URG-04LX-UG01 [47]

Zuerst muss der LiDAR auf der mobilen Roboterplattform montiert werden. Die Holzkonstruktion (s. Abbildung 23) eignet sich besonders gut, um neue Komponenten darauf zu installieren. Der Hokuyo-LiDAR besitzt auf der Unterseite zwei Gewinde, über die er von unten festgeschraubt werden kann. Die Stromversorgung wird über den USB-Hub realisiert, da laut Datenblatt eine Eingangsspannung von 5 V benötigt wird. Die Datenübertragung findet ebenfalls über das USB-Kabel statt. Der LiDAR kann Abstände mit einer maximalen Distanz von 4 m und einer minimalen Distanz von 2 cm messen und besitzt darüber hinaus einen Scan-Winkel von 240° mit einer Auflösung von 0.36° (s. Abbildung 27). Außerdem sendet er Lichtstrahlen mit einer Wellenlänge von 785 nm aus. Zuletzt ist noch zu erwähnen, dass er über eine Scanrate von 10 Hz verfügt [48, S. 3]. Alle diese Parameter sind für eine Indoor-Navigation absolut ausreichend.

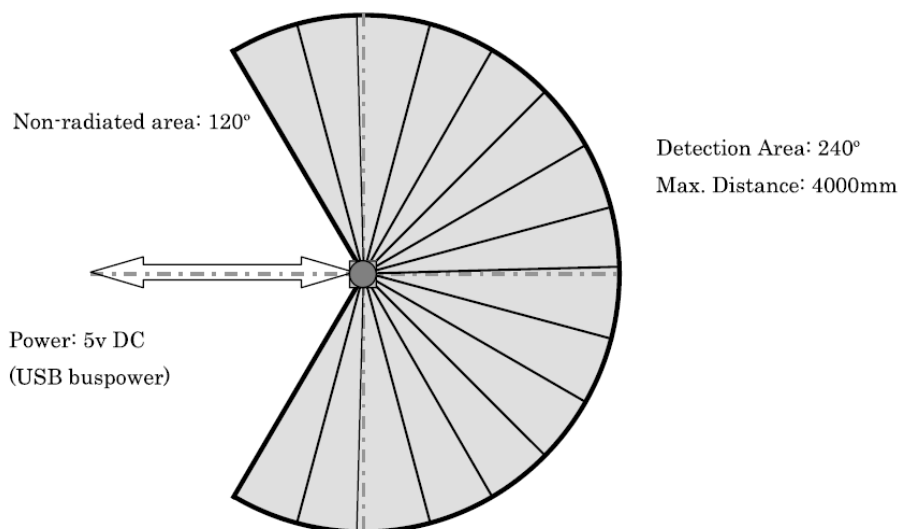


Abbildung 27: Darstellung des Scan-Winkels des Hokuyo-LiDARs [48, S. 2]

Da der LiDAR jetzt hardwareseitig installiert ist, muss die Softwareschnittstelle in Betrieb genommen werden. In Linux wird der LiDAR unter `/dev/ttyACMx` gelistet, wobei das `x` für eine Zahl steht, je nachdem in welcher Reihenfolge die USB-Geräte initialisiert werden. Dadurch kann es passieren, dass der LiDAR nach jedem Neustart unter einem anderen Gerätenamen auftaucht. Um nicht alle Konfigurationsdateien, die auf den Gerätenamen zugreifen, ändern zu müssen, wird ein symbolischer Link erstellt. Dafür wird ausgenutzt, dass die Hersteller-ID "15d1" ist. Damit wird eine Regel erstellt, die den Hokuyo-Laserscanner immer unter dem Gerätenamen `/dev/sensors/hokuyo` anzeigt. Details zur Regelerstellung sind im Anhang unter Abschnitt 2 zu finden.

Im nächsten Schritt muss der Treiber für den LiDAR installiert werden. Dafür wird das Git-Repository `hokuyo_node` [49] verwendet. Dieses implementiert Treiberfunktionalitäten für alle gängigen Hokuyo-Laserscanner. Alle Installationsbefehle sind wieder im Anhang unter Abschnitt 2 zu finden. Wird der Treiberknoten gestartet, werden die Daten des Laserscanners unter dem Topic `/scan` veröffentlicht. Alle versendeten Nachrichten sind vom Typ `sensor_msgs/LaserScan`. Der Aufbau des Nachrichtentyps ist in Tabelle 1 nach [50] zu sehen.

Tabelle 1: Aufbau der Nachricht `sensor_msgs/LaserScan` nach [50]

Datenfeld	Beschreibung
Header header	Gibt den Zeitstempel der ersten Messung an und die <code>frame_id</code> des Koordinatensystems
float32 angle_min	Startwinkel des Scans in [rad]
float32 angle_max	Endwinkel des Scans in [rad]
float32 angle_increment	Winkelauflösung zwischen zwei Messungen in [rad]
float32 time_increment	Zeit zwischen zwei Messungen
float32 scan_time	Zeit zwischen zwei kompletten Scans
float32 range_min	Minimale Distanz in [m]
float32 range_max	Maximale Distanz in [m]
float32[] ranges	Feld der Distanzdaten in [m]. Gemessen gegen den Uhrzeigersinn um die nach oben zeigende z-Achse
float32[] intensities	Feld der Intensitäten

Mit dem Befehl `rostopic echo /scan` werden die Nachrichten ausgegeben. Dabei fällt auf, dass der minimale und maximale Scanwinkel standardmäßig auf $\pm \frac{\pi}{2}$ festgelegt ist. Es wird also nicht der maximal mögliche Scanbereich ausgenutzt. Um dies zu beheben, werden in

der Konfigurationsdatei *sensors.yaml* die Winkel auf $\pm 120^\circ$ geändert (s. Anhang Abschnitt 2). Des Weiteren ist zu sehen, dass die Winkelauflösung (*angle_increment*) auf dem Wert 0.0061359 steht. Umgerechnet ergibt das ungefähr 0.35° , was der im Datenblatt angegebene Auflösung von 0.36° sehr gut entspricht. Der Parameter *scan_time* steht auf 0.1 s. Dies entspricht der angegebenen Scanrate von 10 Hz. Als letztes ist noch auffällig, dass die maximal messbare Distanz (*range_max*) 5.6 m ist. Dies übertrifft die im Datenblatt angegebene Größe um 1.6 m. Die minimal messbare Distanz stimmt mit 2 cm überein.

Um abschließend die Funktionalität des LiDARs bewerten zu können, wird das Grafiktool *rviz* verwendet. Dieses ist bereits in ROS standardmäßig installiert und lässt sich mit dem Befehl *roslaunch rviz rviz* starten. Das Tool wird auf dem Host-Laptop ausgeführt. Damit kann bereits überprüft werden, ob das Lesen von Topics, die von einem entfernten Rechner aus veröffentlicht werden, möglich ist. Mit *rviz* lassen sich also die Nachrichten diverser Topics grafisch darstellen. Abbildung 28 zeigt die bildliche Aufbereitung des Topics */scan*.

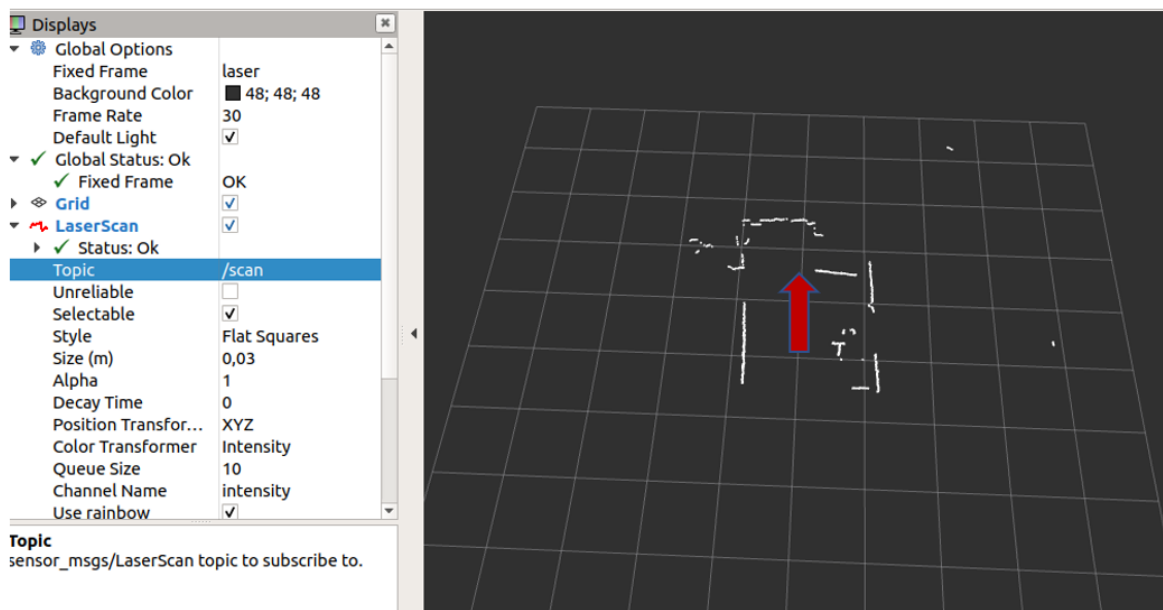


Abbildung 28: Grafische Darstellung des Topics */scan* in *rviz*

Der rote Pfeil stellt die Blickrichtung des Laserscanners dar. Es ist zu sehen, dass alle Messpunkte in einem Bereich von ungefähr $\pm 120^\circ$ angeordnet sind und sich somit eine zweidimensionale Abbildung der Umgebung ergibt. Positionsänderungen des Modellautos werden ebenfalls sofort in *rviz* aktualisiert. Folglich lässt sich sagen, dass der LiDAR voll einsatzbereit ist und, dass das gleichzeitige Mitlesen von Daten auf dem Host-Laptop auch funktioniert.

3.4 Kartenerstellung mit SLAM

In diesem Kapitel wird mithilfe des vorher installierten LiDARs und den Odometriedaten des Electronic Speed Controllers eine zweidimensionale Karte eines Büroraumes erstellt. Um dies zu erreichen wird die ROS-Toolbox gmapping [51] verwendet. Dieses Package implementiert den unter Kapitel 2.6 beschriebenen Algorithmus. Das Ziel ist die Wahrscheinlichkeitsdichteverteilung $p(x_t, m | y_t, u_{t-1})$ (vgl. Gleichung 2.17) der Pose und der Karte unter Einbeziehung der LiDAR-Messpunkte y_t und der Bewegungsdaten u_{t-1} zu schätzen. Dazu wird im ersten Schritt nur die Pose geschätzt, indem mit den Linear- und Winkelgeschwindigkeiten die Position und Orientierung berechnet wird. Da für dieses Vorgehen die Geschwindigkeiten integriert werden müssen (vgl. Gleichung 2.9), addiert sich das Rauschen über der Zeit. Aus diesem Grund werden die Odometriedaten mit den Abstandsmessungen des Laserscanners fusioniert. Dieser Prozess wird auch Scan-Matching genannt und ist analog zum AMCL-Algorithmus zur Lokalisierung (s. Kapitel 2.5). Im zweiten Schritt wird dann mit der geschätzten Pose und den Lasermessungen die Karte erstellt.

3.4.1 Bereitstellung der Eingangsdaten

Zur Umsetzung des SLAM-Verfahrens benötigt gmapping bestimmte Eingangsdaten. Der Input-/Output-Fluss ist in Abbildung 29 dargestellt. Als Eingänge werden die drei Topics `/scan`, `/vesc/odom` und `/tf` benötigt. Als Ausgang wird unter `/map` die erstellte Karte als sogenannte Occupancy Grid Map (vgl. Kapitel 2.6) veröffentlicht. Außerdem sind unter dem Topic `/map_metadata` weitere Informationen zur Karte (z.B. Kartenauflösung) zu finden.

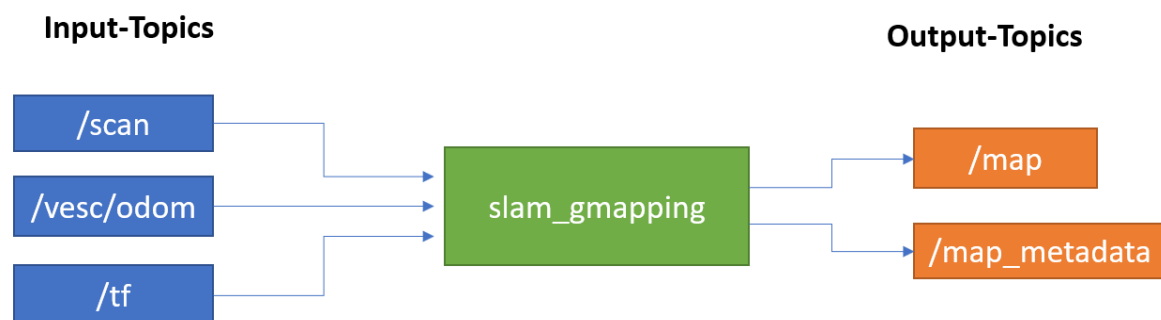


Abbildung 29: Darstellung des Input-/Output-Flusses von gmapping

Das Topic `/scan` wird bereits veröffentlicht. Die beiden anderen Topics `/vesc/odom` und `/tf` werden mithilfe eines ROS-Knotens aus dem F1Tenth-Github-Repository [52] generiert.

Der Quellcode dazu ist auch im Anhang unter Abschnitt 3 zu finden. Die Funktionsweise des Programms stellt sich folgendermaßen dar. Der ROS-Knoten abonniert die Topics `/sensors/core` und `sensors/servo_position_command`, die vom ESC veröffentlicht werden. Die Topics enthalten zum einen die Information bezüglich der Position des Servomotors, was dem aktuellen Lenkwinkel entspricht. Zum anderen wird über die Drehzahl des Elektromotors die Geschwindigkeit bestimmt, da keine Raddrehzahlsensoren verbaut sind. Mit diesen Daten wird sowohl die Roboterpose als auch die Linear- und Winkelgeschwindigkeit berechnet und schließlich unter `/vesc/odom` veröffentlicht. Nachrichten sind dabei vom Typ `nav_msgs/Odometry` Message. Der Aufbau der Nachricht ist in Tabelle 2 nach [53] zu sehen.

Tabelle 2: Aufbau der Nachricht `nav_msgs/Odometry` Message nach [53]

Datenfeld	Beschreibung
Header header	Gibt den Zeitstempel und den Namen des Koordinatensystems an (hier: „odom“)
string child_frame_id	Name des Kind-Koordinatensystems
geometry_msgs/PoseWithCovariance pose Point position Quaternion orientation	Gibt die Pose des Roboters an Position in x, y, z,-Koordinate Orientierung als Quaternion
geometry_msgs/TwistWithCovariance twist Vector3 linear Vector3 angular	Gibt die Geschwindigkeit an Lineare Geschwindigkeit Winkelgeschwindigkeit

Der Header enthält das Weltbezugssystem, von dem aus sich ein Objekt fortbewegt. Das Koordinatensystem des Objekts ist das Kind-Koordinatensystem (`child_frame_id`). Des Weiteren ist zu sehen, dass in der Odometrienachricht die Pose und die Geschwindigkeit gekapselt sind. Die Pose setzt sich aus den Koordinaten der Position und der Orientierung, gegeben als Quaternion, zusammen. Ein Quaternion ist dabei eine alternative Angabe der Rotation eines Körpers. Oft wird die Rotation nämlich auch durch die Eulermatrix ausgedrückt. Die Geschwindigkeit ist durch zwei Vektoren bestimmt. Der eine Vektor bezieht sich auf die lineare Geschwindigkeit und der andere auf die Winkelgeschwindigkeit der drei Raumachsen. Die lineare Geschwindigkeit wird nur in x- und y-Richtung bestimmt und die Winkelgeschwindigkeit nur um die z-Achse, was der Gierrate entspricht.

Das zweite wichtige Topic, das der ROS-Knoten veröffentlicht ist `/tf`. TF bedeutet in diesem Kontext Transformation. In ROS besitzt üblicherweise jede Roboterkomponente ein eigenes Koordinatensystem. Die Koordinatensysteme, die das Modellauto verwendet, sind in Abbildung 30 dargestellt. Es gibt zum einen Koordinatensysteme, deren Position und

Orientierung zueinander sich über der Zeit nicht ändern. Zum Beispiel befindet sich der Laserscanner an einem festen Ort auf dem mobilen Roboter montiert. Somit bleibt die Transformation zwischen den Koordinatensystemen „laser“ und „base_link“ konstant. Es handelt sich um eine statische Transformation. „base_link“ bezeichnet in diesem Fall das Koordinatensystem bezogen auf den Masseschwerpunkt des Roboters.

Zum anderen gibt es auch Koordinatensysteme, deren Beziehung sich über der Zeit verändert. Dies trifft auf die Systeme „odom“ und „base_link“ beispielsweise zu. „odom“ steht für das Weltkoordinatensystem, in dem sich der Roboter bewegt. Wenn der Roboter sich bewegt, verändert sich seine Pose dynamisch. Deswegen muss sich auch die Transformation zwischen den beiden Koordinatensystemen verändern.

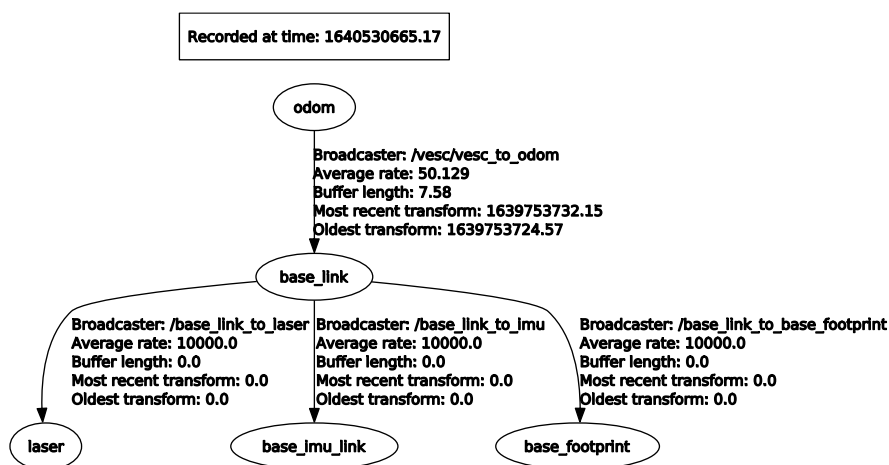


Abbildung 30: Darstellung der Koordinatensysteme des Modellautos

Eine Transformation besteht immer aus einer Translations- und einer Rotationskomponente. Diese Informationen werden im Nachrichtentyp `tf2_msgs/TFMessage` gekapselt. Der genaue Aufbau einer Nachricht ist in Tabelle 3 nach [54] zu sehen.

Tabelle 3: Aufbau der Nachricht `tf2_msgs/TFMessage` nach [54]

Datenfeld	Beschreibung
Header header	Gibt den Zeitstempel und den Namen des Quellkoordinatensystems an
string child_frame_id	Name des Zielkoordinatensystems
Transform transform	Transformation zw. zwei Systemen

Vector3 translation	Translationsvektor
Quaternion rotation	Rotation als Quaternion

Der Header gibt den Zeitstempel und den Namen des Ursprungskoordinatensystems an, von dem die Transformation aus berechnet wird. In vorherigem Beispiel wäre dies somit „odom“. Das Kind-Koordinatensystem stellt das Zielsystem dar (s. Abbildung 30: „base_link“). Die Transformation selbst ist durch den Translationsvektor und das Quaternion, das die Rotation des Koordinatensystems beschreibt, angegeben.

Da das Topic `/tf` standardmäßig nicht vom ROS-Knoten veröffentlicht wird, muss dafür noch der entsprechende Parameter `vesc_to_odom/publish_tf: true` in der zugehörigen Konfigurationsdatei `vesc.yaml` gesetzt werden. Der Aufbau der Datei ist auch im Anhang unter Abschnitt 3 zu finden.

3.4.2 Durchführung der Kartenerstellung

Da nun alle benötigten Daten für den SLAM-Algorithmus bereitgestellt werden, kann gmapping installiert und gestartet werden. Die Befehle dafür sind auch im Anhang unter Abschnitt 3 abgelegt. Als nächstes muss das Modellauto mit gestartetem gmapping-Knoten manuell per Joystick durch den Zielraum bewegt werden. Während der Bewegung durch den Raum, wird simultan eine Karte der Umwelt erstellt. Die Karte kann live mit dem Grafiktool rviz mitverfolgt werden, indem das Topic `/map` eingebunden wird. Das Topic `/map` ist, wie bereits erwähnt, das Output-Topic von gmapping. Die Nachrichten auf diesem Topic sind vom Typ `nav_msgs/OccupancyGrid Message`. Deren Aufbau ist in Tabelle 4 nach [55] dargestellt:

Tabelle 4: Aufbau der Nachricht `nav_msgs/OccupancyGrid Message` nach [55]

Datenfeld	Beschreibung
Header header	Gibt den Zeitstempel und den Namen des Quellkoordinatensystems an
int8[] data	Feld für die Belegungswahrscheinlichkeiten der Karte im Wertebereich [0, 100]. -1 steht für unbekannt.

Der Aufbau der Nachricht ist sehr simpel gehalten. Die Wahrscheinlichkeit, dass ein Feld der Karte belegt ist, wird in ein int8-Array gespeichert. Die Werte werden dabei zeilenweise abgelegt. Die Anzahl an Zeilen und Spalten der Karte kann dem Topic `/map_metadata` entnommen werden. Dieses Topic wird ebenfalls vom Knoten `slam_gmapping`

veröffentlicht. Dessen Nachrichten sind vom Typ `nav_msgs/MapMetaData` und folgendermaßen nach [56] aufgebaut:

Tabelle 5: Aufbau der Nachricht `nav_msgs/MapMetaData` nach [56]

Datenfeld	Beschreibung
<code>time map_load_time</code>	Zeitpunkt zu dem die Karte geladen wurde
<code>float32 resolution</code>	Kartenauflösung in [m/cell]
<code>uint32 width</code>	Anzahl der Zellen, die die Kartenbreite ergeben
<code>uint32 height</code>	Anzahl der Zellen, die die Kartenhöhe ergeben
<code>geometry_msgs/Pose origin</code>	Pose der Zelle (0,0) im Weltsystem
Point position	Position in x, y, z,-Koordinate
Quaternion orientation	Orientierung als Quaternion

Tabelle 5 ist zu entnehmen, dass die beiden Parameter *width* und *height* die Anzahl der Zellen festlegen. Damit ergibt sich schließlich auch die Größe des Felds *data* in der Nachricht `nav_msgs/OccupancyGrid` Message. Des Weiteren wird über das Feld *origin* die Pose, bezogen auf den Kartenursprung, festgelegt.

Eine fertig erstellte Karte des Büroraumes ist in Abbildung 31 dargestellt. Die Wände und Korridore sind klar zu erkennen. Bürostühle und Tische lassen sich auch erahnen. Dabei ist zu erwähnen, dass die optimale Montagehöhe des LiDARs bei ca. 45 cm liegt, da somit die meisten Objekte erkannt werden. Hindernisse, die höher oder tiefer liegen werden nicht berücksichtigt. Insofern hat die Höhe des LiDARs einen enormen Einfluss auf die Qualität der Kartenerstellung. Außerdem sind an den Kartenrändern Strahlen sichtbar, die sich vermeintlich aus den Räumen hinausbewegen. Diese treten auf, wenn die LiDAR-Strahlen auf Glas treffen. Hierbei werden die optischen Signale nicht reflektiert, weswegen der Laserscanner den maximalen Abstand von 5.6 m annimmt. Des Weiteren empfiehlt es sich möglichst langsam und ruhig den Raum abzufahren, sodass der LiDAR die Umwelt komplett erfassen kann und die Daten auch verarbeitet werden können. Teilweise muss eine Strecke auch mehrmals abgefahren werden, dass alle Objekte klar erkannt werden. Außerdem sollten sich zu diesem Zeitpunkt nur statische Hindernisse in dem Raum befinden. Wenn sich Personen zur gleichen Zeit dynamisch mitbewegen, würden diese auch als Hindernisse erkannt und in der Karte verortet werden. Dadurch wird später die Pfadplanung unnötig erschwert. Aus diesem Grund sollte die Person, die das Fahrzeug mittels Joysticks steuert, darauf achten sich stets im blinden Punkt des LiDARs zu bewegen. Dieser befindet sich direkt hinter dem Modellauto, da der Laserscanner einen Winkel von 240° abdeckt. Gewisse Unsicherheiten in der Karte sind jedoch möglich und

können später durch den lokalen Pfadplaner korrigiert werden. Ein Bürostuhl befindet sich beispielsweise selten am genau gleichen Ort. Insofern kann später auf neu auftauchende Hindernisse adaptiv reagiert werden. Eine erstellte Karte kann jedoch nachträglich nicht mehr angepasst werden, außer sie wird komplett neu generiert. Somit ist es geschickt darauf zu achten, dass der Raum nur Objekte enthält, die dort auch die meiste Zeit anzutreffen sind.



Abbildung 31: Aufgezeichnete Karte eines Büroraumes mittels slam_gmapping

3.5 Vollautomatisierte Navigation

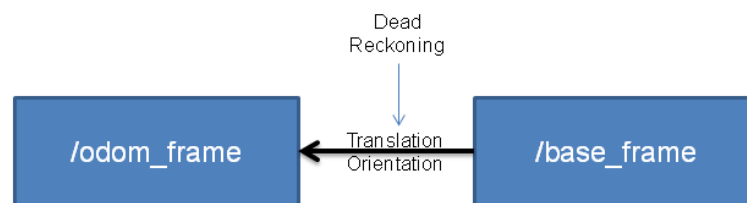
In diesem Kapitel geht es um die vollautomatisierte Navigation des Modellautos. Das bedeutet der Roboter soll selbstständig vorgegebene Zielposen einnehmen. Dafür wird zum einen der AMCL-Algorithmus (s. Kapitel 2.5) zur Lokalisierung auf der Karte benötigt und zum anderen ein globaler und ein lokaler Pfadplaner (s. Kapitel 2.7 und 2.8), die die Zielroute berechnen und ausführen. Die Funktion wird mithilfe der ROS-Toolboxen navigation [57] und teb_local_planner [58] umgesetzt. Die Installationsanweisungen dazu sind im Anhang unter Abschnitt 4 zu finden.

3.5.1 Lokalisierung

Die Bestimmung der Pose auf der vorher erstellten Karte wird durch den Adaptive Monte Carlo Lokalisierungs-Algorithmus durchgeführt. Wie in Kapitel 2.5 beschrieben, basiert dieser auf einem Partikelfilter, der in einer ersten Prädiktionsphase mithilfe des Bewegungsmodells (s. Gleichung 2.9) die eigene Pose schätzt und schließlich in der Updatephase mit externen Abstandsmessungen den Systemzustand aktualisiert. Da das Bewegungsmodell auf verrauschten Odometriedaten fußt, die integriert werden, ist die Korrektur mithilfe einer externen Messung notwendig. Dadurch wird also die Dichtefunktion des Roboterzustands durch Partikel angenähert.

Abbildung 32 verdeutlicht die Funktionsweise von AMCL nochmals.

Odometry Localization



AMCL Map Localization

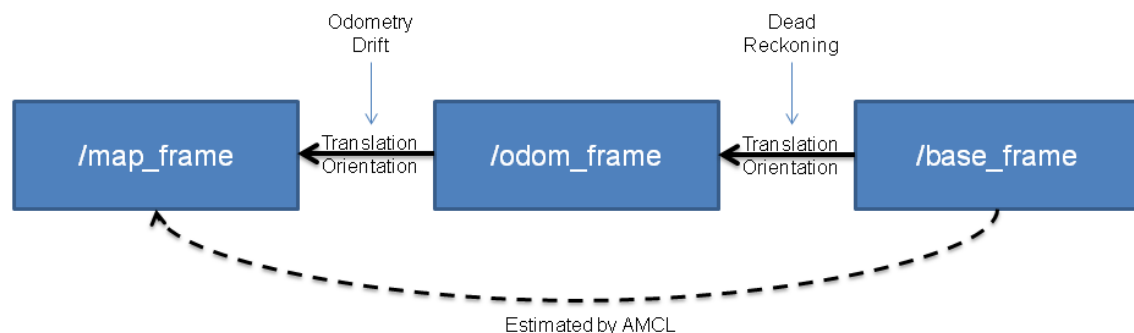


Abbildung 32: Darstellung der Lokalisierung mit AMCL [59]

Oben ist die Lokalisierung nur über Odometriedaten dargestellt. Dabei gibt es eine Transformation vom `base_frame` des Roboters zum `odom_frame`. Diese Transformation zwischen den beiden Koordinatensystemen wird nach dem Bewegungsmodell in Gleichung (2.9) berechnet. Genauer hingegen ist die untere Darstellung, die sich auf die Lokalisierung

mit AMCL bezieht. Hierbei gibt es noch eine weitere Transformation zum `map_frame`, also zum Koordinatensystem der Karte. Das Ziel von AMCL ist es die Transformation von `base_frame` nach `map_frame` zu schätzen, um den Roboter auf der Karte verorten zu können. Dabei fließen neben den Bewegungsdaten auch die Laserscanner-Messungen mit ein, um den Drift in der Odometrie zu kompensieren.

Abbildung 33 zeigt die Karte eines Büroraumes mit initialem AMCL-Zustand. Jeder rote Pfeil repräsentiert ein Partikel. Zu Beginn sind die Partikel willkürlich über den Raum verteilt, weil sich zu Beginn der Roboter theoretisch an jeder Position im Raum aufhalten könnte.

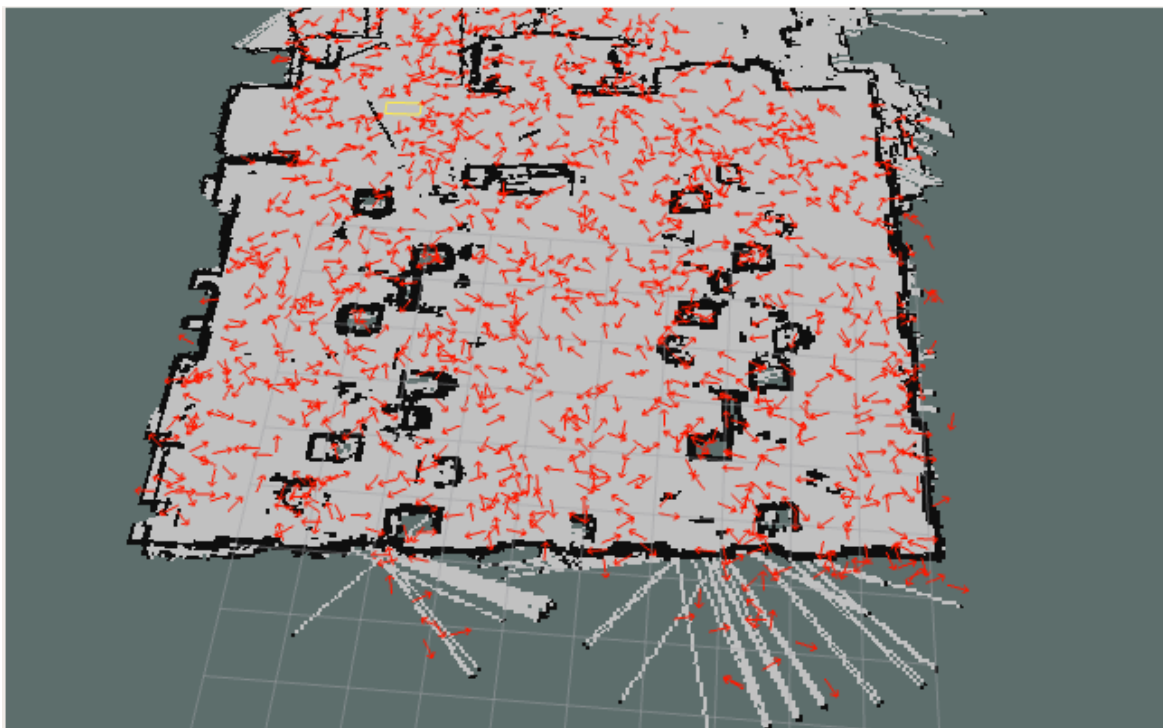


Abbildung 33: Darstellung des initialen AMCL-Zustands

Nach einigen Iterationen des Algorithmus sammeln sich die Partikel an einem Ort. Das heißt mit dem mehrmaligen Durchlaufen der Prädiktions- und Updatephase des Partikelfilters wird eine bestimmte Roboterpose immer wahrscheinlicher. Dies wird erreicht, indem der Roboter im Raum umherbewegt wird. Es ist auch zu sehen, dass eine gewisse Unsicherheit bis zum Schluss bestehen bleibt. Idealerweise wäre die Fläche der sich ansammelnden Partikel möglichst klein. Die Unsicherheit kann durch ungenaue LiDAR-Messungen zustande kommen oder durch die komplexe Umgebung, die eine genaue Lokalisierung erschwert. Eine Umwelt, die beispielsweise nur aus Räumen ohne Möblierung und Korridoren bestehen würde, würde es dem Roboter einfacher machen seine Pose zu bestimmen.

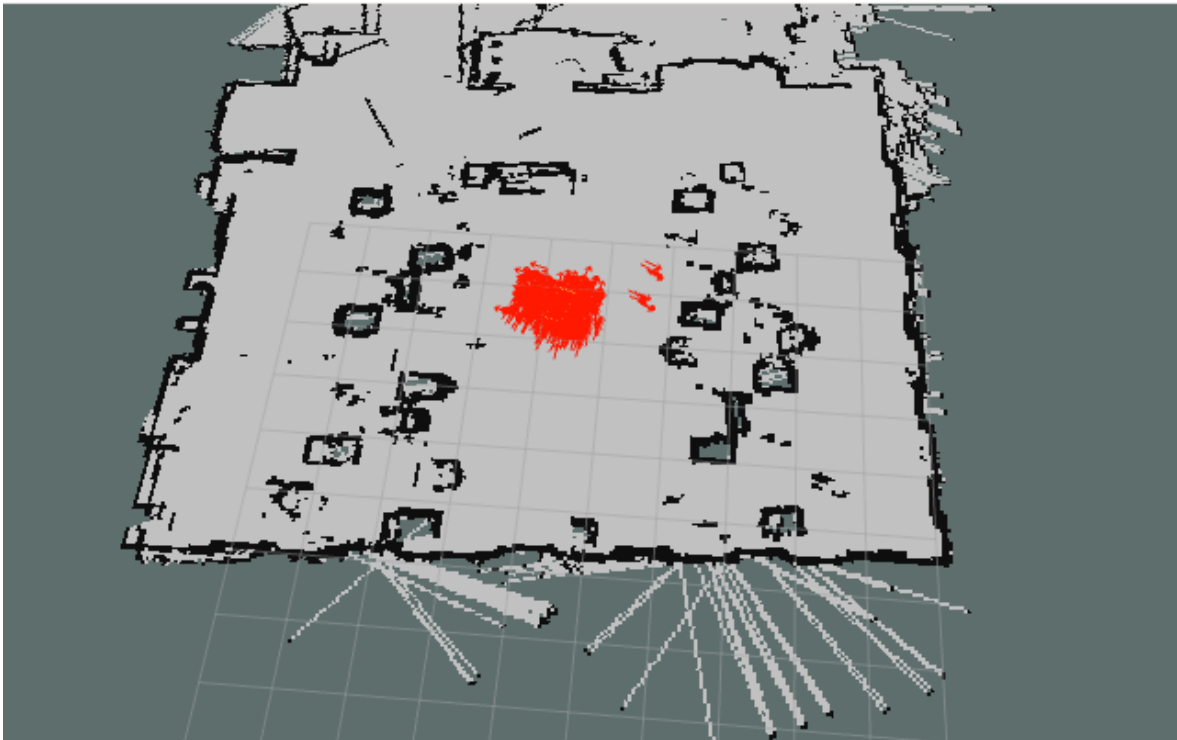


Abbildung 34: Darstellung der AMCL-Partikel nach mehreren Iterationen

3.5.2 Pfadplanung

Neben der Lokalisierung spielt die Pfadplanung eine große Rolle für die automatisierte Navigation. Der globale Pfadplaner findet eine initiale kürzeste Route vom Start zum Ziel, während der lokale Pfadplaner auf Hindernisse reagiert und die Steuerkommandos für den Antrieb generiert. Als Pfadplaner wird die ROS-Toolbox `teb_local_planner` eingesetzt. Vor der Verwendung müssen noch einige Konfigurationsdateien angelegt werden. Die Dateien `local_costmap_params.yaml`, `global_costmap_params.yaml` und `costmap_common_params.yaml` beinhalten Parameter bezüglich der Occupancy Grid Map, die die Karte repräsentiert. Die YAML-Datei `base_local_planner_params.yaml` ist am wichtigsten, da hierüber die meisten Anpassungen vorgenommen werden müssen. Der Inhalt aller Konfigurationsdateien ist im Anhang unter Abschnitt 4 zu finden. In Tabelle 6 sind ausgewählte Parameter aufgelistet, die angepasst wurden, um eine möglichst flüssige Navigation zu erhalten.

Tabelle 6: Auflistung ausgewählter Pfadplanungsparameter

Parameter	Wert
max_vel_x	0.4
max_vel_x_backwards	0.2
max_vel_theta	0.3
acc_lim_x	0.5
acc_lim_theta	0.5
min_turning_radius	0.8
wheelbase	0.33
cmd_angle_instead_rotvel	True
xy_goal_tolerance	0.2
yaw_goal_tolerance	0.1
min_obstacle_dist	0.2
enable_homotopy_class_planning	True

Die Parameter in Zeile eins bis fünf beziehen sich auf Geschwindigkeits- und Beschleunigungslimits, um eine sichere Navigation zu gewährleisten. Des Weiteren würden höhere Limits mehr Rechenleistung benötigen, da die Pfadoptimierung schneller durchgeführt werden müsste. Ein Erreichen eines Limits verursacht im TEB-Algorithmus (s. Kapitel 2.8.1) sofort einen hohen Wert in der Kostenfunktion des Optimierers. Ein weiterer wichtiger Wert ist der minimale Wendekreis des Roboters. Durch die Verwendung der Ackermann-Lenkung ist der maximal fahrbare Kurvenradius limitiert. Dieser Wert wird vom lokalen Pfadplaner benötigt, um einen Weg um Hindernisse herum berechnen zu können. Der minimale Wendekreis lässt sich nach [60, S. 36] ermitteln:

$$R = \frac{L}{\sin \alpha} \quad (3.1)$$

L gibt den Radstand an. Dieser beträgt in Bezug auf das verwendete Modellauto 0.33 m. Unter Verwendung eines maximalen Lenkwinkels von 25° ergibt sich schließlich mit Gleichung (3.1) ein Wendekreis von ca. 0.8 m. Dieser Wert konnte ebenfalls durch eine manuelle Messung bestätigt werden.

Die Variable `cmd_angle_instead_rotvel` muss auf `True` gesetzt werden, sodass der Pfadplaner weiß, dass es sich bei dem verwendeten Roboter um ein Fahrzeug mit Vorderachslenkung handelt. Die beiden Parameter `xy_goal_tolerance` und

yaw_goal_tolerance beziehen sich auf die Toleranz zum Ziel im Hinblick auf die Orientierung und Positionierung des Fahrzeugs. Da die reale Umwelt immer gewisse Unsicherheiten mitbringt und die erstellte Karte diese nicht komplett wahrheitsgetreu widerspiegeln kann, müssen diese Zieltoleranzen ausreichend groß gewählt werden. Zuletzt muss noch der minimale Abstand, der zu Hindernissen eingehalten werden soll (*min_obstacle_dist*) angegeben werden. Wird dieser Wert zu niedrig gewählt, steigt die Wahrscheinlichkeit für Kollisionen. Wird hingegen ein zu hoher Wert angenommen, kann der Roboter stecken bleiben, da von allen Seiten die Hindernisse bereits als kritisch nah betrachtet werden. Zusätzlich empfiehlt es sich den Parameter *enable_homotopy_class_planning* auf *True* zu setzen, um die Online Trajektorien-Optimierung (s. Kapitel 2.8.2) zu aktivieren. Dadurch steigt zwar die Rechenleistung an, es kann aber adaptiver auf Hindernisse reagiert werden.

Abbildung 35 zeigt beispielhaft eine Pfadplanung, bei der kein Hindernis aktiv umfahren werden muss. Der blau dargestellte Pfad entspricht somit dem globalen Pfad, der mit dem A*-Algorithmus (s. Kapitel 2.7.2) generiert wurde. Die pinken Flächen stellen die lokale Kostenkarte dar. Diese Objekte werden vom lokalen Pfadplaner zur aktuellen Ausführungszeit berücksichtigt.

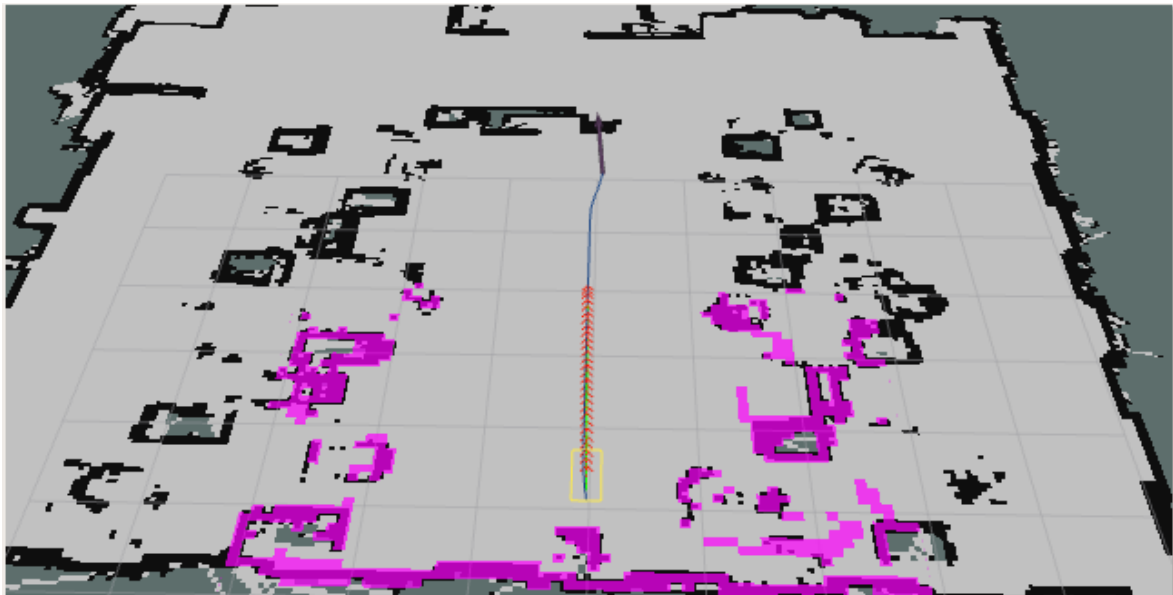


Abbildung 35: Beispielhafte Pfadplanung ohne Hindernis

Als nächstes ist in Abbildung 36 hingegen eine Pfadplanung mit Hindernis dargestellt. Dabei ist schön zu sehen, wie sich der lokale Pfad wie ein elastisches Band um das Hindernis herum verbiegt. Es muss dabei immer der vorher festgelegte Mindestabstand zu erkannten

Objekten eingehalten werden. Ist das Hindernis umfahren, wird die Trajektorie so geplant, dass möglichst schnell auf den globalen Pfad zurückgekehrt wird.

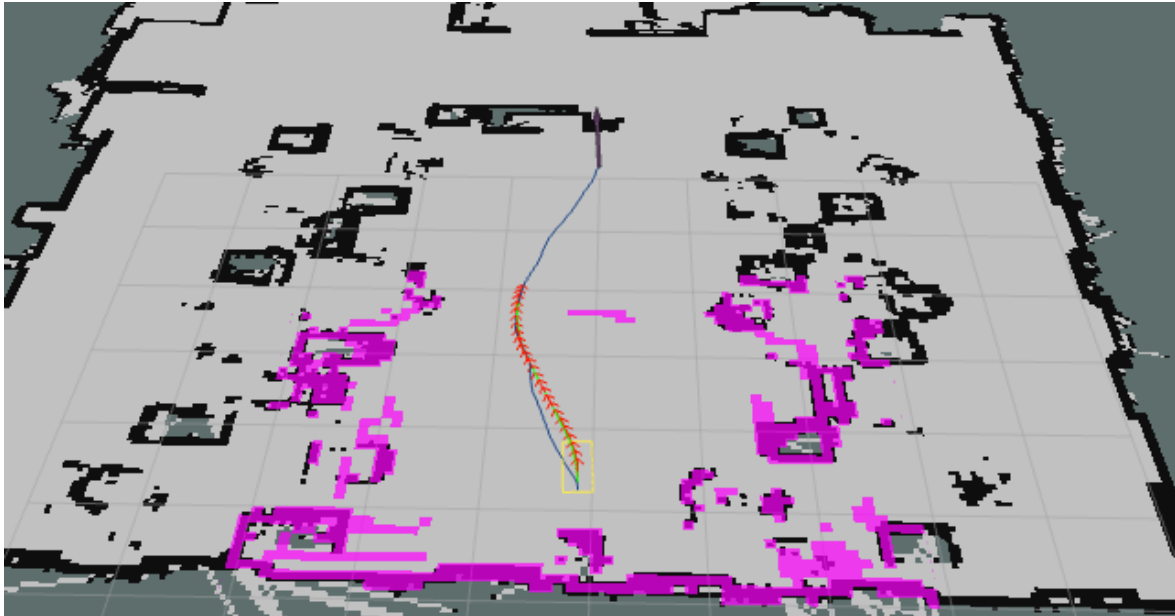


Abbildung 36: Beispielhafte Pfadplanung mit Hindernis

3.5.3 Navigation

In diesem Kapitel wird der komplette Navigationsablauf beleuchtet. Dadurch soll vor allem das Zusammenspiel der bisher beschriebenen Komponenten nochmals genauer verdeutlicht werden. Abbildung 37 zeigt den kompletten Aufbau der Navigationsroutine.

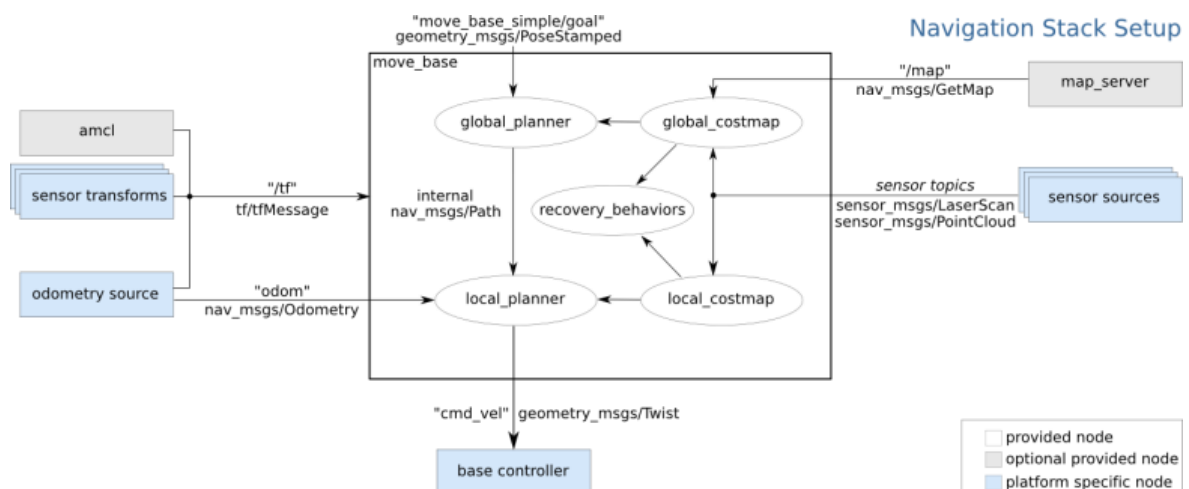


Abbildung 37: Übersicht des Navigationsablaufs [61]

Auf der linken Seite befindet sich der AMCL-Knoten, der mithilfe von externen Sensordaten (LiDAR) und Odometrienachrichten die Lokalisierung durchführt. Auf der rechten Seite befindet sich der Map-Server, der eine vorher aufgezeichnete Karte lädt und zur Verfügung stellt. Aus dieser Karte wird eine globale Costmap erzeugt, die der globale Planer zur Pfadplanung benutzt. Unterhalb des globalen Pfadplaners ist der lokale Planer angesiedelt. Dieser verwendet eine lokale Costmap, die aktuelle Laserscanner-Daten fusioniert. Daraus werden schließlich die Geschwindigkeitssignale (*cmd_vel*) generiert und an den Regler weitergegeben.

Standardmäßig verwendet der ROS-Navigation-Stack das Differential-Drive-Model für Roboter. Das heißt es bezieht sich auf Roboter mit zwei Rädern, die mit unterschiedlichen Geschwindigkeiten angesteuert werden und dadurch eine Drehung ermöglichen. In diesem Fall wird jedoch ein Fahrzeug mit Ackermannlenkung verwendet. Aus diesem Grund müssen die ursprünglichen Signale für den Geschwindigkeitsregler noch umgerechnet werden. Das bedeutet konkret, dass aus der Winkelgeschwindigkeit ein Lenkwinkel berechnet wird. Dafür ist der Python-Knoten *cmd_vel_to_ackermann_cmd.py* (s. Anhang Abschnitt 4) verantwortlich. Der Ausgang aus diesem Knoten ist somit eine Nachricht vom Typ *ackermann_msgs/AckermannDriveStamped Message*. Deren Aufbau ist in Tabelle 7 nach [62] zu sehen.

Tabelle 7: Aufbau der Nachricht *AckermannDriveStamped Message* nach [62]

Datenfeld	Beschreibung
Header header	Gibt den Zeitstempel der ersten Messung an und die <i>frame_id</i> des Koordinatensystems
AckermannDrive drive float32 steering_angle float32 speed	Gibt die Bewegungsbefehle für einen Roboter mit Ackermannlenkung an Lenkwinkel Geschwindigkeit

Der Aufbau der Nachricht ist simpel gehalten. Neben dem üblichen Signalheader werden zwei Gleitkommawerte für den Lenkwinkel und die Geschwindigkeit übertragen. Die Geschwindigkeit wird dann über die E-Maschine des Antriebs umgesetzt und der Lenkwinkel durch den Servomotor (s. Kapitel 3.1) gestellt.

Alle beschriebenen Teilkomponenten des Navigation-Stacks werden mit dem zentralen Launch-File *move_base.launch* gestartet. Dessen Aufbau ist im Anhang unter Abschnitt 4 zu finden.

3.6 Erstellung von Routen

Wie bereits in der Einleitung erwähnt, ist es das Ziel das Fahrzeug einfache Transportaufgaben im Sinne eines Service-Roboters übernehmen zu lassen. Das Festlegen des Start- und Zielpunktes kann zum einen über die Visualisierungsplattform rviz (s. Abbildung 28) erfolgen. Zum anderen können die Zielvorgaben auch über Python mithilfe der Bibliothek rospy angesteuert werden. Das Erstellen von Routen kann dadurch automatisiert und benutzerfreundlich gestaltet werden, da der Anwender keine tieferen Kenntnisse in Bezug auf ROS benötigt. Die entwickelte grafische Oberfläche ist in Abbildung 38 zu sehen.

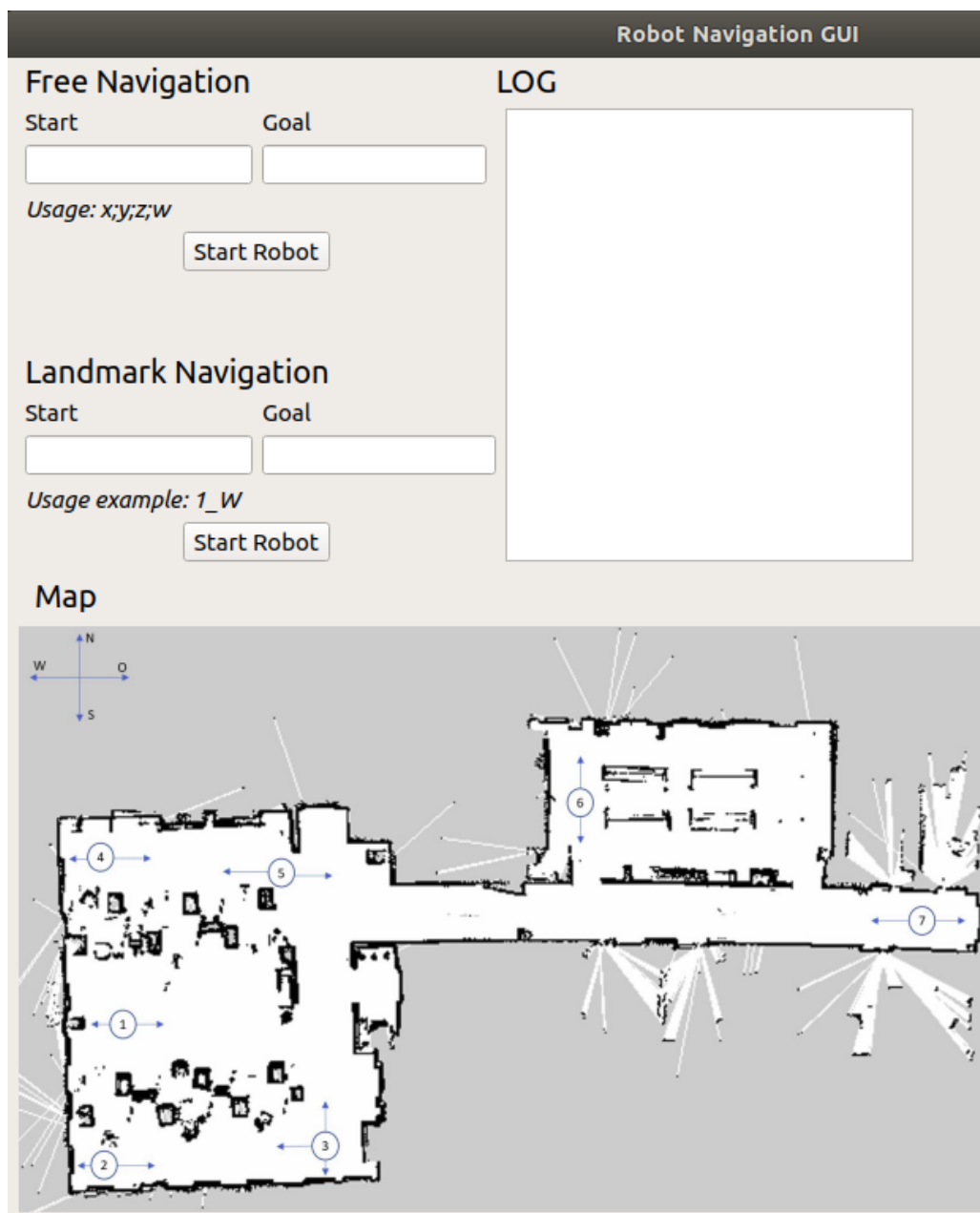


Abbildung 38: Darstellung der GUI zur automatisierten Navigation

Im oberen Bereich der GUI können Start- und Zielpunkt eingegeben werden. Zum einen unter dem Punkt „Free Navigation“ als Pose-Koordinaten. x und y beziehen sich dabei auf die Koordinaten im zweidimensionalen Kartenraum und z und w sind Parameter eines Quaternions, das die Orientierung des Roboters spezifiziert (s. Tabelle 2). Darunter unter „Landmark Navigation“ können zum anderen festgelegte Wegpunkte eingegeben werden. Diese sind über das Büro so verteilt, dass jeder Teil angefahren werden kann. Die Wegpunkte sind im unteren Bereich der grafischen Benutzeroberfläche zu finden und mit Nummern gekennzeichnet. Die Pfeile geben jeweils an in welche Richtung der Roboter am Ende der Navigation orientiert sein soll. Die Pfeilrichtung entspricht den vier Himmelsrichtungen (s. oben links in der GUI). Eine gültige Eingabe ist beispielsweise 1_O. Somit muss der Benutzer nur den richtigen Wegpunkt aus der Karte heraussuchen und eingeben. Die Navigation wird dann über den Knopf „Start Robot“ gestartet. Im Feld „LOG“ wird schlussendlich ausgegeben, ob die Navigation erfolgreich war oder nicht.

Der Python-Code der GUI ist im Anhang unter Abschnitt 5 zu finden. Die Oberfläche ist mithilfe des Grafikframeworks Qt implementiert. Um Start und Ziel festlegen zu können, müssen die entsprechenden ROS-Nachrichten `PoseWithCovarianceStamped` und `MoveBaseGoal` noch importiert werden. Die Wegpunkte sind in Form eines Python-Dictionarys abgelegt. Insofern wird eine Route nur ausgeführt, wenn die entsprechenden Punkte auch darin existieren. Die Koordinaten der Wegpunkte wurden über den ROS-Knoten AMCL ermittelt. Dieser schätzt die Transformation des Base-Frames des Roboters zum Map-Frame, also den Kartenkoordinaten (s. Abbildung 32). Somit muss mit rviz lediglich auf der Karte ein Punkt angeklickt werden und die Koordinaten werden über das AMCL-Topic `amcl_pose` veröffentlicht. Nachdem die initiale Startpose gesetzt wurde, wird die Ausführung der Route über einen `SimpleActionClient` der ROS-actionlib gestartet. Der Client gibt dann automatisch Rückmeldung, ob die Ausführung der Route erfolgreich war oder abgebrochen wurde.

4. Ergebnisse und Diskussion

In diesem Kapitel soll nun die entwickelte Navigationslogik erprobt werden. Dabei werden als erstes Routen ohne zusätzliche Hindernisse abgefahren. Das heißt es wird auf die Hindernisse, die in der Karte erfasst wurden, beschränkt. Im folgenden Schritt wird die Navigation dann um statische und auch dynamisch bewegte Objekte erweitert. Abschließend wird noch darauf eingegangen, wo die Grenzen der automatisierten Navigation liegen.

4.1 Navigation ohne zusätzliche Hindernisse

Als erstes soll beurteilt werden, wie gut das Modellauto sich in der aufgenommenen Karte zurechtfinden kann. Dafür muss die Lokalisierung mit AMCL, Odometrie und die Umsetzung der Steuerkommandos via lokalen Pfadplaner funktionieren. Exemplarisch ist in Abbildung 39 eine abgefahrte Route vom Ende des Büros zur Küche dargestellt.

Die blaue Linie stellt den globalen Pfad vom Startpunkt des Roboters bis zum Ziel dar. Es fällt auf, dass in der Mitte des Pfades und am Schluss in der Küche unnötige Kurven eingebaut werden. Diese resultieren aus Unsicherheiten der Karte. In der Mitte des Ganges befinden sich in der Realität keine Hindernisse. Während der Kartenerstellung mittels SLAM-Algorithmus kann es jedoch sein, dass einige Lichtstrahlen des LiDARs reflektiert wurden und deswegen folglich Objekte an diesen Stellen angenommen werden. Die roten Pfeile repräsentieren die geplanten Posen des lokalen Pfadplaners. Dieser versucht beispielsweise die erste Kurve etwas runder zu fahren und holt etwas mehr aus. Dadurch kann eine höhere Kurvengeschwindigkeit gefahren werden, da ein kleinerer Lenkwinkel ausreicht. Die höhere Geschwindigkeit führt wiederum zu einer kürzeren Ankunftszeit am Ziel. Dieser Parameter gehört zu den Optimierungswerten des TEB-Planers.

Zuletzt sind die pinkfarbenen Bereiche noch zu erwähnen. Diese repräsentieren die lokale Kostenkarte des Pfadplaners. Darin werden alle Objekte in der näheren Umgebung des Roboters berücksichtigt. Diese decken sich gut mit den Objekten in der Karte. Das bedeutet, dass der LiDAR seine Umwelt adäquat erfasst und die Lokalisierung innerhalb der Karte ebenfalls erfolgreich ist.

Das Abfahren dieser Route zeigt also, dass das Navigieren um Kurven oder gradeaus bereits sehr gut möglich ist. Es kommt zu keiner Kollision mit den Hindernissen, die in der Karte verzeichnet sind. Es bleibt lediglich anzumerken, dass die Umsetzung der

Geschwindigkeit teilweise etwas ruckelig wirkt. Dies liegt an der starken Auslastung der CPU des NVIDIA Jetson TX2. Die Frequenz der Steuersignale des lokalen Pfadplaners ist auf 5 Hz gesetzt. Die Debug-Nachrichten zeigen jedoch, dass die Zykluszeit von 200 ms nicht eingehalten werden kann. Wäre der Rechner leistungsfähiger, könnte die Frequenz sogar erhöht werden und es würde sich ein weicherer Steuerverlauf ergeben.



Abbildung 39: Darstellung einer Route ohne zusätzliche Hindernisse

4.2 Navigation mit zusätzlichen Hindernissen

Als nächstes wird die Navigationsaufgabe des Fahrzeugs erschwert, indem zusätzliche Hindernisse eingeführt werden. Zunächst soll der Roboter statischen Objekten ausweichen und später auch dynamisch bewegte Hindernisse erkennen und alternative Pfade finden.

4.2.1 Statische Hindernisse

Statische Hindernisse sind Objekte, die immer an ihrem Ort verweilen. Zusätzlich zu den statischen Objekten, die mit der Karte bereits erfasst sind, werden dem Roboter nun weitere Hindernisse in den Weg gestellt. Dies hat hohe Praxisrelevanz, da auch in der Realität ungewollte Gegenstände auf der Fahrbahn liegen können. In diesem Fall muss unbedingt eine Kollision vermieden werden. Anstatt die Routenausführung abubrechen, soll das Fahrzeug die Hindernisse so gut wie möglich umfahren und dann auf den initialen Pfad zurückkehren, um den Zielwegpunkt zu erreichen.

Abbildung 40 zeigt die Trajektorienplanung um drei Hindernisse herum. Die Störobjekte sind versetzt aufgestellt und jeweils mit einer blauen Ellipse markiert.

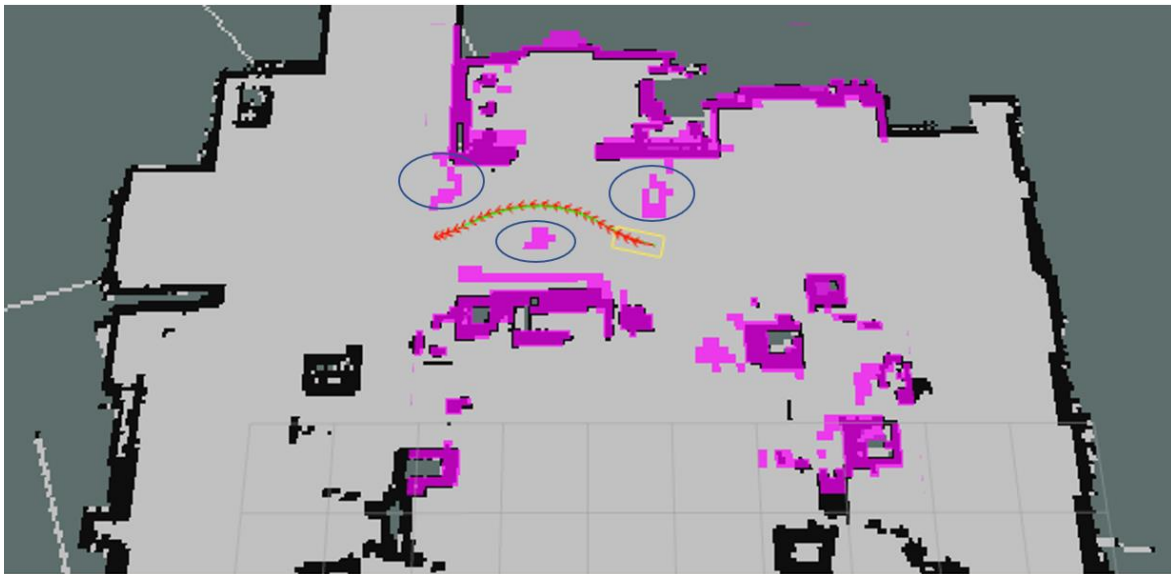


Abbildung 40: Abbildung der Trajektorie um drei statische Hindernisse

Es ist zu sehen, dass der optimale Pfad gefunden wird. Die Trajektorie verbiegt sich wie ein elastisches Band um die Objekte herum. Es wird dabei auch nicht zu viel Abstand gehalten oder ein unnötiger Umweg gefahren, da dies die zeitliche Laufzeit der Zielerreichung negativ beeinflussen würde.

4.2.2 Dynamisch bewegte Hindernisse

Als nächste Schwierigkeitsstufe soll das Fahrzeug jetzt Hindernissen, die sich im Raum dynamisch bewegen ausweichen. Dies ist in der Realität vor allem wichtig, wenn das automatisierte Fahren auf Stadtbereiche ausgeweitet wird. Dort gibt es beispielsweise spielende Kinder, Fußgänger oder Radfahrer, die sich mit großer Dynamik fortbewegen. Um diesen Anforderungen gerecht zu werden, müssen autonome Systeme Objekte frühzeitig erkennen und ihre Trajektorie richtig einschätzen. Außerdem muss sehr schnell auf neue Situationen reagiert werden, um Kollisionen zu vermeiden.

In dieser Arbeit wird die Komponente der Online-Trajektorien-Optimierung des Timed Elastic Band-Algorithmus ausgenutzt (s. Kapitel 2.8.2). Wie bereits beschrieben werden bei diesem Ansatz fortlaufend während der Navigation alternative Pfade mithilfe von Homologie-Klassen berechnet. Dadurch muss im Falle einer drohenden Kollision nur der beste alternative Pfad ausgewählt und umgesetzt werden. Ein großer Vorteil ist dabei, dass die alternativen Trajektorien bereits alle berechnet sind. Somit ist ein sehr schnelles Umschalten des Roboters möglich.

In Abbildung 41 ist die ausgeführte Route um ein bewegtes Objekt herum zu sehen. Das Hindernis ist in diesem Fall ein Bürorollcontainer, der von der Wand langsam in den Gang hineinbewegt wird. Das Objekt ist mit einer blauen Ellipse gekennzeichnet und die Bewegungsrichtung ist durch den blauen Pfeil angegeben.



Abbildung 41: Trajektorienplanung um ein bewegtes Objekt von der Seite

Der Roboter erkennt das Objekt mit seiner Bewegungsrichtung frühzeitig und plant somit eine alternative Navigationsroute, die sich links vom Rollcontainer befindet. Dadurch kann dem Hindernis rechtzeitig ausgewichen werden und eine Kollision wird vermieden.

Im nächsten Szenario soll sich ein Fußgänger von vorne auf das Fahrzeug zubewegen und so eine Korrektur der Route bewirken. Dies ist in Abbildung 42 dargestellt.

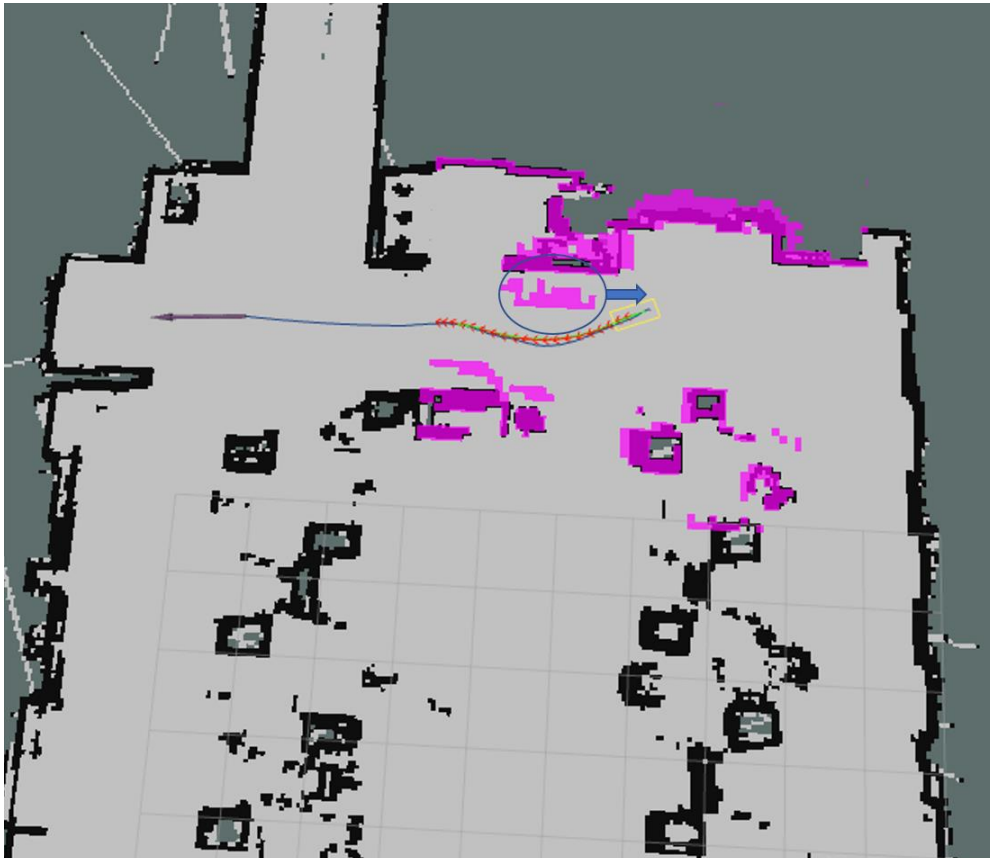


Abbildung 42: Trajektorienplanung um ein bewegtes Objekt von vorne

Auch in diesem Fall wird dem Fußgänger erfolgreich ausgewichen. Es ist allerdings zu erwähnen, dass die Geschwindigkeit des Fußgängers und die Beschleunigungs- und Geschwindigkeitslimits des Roboters (s. Tabelle 6) einen großen Einfluss auf den Erfolg des Fahrmanövers haben. Nähert sich der Fußgänger zu schnell, kann es sein, dass das Fahrzeug nicht genügend Zeit zum Ausweichen hat. Würden die Geschwindigkeitslimits des Roboters angepasst werden, könnte dem entgegengewirkt werden. Jedoch stellt eine höhere Geschwindigkeit auch ein höheres Risiko für die Navigation in geschlossenen Räumen dar, da das Auto von Personen schneller übersehen wird und es folglich zu Unfällen kommen kann. Des Weiteren arbeitet die CPU des NVIDIA Jetson TX2 sowieso bereits an der Lastgrenze. Das bedeutet höhere Geschwindigkeits- und

Beschleunigungslimits können nicht komplett ausgenutzt werden, da der Rechner die Eingangsdaten schneller verarbeiten müsste. Das heißt eine alternative Ausweichroute müsste schneller berechnet und umgesetzt werden. Somit stellen die in Tabelle 6 gefundenen Parameter einen Kompromiss dar, um zum einen mit der vorhandenen Rechenkapazität die Trajektorienplanung in einer angemessenen Zeit umzusetzen. Zum anderen sind die Limits so gewählt, dass Personen, die sich mit moderater Geschwindigkeit nähern erkannt werden und ein Ausweichmanöver initiiert wird. Außerdem darf die maximale Geschwindigkeit und Beschleunigung des Systems aus sicherheitstechnischen Aspekten in geschlossenen Räumen nicht zu hoch gesetzt werden, um Arbeitsunfälle zu vermeiden.

4.3 Einnahme des sicheren Zustands bei Blockierung

In der Einleitung der Arbeit wurde bereits erwähnt, dass es ein Kennzeichen des automatisierten Fahrens der Stufe 4 ist, wenn das Fahrzeug in Situationen, in denen es selbst keine Lösung findet, dem Benutzer eine Übernahmeaufforderung gibt. Kommt der Benutzer dieser Aufforderung nicht nach, so wird der sichere Zustand eingenommen. Das bedeutet das Fahrzeug wird an einem passenden Ort in den Stand gebremst und abgestellt.

In dieser Arbeit wird dieses Verhalten simuliert, indem der Roboter mit Hindernissen eingesperrt wird, sodass es keinen validen Pfad zum Ziel gibt. Dieses Szenario ist in Abbildung 43 zu sehen.

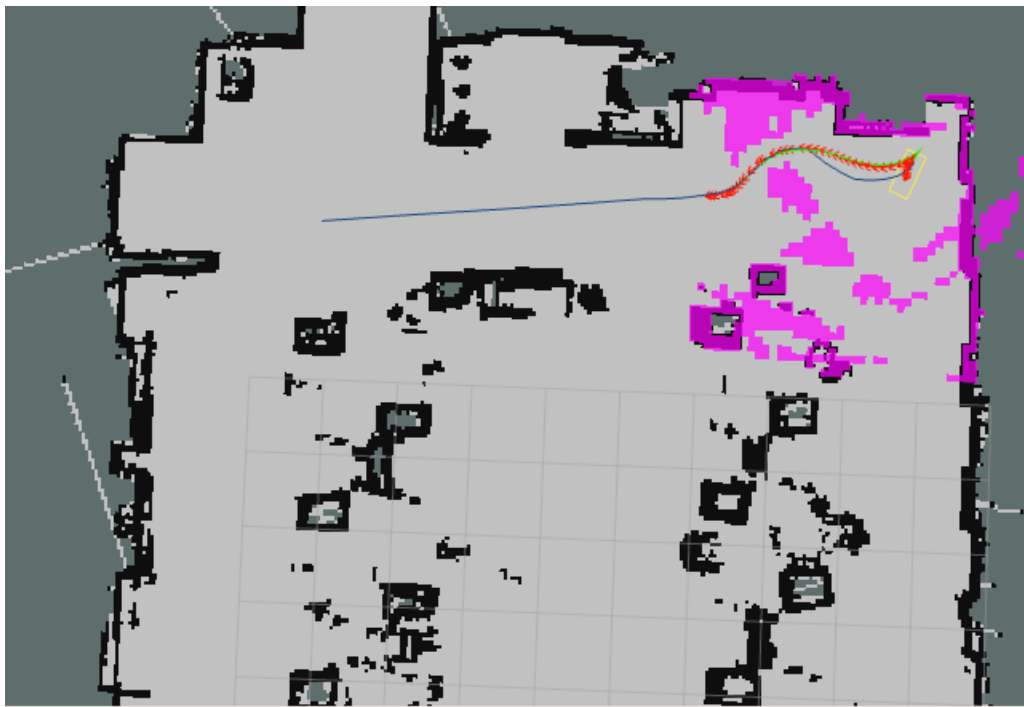


Abbildung 43: Einnahme des sicheren Zustands bei Blockierung

Die pinken Flächen der lokalen Kostenkarte repräsentieren die Hindernisse, die den Weg in alle Richtungen versperren. Abbildung 43 zeigt zwar einen geplanten Pfad, dieser wird allerdings nach anfänglichen Fahrversuchen nicht umgesetzt, da der lokale Pfadplaner den spezifizierten Mindestabstand zu den Objekten nicht einhalten kann. Dies wird dem Nutzer nach einigen Sekunden in der Terminalausgabe des ROS-Knotens mitgeteilt. Es wird also nicht versucht den Hindernissen um jeden Preis auszuweichen, sondern es wird erkannt, dass es für die gegenwärtige Fahrsituation keine gültige Lösung gibt. Folglich wird die Navigation eingestellt und der sichere Zustand eingenommen. Dieser besteht in der Umsetzung des Pfadplaners darin keine Antriebsmomente mehr umzusetzen.

Abschließend kann also gesagt werden, dass das Fahrzeug den Sicherheitsanforderungen bezüglich nicht umsetzbarer Navigationsrouten gerecht wird. Das Auto nimmt den sicheren Zustand ein und vermeidet potenzielle Unfälle.

4.4 Grenzen der Navigation

In den letzten Abschnitten wurden alle Fahrsituationen dargestellt, die von dem Roboter zufriedenstellend gemeistert werden. Dazu gehört die freie Navigation in der erstellten Karte, das Ausweichen von statischen und dynamischen Hindernissen und zuletzt die Einnahme des sicheren Zustands bei keinem validen Pfad. Nun wird auf Szenarien eingegangen, in denen das Fahrzeug kein optimales Verhalten gezeigt hat und somit die Grenzen der machbaren Navigation mit dem aufgebauten System darstellen.

4.4.1 Pfadfindung

Der vom lokalen Planer gefundene Pfad vom Start zum Ziel stellt sich nicht immer als optimal heraus. In Abbildung 44 ist eine Route im Büro zu sehen, die direkt durch Tische und Stühle führt. Diese Trajektorie wird als am kürzesten betrachtet, da nicht alle Büromöbel mit dem Laserscanner während der Kartenerstellung erfasst wurden. Bei der Montagehöhe des LiDARs musste ein Kompromiss eingegangen werden. Zum einen, darf er nicht zu niedrig montiert sein, da sonst beispielsweise Bürostühle, wegen ihrer dünnen Haltestange nicht erkannt werden. Zum anderen darf der Laserscanner auch nicht zu hoch angebracht werden, da sonst niedrige Objekte nicht mehr erfasst werden und Kollisionen wahrscheinlicher werden. Insofern stellt die aktuelle Montagehöhe von ca. 45 cm einen Mittelweg dar, um möglichst viele verschiedene Hindernisse erkennen zu können. Der Roboter erkennt schließlich, dass der Pfad zu eng um Hindernisse führen würde und bricht die Ausführung ab.

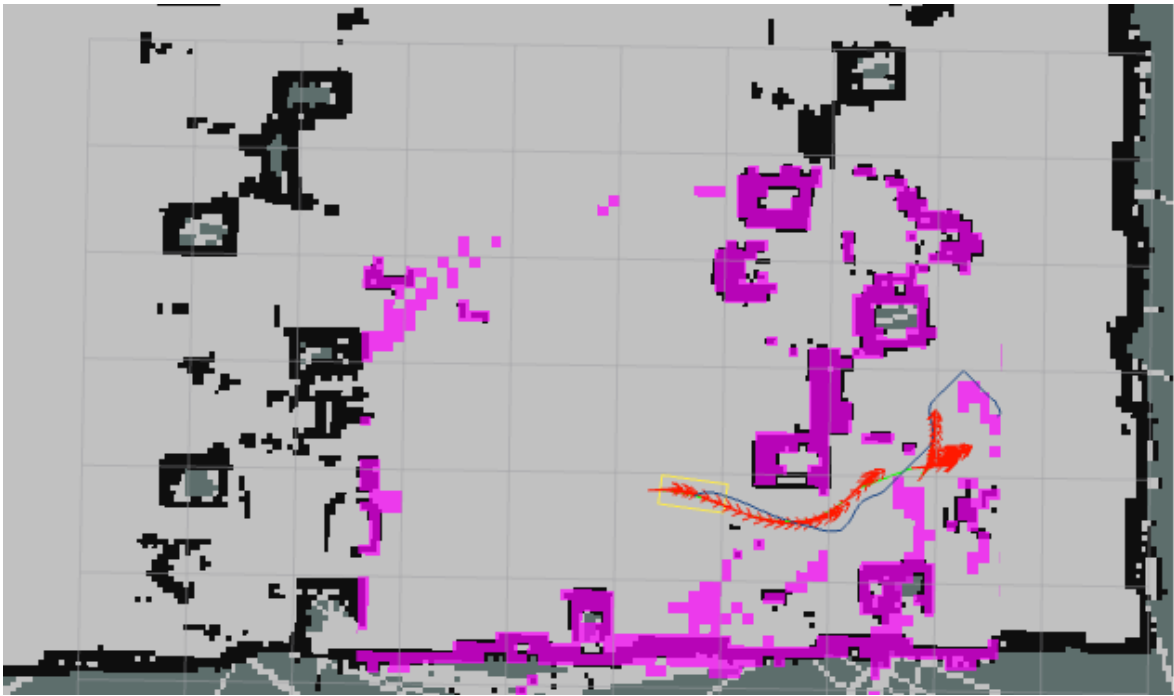


Abbildung 44: Planung eines nicht optimalen Pfades durch Hindernisse hindurch

4.4.2 Wendemanöver

Des Weiteren bereitet das Rückwärtsfahren, das bei Wendemanövern benötigt wird, große Probleme. Wie in Abbildung 45 dargestellt ist, wird ein Pfad inklusive Wende geplant, allerdings kann das Rückwärtsfahren nicht umgesetzt werden. Der Planer generiert negative Geschwindigkeitssignale, diese sind jedoch zu gering, sodass sie das Motorsteuergerät nicht realisieren kann. Ein Anpassen des maximalen Geschwindigkeitslimits hat nicht zum Erfolg geführt. Außerdem hat auch das künstliche Erhöhen der Geschwindigkeit, nachdem das Signal vom Planer generiert wird, zu keiner Verbesserung geführt. Der Roboter fährt zwar dadurch rückwärts, aber der Planer wird dadurch verwirrt, weil das Signal extern verfälscht wird. Somit kennt sich der lokale Pfadplaner nicht mehr aus und fährt eine falsche Route ab.

Mögliche Ursache für das fehlerhafte Verhalten könnte der LiDAR sein, der einen Blickwinkel von 240° Grad besitzt. Das heißt er ist blind bezüglich der Umgebung direkt hinter dem Fahrzeug. Durch diese Unsicherheit ist es möglich, dass der Planer behindert wird und keine sicheren Geschwindigkeitssignale generiert.



Abbildung 45: Darstellung eines geplanten Wendemanövers

4.4.3 Objekterkennung

Zuletzt gibt es auch Limitierungen bezüglich der Objekterkennung und der Reaktion des Roboters auf Hindernisse. Wie bereits erwähnt bildet der zweidimensionale Laserscanner nur einen Schnitt durch den Raum ab. Somit werden Hindernisse, die unter oder über der Schnittebene liegen nicht erfasst, was zu Kollisionen oder nicht optimalen Trajektorien führt. Des Weiteren ist die Reaktionszeit des Fahrzeugs durch die Rechenkapazität des NVIDIA Jetson TX2 beschränkt. Bewegen sich dynamische Hindernisse zu schnell auf das Fahrzeug zu, bleibt nicht genügend Zeit, um eine alternative Route zu berechnen und auch umzusetzen. Diese Limitierung liegt somit allerdings am System und nicht an der Implementierung der Algorithmen.

Am Schluss bleibt noch zu erwähnen, dass es teilweise zur Detektion von Geisterobjekten kommt. Diese entstehen, wenn Lichtstrahlen des LiDARs diffus reflektiert werden und somit in die Empfangsapparatur des Laserscanner zurückgelangen. Dadurch werden manchmal unnötige Ausweichmanöver gefahren oder im schlimmsten Fall wird keine valide Route gefunden, weil der Roboter sich für festgefahren hält.

5. Fazit und Ausblick

In dieser Arbeit wurde an einem Miniatur-Modellauto eine Fahrfunktion nach Vorbild der Stufe vier des Standards J3016 der SAE (s. Abbildung 4) umgesetzt. Ziel war es das Fahrzeug als firmeninternen Demonstrator zu verwenden, um im Hinblick auf Fahrerassistenzsysteme zum einen Wissen zu erlangen und zum anderen das Thema autonomes Fahren näher zu bringen und dafür zu sensibilisieren.

Zu Beginn wurde ein neuer 2D-Laserscanner angeschafft und auf dem Fahrzeug montiert. Nach der Inbetriebnahme konnte mit einem SLAM-Algorithmus eine Karte der Büroräumlichkeiten erstellt werden. Auf Basis dieser Karte und mithilfe der Odometriedaten und LiDAR-Messungen konnte schließlich die automatisierte Fahrfunktion umgesetzt werden. Dabei wurde die Lokalisierung des Roboters mit dem AMCL-Algorithmus realisiert, der die Odometrieinformationen mit den LiDAR-Scans fusioniert. Die initiale globale Pfadplanung ist durch den A*-Algorithmus gegeben. Die weitere Umsetzung der Trajektorie inklusive einer Umplanung der Route bei Hindernissen wurde mit dem TEB-Local-Planner implementiert. Mit diesem ist es möglich sowohl statischen, also auch dynamischen Hindernissen auszuweichen. Zuletzt wurde noch ein graphisches Userinterface programmiert, mit dem man benutzerfreundlich vorher festgelegte Wegpunkte innerhalb der Karte ansteuern kann. Damit kann der Roboter als Transportfahrzeug innerhalb des Büros genutzt werden.

Fahrexperimente haben gezeigt, dass eine Navigation in der Karte problemlos umgesetzt wird. Eine Erweiterung der Umgebung mit statischen und dynamischen Hindernissen hat ebenfalls zu positiven Ergebnissen geführt. Solange die statischen Hindernisse nicht zu eng aufgestellt sind, ist ein Ausweichen möglich. Ansonsten wird die Route abgebrochen da immer ein Sicherheitsabstand zu Objekten eingehalten werden muss. Im Hinblick auf dynamisch bewegte Objekte wird eine neue Route frühzeitig geplant und umgehend ausgeführt. Dabei dürfen sich die Objekte nicht zu schnell bewegen, da der Roboter durch seine Verarbeitungsgeschwindigkeit limitiert ist. Des Weiteren wird in Situationen, in denen kein gültiger Pfad existiert der sichere Zustand eingenommen und an den Benutzer die Steuerung übergeben.

Einschränkungen sind durch den 2D-LiDAR gegeben, da dieser nur eine Schnittebene des Raumes abbildet und somit nicht alle Hindernisse erfasst werden. Dadurch kommt es teilweise zu Pfadplanungen, die nicht umgesetzt werden können oder Kollisionen. Außerdem ist das Rückwärtsfahren durch den Pfadplaner nicht umsetzbar, da dieser womöglich durch das eingeschränkte Blickfeld des Laserscanners behindert wird. Zuletzt

bleibt noch zu erwähnen, dass durch die Verarbeitungsgeschwindigkeit des NVIDIA Jetson ebenfalls Einschränkungen entstehen. Dadurch muss die Robotergeschwindigkeit limitiert werden und alle Objekte dürfen sich nicht zu schnell bewegen. Außerdem wirkt die Umsetzung der Motorsteuersignale mitunter etwas ruckelig, da nur eine relativ große Zykluszeit verwendet werden kann.

Insgesamt lässt sich sagen, dass die automatisierte Fahrfunktion nach Vorbild der Stufe vier erfolgreich im vorgegebenen Szenario umgesetzt wurde. Innerhalb der Systemgrenzen funktioniert die Navigation inklusive dem Ausweichen von Hindernissen sehr gut. Außerdem konnte viel Wissen in Bezug auf die Umsetzung von hochautomatisierten und vollautomatisierten Fahrfunktionen gesammelt werden. Dieses ist im Entwicklungsumfeld der Automobilindustrie sehr wertvoll. Aktuell arbeitet beispielsweise BMW daran die neue Generation des 7er mit einer Level 3-Funktion auszustatten [63]. Damit ist das generierte Wissen für ARRK Engineering nützlich, um weiterhin als erfolgreicher Entwicklungspartner zu agieren.

Für zukünftige Projekte im Zusammenhang mit dem autonomen Modellauto können folgende Punkte in Betracht gezogen werden. Zum einen kann durch eine Hardwareoptimierung des NVIDIA Jetson eine bessere Performanz erzielt werden, indem auf das Nachfolgemodell zurückgegriffen wird. Somit kann vom Fahrzeug eine höhere Geschwindigkeit realisiert werden und es kommt zu einer flüssigeren Interaktion mit der Umgebung. Es ist ebenfalls denkbar in diesem Kontext die Navigation um einen Rennparcour zu optimieren, bei dem die Rundenzeit ein wichtiger Regelparameter ist. Zum anderen würde die Verwendung eines hochwertigeren LiDARs mit größerer Reichweite, höherer Scanrate und besserer Verarbeitung zu wirksameren Ergebnissen führen. Damit wären unter anderem auch Outdoor-Anwendungen denkbar. Des Weiteren könnte auch die Verwendung eines 3D-Laserscanners in Betracht gezogen werden. Damit können Hindernisse besser erkannt werden und man kann sogar Objektklassifikation beispielsweise mit einer künstlichen Intelligenz durchführen. Ein weiterer Aspekt ist der Vergleich von SLAM-Algorithmen, die nur eine Kamera oder nur einen LiDAR oder beides zusammen fusioniert, verwenden. Die Fusion von Kamera- und Laserscannerdaten hört sich besonders vielversprechend an und ist nach Graeter et al. [64, S. 1] bereits beschrieben.

Literatur

- [1] W. H. Organization, *Global Status Report on Road Safety 2018*. Geneva: World Health Organization, 2019. [Online]. Verfügbar unter: <https://ebookcentral.proquest.com/lib/kxp/detail.action?docID=5910092>
- [2] Statistisches Bundesamt, *Gesellschaft und Umwelt. Verkehrsunfälle*. [Online]. Verfügbar unter: https://www.destatis.de/DE/Themen/Gesellschaft-Umwelt/Verkehrsunfaelle/_inhalt.html#sprg229230 (Zugriff am: 16. Mai 2022).
- [3] National Highway Traffic Safety Administration, *Early Estimate of Motor Vehicle Traffic Fatalities in 2020*. [Online]. Verfügbar unter: <https://crashstats.nhtsa.dot.gov/Api/Public/ViewPublication/813115> (Zugriff am: 22. April 2022).
- [4] Association for Safe International Road Travel, *Road Safety Facts*. [Online]. Verfügbar unter: <https://www.asirt.org/safe-travel/road-safety-facts/> (Zugriff am: 14. Mai 2022).
- [5] M. Botsch und W. Utschick, *Fahrzeugsicherheit und automatisiertes Fahren: Methoden der Signalverarbeitung und des maschinellen Lernens*. München: Hanser; Ciando, 2020. [Online]. Verfügbar unter: http://ebooks.ciando.com/book/index.cfm/bok_id/2839583
- [6] Statistisches Bundesamt, „Verkehrsunfälle - Fachserie 8 Reihe 7 - 2020“, 2021. [Online]. Verfügbar unter: https://www.destatis.de/DE/Themen/Gesellschaft-Umwelt/Verkehrsunfaelle/Publikationen/Downloads-Verkehrsunfaelle/verkehrsunfaelle-jahr-2080700207004.pdf?__blob=publicationFile
- [7] Ralf Bretting, *Das Dilemma des autonomen Fahrens*. [Online]. Verfügbar unter: <https://www.automotiveit.eu/exklusiv/das-dilemma-des-autonomen-fahrens-122.html> (Zugriff am: 3. März 2021).
- [8] M. Maurer, J. C. Gerdes, B. Lenz und H. Winner, Hg., *Autonomes Fahren: Technische, rechtliche und gesellschaftliche Aspekte*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2015. [Online]. Verfügbar unter: <http://nbn-resolving.org/urn:nbn:de:bsz:31-epflicht-1531313>
- [9] G. Meyer und S. Beiker, Hg., *Road Vehicle Automation 4*. Cham: Springer, 2018. [Online]. Verfügbar unter: <http://swbplus.bsz-bw.de/bsz490765904cov.htm>
- [10] Thomas Geiger, *Automatisiertes Fahren mit Staupilot: Freihändig in der S-Klasse*. [Online]. Verfügbar unter: <https://www.adac.de/rund-ums-fahrzeug/ausstattung-technik-zubehoer/autonomes-fahren/technik-vernetzung/autonomes-fahren-staupilot-s-klasse/> (Zugriff am: 22. März 2022).

- [11] ARRK Engineering GmbH, *ARRK im Überblick*. [Online]. Verfügbar unter: <https://www.arrkeurope.com/de/unternehmen/arrk-im-ueberblick/> (Zugriff am: 3. März 2022).
- [12] ARRK Engineering GmbH, *ARRK - Ihr globaler Partner für die Produktentwicklung* (Zugriff am: 3. März 2022).
- [13] Bundesanstalt für Straßenwesen, *Selbstfahrende Autos – assistiert, automatisiert oder autonom?* [Online]. Verfügbar unter: https://www.bast.de/BAST_2017/DE/Presse/Mitteilungen/2021/06-2021.html (Zugriff am: 2. April 2022).
- [14] SAE Internationl, *Levels of Driving Automation*. [Online]. Verfügbar unter: <https://www.sae.org/news/2019/01/sae-updates-j3016-automated-driving-graphic> (Zugriff am: 15. April 2022).
- [15] M. Mitteregger et al., *AVENUE21. Automatisierter und vernetzter Verkehr: Entwicklungen des urbanen Europa*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2020.
- [16] K. Reif, Hg., *Fahrstabilisierungssysteme und Fahrerassistenzsysteme*, 1. Aufl. Wiesbaden: Vieweg + Teubner, 2010. [Online]. Verfügbar unter: <http://swbplus.bsz-bw.de/bsz325728399cov.htm>
- [17] Bosch, *Spurhalteassistent. Unterstützt den Fahrer aktiv beim Halten der markierten Fahrspur*. [Online]. Verfügbar unter: <https://www.bosch-mobility-solutions.com/de/loesungen/assistentssysteme/spurhalteassistent/> (Zugriff am: 4. April 2022).
- [18] P. Pannaggar, D. Nalic, F. Orucevic, A. Eichberger und B. Rogic, „Advanced Lane Detection Model for the Virtual Development of Highly Automated Functions“, 15. Apr. 2021. [Online]. Verfügbar unter: <http://arxiv.org/pdf/2104.07481v1>.
- [19] Bosch, *Multifunktionskamera kombiniert Methoden der künstlichen Intelligenz mit klassischen Bildverarbeitungsalgorithmen*. [Online]. Verfügbar unter: <https://www.bosch-mobility-solutions.com/de/loesungen/kamera/multifunktionskamera/> (Zugriff am: 4. April 2022).
- [20] Vivienne Brando, *Einfach Technik: Hochautomatisiertes Fahren mit dem DRIVE PILOT*. [Online]. Verfügbar unter: <https://www.daimler.com/magazin/technologie-innovation/einfach-technik-drive-pilot.html>.
- [21] G. Wautelet, S. Loyer, F. Mercier und F. Perosanz, „Computation of GPS P1–P2 Differential Code Biases with JASON-2“, *GPS Solut*, Jg. 21, Nr. 4, S. 1619–1631, 2017, doi: 10.1007/s10291-017-0638-1.
- [22] H. Winner, S. Hakuli, F. Lotz und C. Singer, Hg., *Handbuch Fahrerassistenzsysteme: Grundlagen, Komponenten und Systeme für aktive Sicherheit und Komfort*, 3. Aufl. Wiesbaden: Springer Vieweg, 2015.

-
- [23] K. Reif, *Automobilelektronik: Eine Einführung für Ingenieure*, 4. Aufl. Wiesbaden: Vieweg + Teubner, 2012.
- [24] D. Eberhard und H. Härter, *Die Vorteile von Lidar-Sensoren im autonomen Fahrzeug*. [Online]. Verfügbar unter: <https://www.next-mobility.de/die-vorteile-von-lidar-sensoren-im-autonomen-fahrzeug-a-710222/> (Zugriff am: 28. April 2022).
- [25] J. Hertzberg, K. Lingemann und A. Nüchter, *Mobile Roboter: Eine Einführung aus Sicht der Informatik*. Berlin, Heidelberg: Springer Vieweg, 2012. [Online]. Verfügbar unter: <http://swbplus.bsz-bw.de/bsz310220548cov.htm>
- [26] M. Trzesniowski, „Lenkung Steering“ in *Fahrwerk*, M. Trzesniowski, Hg., Wiesbaden: Springer Fachmedien Wiesbaden, 2017, S. 281–341, doi: 10.1007/978-3-658-15545-2_5.
- [27] G. Kitagawa, „Monte Carlo Filter and Smoother for Non-Gaussian Nonlinear State Space Models“, *Journal of Computational and Graphical Statistics*, Jg. 5, Nr. 1, S. 1, 1996, doi: 10.2307/1390750.
- [28] J. S. Liu und R. Chen, „Sequential Monte Carlo Methods for Dynamic Systems“, *Journal of the American Statistical Association*, Jg. 93, Nr. 443, S. 1032–1044, 1998, doi: 10.1080/01621459.1998.10473765.
- [29] F. Dellaert, D. Fox, W. Burgard und S. Thrun, *Monte Carlo Localization for Mobile Robots*. Bonn: University of Bonn, 1999.
- [30] C. Cadena et al., „Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age“, *IEEE Trans. Robot.*, Jg. 32, Nr. 6, S. 1309–1332, 2016, doi: 10.1109/TRO.2016.2624754.
- [31] K. Sugiura und H. Matsutani, „Particle Filter-based vs. Graph-based: SLAM Acceleration on Low-end FPGAs“, 17. März 2021. [Online]. Verfügbar unter: <http://arxiv.org/pdf/2103.09523v1>.
- [32] G. Grisetti, C. Stachniss und W. Burgard, „Improved Techniques for Grid Mapping With Rao-Blackwellized Particle Filters“, *IEEE Trans. Robot.*, Jg. 23, Nr. 1, S. 34–46, 2007, doi: 10.1109/TRO.2006.889486.
- [33] K. Mehlhorn und P. Sanders, *Algorithms and data structures: The basic toolbox*. Berlin, Heidelberg: Springer, 2008. [Online]. Verfügbar unter: <http://swbplus.bsz-bw.de/bsz283175338cov.htm>
- [34] L. Velden, *Der Dijkstra-Algorithmus*. [Online]. Verfügbar unter: https://algorithms.discrete.ma.tum.de/graph-algorithms/spp-dijkstra/index_en.html (Zugriff am: 15. Mai 2022).
- [35] W. Zeng und R. L. Church, „Finding shortest paths on real road networks: the case for A*“, *International Journal of Geographical Information Science*, Jg. 23, Nr. 4, S. 531–543, 2009, doi: 10.1080/13658810801949850.
-

- [36] C. Rösmann, W. Feiten, T. Wösch, F. Hoffmann und T. Bertram, „Trajectory modification considering dynamic constraints of autonomous robots“ in *Proceedings for the conference of ROBOTIK 2012: 7th German Conference on Robotics, 21 - 22 May 2012, International Congress Center Munich (ICM) in conjunction with AUTOMATICA*, Berlin: VDE-Verl., 2012, S. 74–79.
- [37] C. Rösmann, F. Hoffmann und T. Bertram, „Integrated online trajectory planning and optimization in distinctive topologies“, *Robotics and Autonomous Systems*, Jg. 88, S. 142–153, 2017, doi: 10.1016/j.robot.2016.11.007.
- [38] S. Bhattacharya, *Topological and Geometric Techniques in Graph Search-Based Robot Planning (Ph.D. thesis)*. University of Pennsylvania, 2012.
- [39] Traxxas, *Slash 4X4 Platinum Edition: Engineered the Traxxas Way, to Race Your Way*. [Online]. Verfügbar unter: <https://traxxas.com/products/models/electric/6804Rslash4x4platinum> (Zugriff am: 18. Dezember 2021).
- [40] A. Hortobagyi, *esk8 controller - made in Germany based upon the VESC® Open Source Project*. [Online]. Verfügbar unter: <http://www.hellray.de/esk8-controller/>.
- [41] B. Vedder, *The VESC Project*. [Online]. Verfügbar unter: <https://vesc-project.com/> (Zugriff am: 19. Dezember 2021).
- [42] NVIDIA, *Jetson TX2-Module*. [Online]. Verfügbar unter: <https://www.nvidia.com/de-de/autonomous-machines/embedded-systems/jetson-tx2/> (Zugriff am: 18. Dezember 2021).
- [43] Stereolabs, *Meet ZED: From lens to sensors, the ZED camera is filled with cutting-edge technology that takes depth and motion tracking to a whole new level*. [Online]. Verfügbar unter: <https://www.stereolabs.com/zed/> (Zugriff am: 18. Dezember 2021).
- [44] InvenSense, *MPU-9250 MPU-9250. Product Specification. Revision 1.0*. San Jose, 2014.
- [45] B. R. Japón, *Hands-on ROS for robotics programming: Program highly autonomous and AI-capable mobile robots powered by ROS*. Birmingham: Packt Publishing Limited, 2020. [Online]. Verfügbar unter: <https://ebookcentral.proquest.com/lib/kxp/detail.action?docID=6124234>
- [46] L. Joseph, *Robot Operating System for Absolute Beginners*. Berkeley, CA: Apress, 2018.
- [47] Hokuyo, *URG-04LX-UG01*. [Online]. Verfügbar unter: <https://www.hokuyo-aut.jp/search/single.php?serial=166> (Zugriff am: 19. Dezember 2021).
- [48] Hokuyo, *Scanning Laser Range Finder. URG-04LX-UG01*. Osaka, 2009.
- [49] Hokuyo, *hokuyo_node*. [Online]. Verfügbar unter: http://wiki.ros.org/hokuyo_node (Zugriff am: 19. Dezember 2021).
- [50] ROS, *sensor_msgs/LaserScan Message*. [Online]. Verfügbar unter: http://docs.ros.org/en/api/sensor_msgs/html/msg/LaserScan.html.

- [51] ROS, *gmapping*. [Online]. Verfügbar unter: <http://wiki.ros.org/gmapping> (Zugriff am: 5. Januar 2022).
- [52] f1tenth, *f1tenth/vesc*. [Online]. Verfügbar unter: https://github.com/f1tenth/vesc/tree/main/vesc_ackermann (Zugriff am: 21. März 2022).
- [53] ROS, *nav_msgs/Odometry Message*. [Online]. Verfügbar unter: http://docs.ros.org/en/noetic/api/nav_msgs/html/msg/Odometry.html (Zugriff am: 28. März 2022).
- [54] ROS, *tf2_msgs/TFMessage Message*. [Online]. Verfügbar unter: http://docs.ros.org/en/melodic/api/tf2_msgs/html/msg/TFMessage.html (Zugriff am: 1. April 2022).
- [55] ROS, *nav_msgs/OccupancyGrid Message*. [Online]. Verfügbar unter: http://docs.ros.org/en/noetic/api/nav_msgs/html/msg/OccupancyGrid.html (Zugriff am: 30. März 2022).
- [56] ROS, *nav_msgs/MapMetaData Message*. [Online]. Verfügbar unter: http://docs.ros.org/en/noetic/api/nav_msgs/html/msg/MapMetaData.html (Zugriff am: 30. März 2022).
- [57] ROS, *navigation*. [Online]. Verfügbar unter: <http://wiki.ros.org/navigation> (Zugriff am: 26. März 2022).
- [58] ROS, *teb_local_planner*. [Online]. Verfügbar unter: http://wiki.ros.org/teb_local_planner (Zugriff am: 26. März 2022).
- [59] ROS, *AMCL*. [Online]. Verfügbar unter: <http://wiki.ros.org/amcl> (Zugriff am: 26. Februar 2022).
- [60] H. Kemper, *Maschinist für Löschfahrzeuge*, 2. Aufl. Landsberg am Lech: ecomed Sicherheit, 2016.
- [61] ROS, *Setup and Configuration of the Navigation Stack on a Robot*. [Online]. Verfügbar unter: <http://wiki.ros.org/navigation/Tutorials/RobotSetup> (Zugriff am: 26. Februar 2022).
- [62] *ackermann_msgs/AckermannDriveStamped Message*. [Online]. Verfügbar unter: http://docs.ros.org/en/api/ackermann_msgs/html/msg/AckermannDriveStamped.html (Zugriff am: 26. Februar 2022).
- [63] F. Flömer, *Aus für Mobileye bei BMW: Level 3 ab 2025 mit Qualcomm und Arriver*. [Online]. Verfügbar unter: <https://t3n.de/news/fuer-mobileye-bmw-level-3-ab-1458912/> (Zugriff am: 2. April 2022).
- [64] J. Graeter, A. Wilczynski und M. Lauer, „LIMO: Lidar-Monocular Visual Odometry“, 19. Juli 2018. [Online]. Verfügbar unter: <http://arxiv.org/pdf/1807.07524v1>.

Anhang

Im Anhang sind alle Installations- und Konfigurationsbefehle zu finden. Außerdem wird der Quellcode für die wichtigsten Softwaremodule präsentiert. Darüber hinaus werden relevante Konfigurations- und Launchdateien bzgl. der verwendeten ROS-Knoten aufgeführt.

1. Installation und Konfiguration des Host Laptops

Der Host Laptop wird mit Ubuntu 18.04 und ROS Melodic installiert. Für die Ubuntu-Installation wird ein bootfähiger USB-Stick mit einem Image verwendet. Um schließlich ROS installieren zu können, müssen folgende Befehle in einem Terminal ausgeführt werden.

Als erstes müssen in Ubuntu die Repositories aktualisiert werden, um ROS herunterladen zu können:

```
$ sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list'
```

```
$ sudo apt install curl  
$ curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc | sudo apt-key add
```

Installation von ROS:

```
$ sudo apt update  
$ sudo apt install ros-melodic-desktop-full
```

Mit folgendem Befehl wird die ROS-Umgebung konfiguriert. Damit werden Befehle wie *roscore* im Terminal automatisch erkannt. Durch das hinzufügen zum *bashrc*-File wird die *setup.bash* bei jedem Terminalstart aufgerufen.

```
$ echo "source /opt/ros/melodic/setup.bash" >> ~/.bashrc  
source ~/.bashrc
```

Um eigene ROS-Workspaces und ROS-Pakete erstellen zu können, müssen noch die folgenden Abhängigkeiten nachinstalliert werden:

```
$ sudo apt install python-rosdep python-rosinstall python-rosinstall-generator python-wstool build-essential
```

```
$ sudo rosdep init  
$ rosdep update
```

Nach der Installation aller Komponenten muss der eigene Workspace aufgesetzt werden.

```
$ mkdir -p ~/catkin_ws/src
$ cd ~/catkin_ws/
$ catkin_make
```

Zu diesem Workspace können nun Pakete hinzugefügt und ausgeführt werden. Dafür ist es wichtig nach dem Kompilieren des Sourcecodes mit dem Befehl `catkin_make` in dem Ordner `devel` das Skript `setup.bash` auszuführen. Damit werden die Paketnamen global sichtbar und können direkt vom Terminal aus angesprochen werden.

```
$ source devel/setup.bash
```

Als nächste wird der Jetson TX2 als ROS-Master auserwählt. Dafür müssen folgende Zeilen in das `bashrc`-File eingefügt werden:

```
export ROS_MASTER_URI=http://10.42.0.1:11311
export ROS_IP=10.42.0.1
```

In das `bashrc`-File des Host-Laptops müssen hingegen diese Zeilen eingefügt werden:

```
export ROS_MASTER_URI=http://10.42.0.1:11311
export ROS_IP=10.42.0.2
```

Mit den folgenden Befehlen wird auf dem Host-Laptop ein NTP-Server zur Zeitsynchronisierung installiert:

```
$ sudo apt-get install ntp
$ sudo service ntp status
```

Um den den Server von außen erreichbar zu machen, muss in der Firewall der UDP Port 123 freigeschalten werden:

```
$ sudo ufw allow from any to any port 123 proto udp
```

Auf dem Jetson TX2 kann dann der NTP-Server angesprochen werden. Dazu muss die IP-Adresse 10.42.0.2 in der Datei `/etc/systemd/timesyncd.conf` eingetragen und aktiviert werden:

```
$ timedatectl set-ntp 1
```

2. Inbetriebnahme des LiDARs

Um den Gerätenamen des LiDARs nach jedem Neustart gleich halten zu können, wird unter `/etc/udev/rules.d/99-hokuyo.rules` folgende Regel erstellt:

```
KERNEL=="ttyACM[0-9]*", ACTION=="add", ATTRS{idVendor}=="15d1", MODE="0666",  
GROUP="dialout", SYMLINK+="sensors/hokuyo"
```

Mit folgenden Befehlen wird die Regel aktiviert:

```
$ sudo udevadm control --reload-rules  
$ sudo udevadm trigger
```

Damit ist der LiDAR immer unter dem Gerätenamen `/dev/sensors/hokuyo` ansprechbar.

Als nächstes muss der Treiber für den LiDAR installiert werden. Dafür wird das Git-Repository `hokuyo_node` kopiert:

```
$ cd ~/robotcar_ws/src  
$ git clone https://github.com/ros-drivers/hokuyo\_node  
$ catkin_make
```

Abschließend muss noch der minimale und maximale Scanwinkel angepasst werden. Dieser ist nämlich standardmäßig auf $\pm 90^\circ$ eingestellt. Dies wird in der Konfigurationsdatei `sensors.yaml` getan, die folgendermaßen aufgebaut ist:

laser_node:

```
device: /dev/sensors/hokuyo  
min_ang: -2.094  
max_ang: 2.094
```

3. Installation und Inbetriebnahme von gmapping

Als erstes müssen die beiden Topics `/vesc/odom` und `/tf` veröffentlicht werden. Dazu wird der ROS-Knoten „`vesc_to_odom_node.cpp`“ verwendet:

vesc_to_odom_node.cpp

```
#include <ros/ros.h>  
  
#include "vesc_ackermann/vesc_to_odom.h"  
  
int main(int argc, char** argv)  
{
```



```
ros::init(argc, argv, "vesc_to_odom_node");
ros::NodeHandle nh;
ros::NodeHandle private_nh("~");

vesc_ackermann::VescToOdom vesc_to_odom(nh, private_nh);

ros::spin();

return 0;
}
```

Das verwendete Objekt vom Typ `vesc_ackermann::VescToOdom` wird durch die beiden Dateien „`vesc_to_odom.h`“ und „`vesc_to_odom.cpp`“ festgelegt.

vesc_to_odom.h

```
#ifndef VESC_ACKERMANN_VESC_TO_ODOM_H_
#define VESC_ACKERMANN_VESC_TO_ODOM_H_

#include <memory>
#include <string>

#include <ros/ros.h>
#include <vesc_msgs/VescStateStamped.h>
#include <std_msgs/Float64.h>
#include <tf/transform_broadcaster.h>

namespace vesc_ackermann
{
class VescToOdom
{
public:
    VescToOdom(ros::NodeHandle nh, ros::NodeHandle private_nh);

private:
    // ROS parameters
    std::string odom_frame_;
```

```
std::string base_frame_;
/** State message does not report servo position, so use the command instead */
bool use_servo_cmd_;
// conversion gain and offset
double speed_to_erpm_gain_, speed_to_erpm_offset_;
double steering_to_servo_gain_, steering_to_servo_offset_;
double wheelbase_;
bool publish_tf_;

// odometry state
double x_, y_, yaw_;
std_msgs::Float64::ConstPtr last_servo_cmd_; ///< Last servo position commanded value
vesc_msgs::VescStateStamped::ConstPtr last_state_; ///< Last received state message

// ROS services
ros::Publisher odom_pub_;
ros::Subscriber vesc_state_sub_;
ros::Subscriber servo_sub_;
std::shared_ptr<tf::TransformBroadcaster> tf_pub_;

// ROS callbacks
void vescStateCallback(const vesc_msgs::VescStateStamped::ConstPtr& state);
void servoCmdCallback(const std_msgs::Float64::ConstPtr& servo);
};

} // namespace vesc_ackermann

#endif // VESC_ACKERMANN_VESC_TO_ODOM_H_
```

vesc_to_odom.cpp

```
#include "vesc_ackermann/vesc_to_odom.h"

#include <cmath>
#include <string>

#include <nav_msgs/Odometry.h>
#include <geometry_msgs/TransformStamped.h>
```

```
namespace vesc_ackermann
{

template <typename T>
inline bool getRequiredParam(const ros::NodeHandle& nh, const std::string& name, T*
value);

VescToOdom::VescToOdom(ros::NodeHandle nh, ros::NodeHandle private_nh) :
    odom_frame_("odom"), base_frame_("base_link"),
    use_servo_cmd_(true), publish_tf_(false), x_(0.0), y_(0.0), yaw_(0.0)
{
    // get ROS parameters
    private_nh.param("odom_frame", odom_frame_, odom_frame_);
    private_nh.param("base_frame", base_frame_, base_frame_);
    private_nh.param("use_servo_cmd_to_calc_angular_velocity", use_servo_cmd_,
use_servo_cmd_);
    if (!getRequiredParam(nh, "speed_to_erpm_gain", &speed_to_erpm_gain_))
        return;
    if (!getRequiredParam(nh, "speed_to_erpm_offset", &speed_to_erpm_offset_))
        return;
    if (use_servo_cmd_)
    {
        if (!getRequiredParam(nh, "steering_angle_to_servo_gain", &steering_to_servo_gain_))
            return;
        if (!getRequiredParam(nh, "steering_angle_to_servo_offset",
&steering_to_servo_offset_))
            return;
        if (!getRequiredParam(nh, "wheelbase", &wheelbase_))
            return;
    }
    private_nh.param("publish_tf", publish_tf_, publish_tf_);

    // create odom publisher
    odom_pub_ = nh.advertise<nav_msgs::Odometry>("odom", 10);

    // create tf broadcaster
```

```
if (publish_tf_)
{
    tf_pub_.reset(new tf::TransformBroadcaster);
}

// subscribe to vesc state and. optionally, servo command
vesc_state_sub_ = nh.subscribe("sensors/core", 10, &VescToOdom::vescStateCallback,
this);
if (use_servo_cmd_)
{
    servo_sub_ = nh.subscribe("sensors/servo_position_command", 10,
        &VescToOdom::servoCmdCallback, this);
}
}

void VescToOdom::vescStateCallback(const vesc_msgs::VescStateStamped::ConstPtr&
state)
{
    // check that we have a last servo command if we are depending on it for angular velocity
    if (use_servo_cmd_ && !last_servo_cmd_)
        return;

    // convert to engineering units
    double current_speed = (-state->state.speed - speed_to_erpm_offset_) /
speed_to_erpm_gain_;
    if (std::fabs(current_speed) < 0.05)
    {
        current_speed = 0.0;
    }
    double current_steering_angle(0.0), current_angular_velocity(0.0);
    if (use_servo_cmd_)
    {
        current_steering_angle =
            (last_servo_cmd_->data - steering_to_servo_offset_) / steering_to_servo_gain_;
        current_angular_velocity = current_speed * tan(current_steering_angle) / wheelbase_;
    }
}
```

```
// use current state as last state if this is our first time here
if (!last_state_)
    last_state_ = state;

// calc elapsed time
ros::Duration dt = state->header.stamp - last_state_->header.stamp;

/** @todo could probably do better propagating odometry, e.g. trapezoidal integration */

// propagate odometry
double x_dot = current_speed * cos(yaw_);
double y_dot = current_speed * sin(yaw_);
x_ += x_dot * dt.toSec();
y_ += y_dot * dt.toSec();
if (use_servo_cmd_)
    yaw_ += current_angular_velocity * dt.toSec();

// save state for next time
last_state_ = state;

// publish odometry message
nav_msgs::Odometry::Ptr odom(new nav_msgs::Odometry);
odom->header.frame_id = odom_frame_;
odom->header.stamp = state->header.stamp;
odom->child_frame_id = base_frame_;

// Position
odom->pose.pose.position.x = x_;
odom->pose.pose.position.y = y_;
odom->pose.pose.orientation.x = 0.0;
odom->pose.pose.orientation.y = 0.0;
odom->pose.pose.orientation.z = sin(yaw_ / 2.0);
odom->pose.pose.orientation.w = cos(yaw_ / 2.0);

// Position uncertainty
/** @todo Think about position uncertainty, perhaps get from parameters? */
odom->pose.covariance[0] = 0.2; ///< x
```

```
odom->pose.covariance[7] = 0.2; ///< y
odom->pose.covariance[35] = 0.4; ///< yaw

// Velocity ("in the coordinate frame given by the child_frame_id")
odom->twist.twist.linear.x = current_speed;
odom->twist.twist.linear.y = 0.0;
odom->twist.twist.angular.z = current_angular_velocity;

// Velocity uncertainty
/** @todo Think about velocity uncertainty */

if (publish_tf_)
{
    geometry_msgs::TransformStamped tf;
    tf.header.frame_id = odom_frame_;
    tf.child_frame_id = base_frame_;
    tf.header.stamp = ros::Time::now();
    tf.transform.translation.x = x_;
    tf.transform.translation.y = y_;
    tf.transform.translation.z = 0.0;
    tf.transform.rotation = odom->pose.pose.orientation;
    if (ros::ok())
    {
        tf_pub_->sendTransform(tf);
    }
}

if (ros::ok())
{
    odom_pub_.publish(odom);
}
}

void VescToOdom::servoCmdCallback(const std_msgs::Float64::ConstPtr& servo)
{
    last_servo_cmd_ = servo;
}
```

```

template <typename T>
inline bool getRequiredParam(const ros::NodeHandle& nh, const std::string& name, T*
value)
{
    if (nh.getParam(name, *value))
        return true;

    ROS_FATAL("VescToOdom: Parameter %s is required.", name.c_str());
    return false;
}

} // namespace vesc_ackermann

```

Die Konfigurationsdatei *vesc.yaml* für obigen Knoten ist im Folgenden gegeben. Wichtig ist, dass für das Veröffentlichen des Topics */tf* der Parameter *vesc_to_odom/publish_tf* auf *true* gesetzt werden muss.

vesc.yaml

```

# erpm (electrical rpm) = speed_to_erpm_gain * speed (meters / second) +
speed_to_erpm_offset
speed_to_erpm_gain: 6300 #default 4614, gave wrong velocity for odometry
speed_to_erpm_offset: 0.0

tachometer_ticks_to_meters_gain: 0.00225
# servo smoother - limits rotation speed and smooths anything above limit
max_servo_speed: 3.2 # radians/second
servo_smoother_rate: 75.0 # messages/sec

# servo smoother - limits acceleration and smooths anything above limit
max_acceleration: 2.5 # meters/second^2
throttle_smoother_rate: 75.0 # messages/sec

# servo value (0 to 1) = steering_angle_to_servo_gain * steering angle (radians) +
steering_angle_to_servo_offset
steering_angle_to_servo_gain: -0.8

```

```
steering_angle_to_servo_offset: 0.467 #adjusted to 0.467
```

```
# publish odom to base link tf
vesc_to_odom/publish_tf: true
```

```
# car wheelbase
wheelbase: .34
```

```
vesc_driver:
  port: /dev/sensors/vesc
  duty_cycle_min: 0.0
  duty_cycle_max: 0.0
  current_min: 0.0
  current_max: 100.0
  brake_min: -20000.0
  brake_max: 200000.0
  speed_min: -5250 #-8000#-23250
  speed_max: 5250 #8000 #23250
  position_min: 0.0
  position_max: 0.0
  servo_min: 0.15
  servo_max: 0.85
```

Der Knoten wird mit diesem Launch-File gestartet:

vesc.launch.xml

```
<?xml version="1.0"?>

<!-- -*- mode: XML -*- -->

-<launch>

<arg name="racecar_version"/>

<arg name="vesc_config" default="$(find racecar)/config/${arg
racecar_version}/vesc.yaml"/>
```



```
<roscpp command="load" file="$(arg vesc_config)"/>

-<node name="ackermann_to_vesc" type="ackermann_to_vesc_node"
pkg="vesc_ackermann">

<!-- Remap to make mux control work with the VESC -->

<remap to="low_level/ackermann_cmd_mux/output" from="ackermann_cmd"/>

<!-- Remap to make vesc have trapezoidal control on the throttle to avoid skipping -->

<remap to="commands/motor/unsmoothed_speed" from="commands/motor/speed"/>

<!-- Remap to make vesc have trapezoidal control on the servo to avoid incorrect
odometry and damage -->

<remap to="commands/servo/unsmoothed_position" from="commands/servo/position"/>

</node>

<node name="vesc_driver" type="vesc_driver_node" pkg="vesc_driver"/>

<node name="vesc_to_odom" type="vesc_to_odom_node" pkg="vesc_ackermann"/>

<node name="throttle_interpolator" type="throttle_interpolator.py"
pkg="ackermann_cmd_mux"/>

</launch>
```

Da nun alle Vorbedingungen erfüllt sind, wird gmapping und der Knoten Map-Server zum Abspeichern der Karte, installiert:

```
sudo apt-get install ros-melodic-gmapping
sudo apt-get install ros-melodic-map-server
```

slam_gmapping wird mit dem Launch-File *slam_gmapping.launch* gestartet:

slam_gmapping.launch

```
<launch>
  <node pkg="gmapping" type="slam_gmapping" name="slam_gmapping" output="screen"
  >
    <param name="maxUrange" value="5.2" />
    <param name="maxRange" value="5.8" />
    <param name="xmin" value="-100.0" />
    <param name="ymin" value="-100.0" />
    <param name="xmax" value="100.0" />
    <param name="ymax" value="100.0" />
  </node>
</launch>
```

Wenn die Karte erstellt ist, wird sie mit diesem Befehl abgespeichert:

```
roslaunch map_server map_saver -f my_map_1
```

4. Umsetzung der vollautomatisierten Navigation mittels ROS-Navigation-Stack

Navigation-Stack installieren (inkl. AMCL):

```
sudo apt-get install ros-melodic-navigation
```

TEB-Local-Planner installieren:

```
sudo apt-get install ros-melodic-teb-local-planner
```

Als nächstes müssen folgenden Konfigurationsdateien erstellt werden:

```
local_costmap_params.yaml
global_costmap_params.yaml
costmap_common_params.yaml
base_local_planner_params.yaml
```

local_costmap_params.yaml

```
local_costmap:
```

```

global_frame: map
robot_base_frame: base_link
update_frequency: 5.0
publish_frequency: 2.0
static_map: false
rolling_window: true
width: 5.5
height: 5.5
resolution: 0.1
transform_tolerance: 0.5

plugins:
- {name: static_layer,      type: "costmap_2d::StaticLayer"}
- {name: obstacle_layer,   type: "costmap_2d::ObstacleLayer"}

```

global_costmap_params.yaml

```

global_costmap:
  global_frame: map
  robot_base_frame: base_link
  update_frequency: 1.0
  publish_frequency: 0.5
  static_map: true

  transform_tolerance: 0.5
  plugins:
    - {name: static_layer,      type: "costmap_2d::StaticLayer"}
    - {name: obstacle_layer,    type: "costmap_2d::VoxelLayer"}
    - {name: inflation_layer,   type: "costmap_2d::InflationLayer"}

```

costmap_common_params.yaml

```

footprint: [ [-0.1,-0.125], [0.5,-0.125], [0.5,0.125], [-0.1,0.125] ]

transform_tolerance: 0.2
map_type: costmap

obstacle_layer:

```

```
enabled: true
obstacle_range: 3.0
raytrace_range: 3.5
inflation_radius: 0.2
track_unknown_space: false
combination_method: 1

observation_sources: laser_scan_sensor
laser_scan_sensor: {data_type: LaserScan, topic: scan, marking: true, clearing: true}

inflation_layer:
  enabled:      true
  cost_scaling_factor: 10.0 # exponential rate at which the obstacle cost drops off (default:
10)
  inflation_radius: 0.5 # max. distance from an obstacle at which costs are incurred for
planning paths.

static_layer:
  enabled:      true
  map_topic:    "/map"
```

base_local_planner_params.yaml

TebLocalPlannerROS:

odom_topic: /vesc/odom

map_frame: /map

Trajectory

teb_autosize: True

dt_ref: 0.3

dt_hysteresis: 0.1

max_samples: 500

global_plan_overwrite_orientation: True

allow_init_with_backwards_motion: True

max_global_plan_lookahead_dist: 3.0

global_plan_viapoint_sep: -1

```
global_plan_prune_distance: 1
exact_arc_length: False
feasibility_check_no_poses: 2
publish_feedback: False

# Robot

max_vel_x: 0.4
max_vel_x_backwards: 0.2
max_vel_y: 0.0
max_vel_theta: 0.3 # the angular velocity is also bounded by min_turning_radius in case of
a carlike robot ( $r = v / \omega$ )
acc_lim_x: 0.5
acc_lim_theta: 0.5

min_turning_radius: 0.8 #max steering_angle ~25° => min_turning_radius=~0.8 => using
0.9 for safety
wheelbase: 0.33
cmd_angle_instead_rotvel: True

footprint_model: # types: "point", "circular", "two_circles", "line", "polygon"
  type: "line"
  radius: 0.2 # for type "circular"
  line_start: [0.0, 0.0] # for type "line"
  line_end: [0.4, 0.0] # for type "line"
  front_offset: 0.2 # for type "two_circles"
  front_radius: 0.2 # for type "two_circles"
  rear_offset: 0.2 # for type "two_circles"
  rear_radius: 0.2 # for type "two_circles"
  vertices: [ [0.25, -0.05], [0.18, -0.05], [0.18, -0.18], [-0.19, -0.18], [-0.25, 0], [-0.19, 0.18],
[0.18, 0.18], [0.18, 0.05], [0.25, 0.05] ] # for type "polygon"

# GoalTolerance

xy_goal_tolerance: 0.2
yaw_goal_tolerance: 0.1
free_goal_vel: False
```

```
complete_global_plan: True

# Obstacles

min_obstacle_dist: 0.2
inflation_dist: 0.6
include_costmap_obstacles: True
costmap_obstacles_behind_robot_dist: 1.0
obstacle_poses_affected: 15

dynamic_obstacle_inflation_dist: 0.6
include_dynamic_obstacles: True

costmap_converter_plugin: ""
costmap_converter_spin_thread: True
costmap_converter_rate: 5

# Optimization

no_inner_iterations: 5
no_outer_iterations: 4
optimization_activate: True
optimization_verbose: False
penalty_epsilon: 0.1
obstacle_cost_exponent: 4
weight_max_vel_x: 2
weight_max_vel_theta: 1
weight_acc_lim_x: 1
weight_acc_lim_theta: 1
weight_kinematics_nh: 1000
weight_kinematics_forward_drive: 1
weight_kinematics_turning_radius: 1
weight_optimaltime: 1 # must be > 0
weight_shortest_path: 0
weight_obstacle: 100
weight_inflation: 0.2
weight_dynamic_obstacle: 10 # not in use yet
```

```
weight_dynamic_obstacle_inflation: 0.2
weight_viapoint: 1
weight_adapt_factor: 2

# Homotopy Class Planner

enable_homotopy_class_planning: True
enable_multithreading: True
max_number_classes: 4
selection_cost_hysteresis: 1.0
selection_prefer_initial_plan: 0.95
selection_obst_cost_scale: 1.0
selection_alternative_time_cost: False

roadmap_graph_no_samples: 15
roadmap_graph_area_width: 5
roadmap_graph_area_length_scale: 1.0
h_signature_prescaler: 0.5
h_signature_threshold: 0.1
obstacle_heading_threshold: 0.45
switching_blocking_period: 0.0
viapoints_all_candidates: True
delete_detours_backwards: True
max_ratio_detours_duration_best_duration: 3.0
visualize_hc_graph: False
visualize_with_time_as_z_axis_scale: False

# Recovery

shrink_horizon_backup: True
shrink_horizon_min_duration: 10
oscillation_recovery: True
oscillation_v_eps: 0.1
oscillation_omega_eps: 0.1
oscillation_recovery_min_duration: 10
oscillation_filter_duration: 10
```

Folgendes Pythonskript wird benötigt, um die Geschwindigkeitssignale des lokalen Pfadplaners in Ackermann-Nachrichten bzgl. eines Fahrzeugs mit Vorderachslenkung umzurechnen.

cmd_vel_to_ackermann_cmd.py

```
#!/usr/bin/env python

import rospy, math
from geometry_msgs.msg import Twist
from ackermann_msgs.msg import AckermannDriveStamped

def convert_trans_rot_vel_to_steering_angle(v, omega, wheelbase):
    if omega == 0 or v == 0:
        return 0

    radius = v / omega
    return math.atan(wheelbase / radius)

def cmd_callback(data):
    global wheelbase
    global ackermann_cmd_topic
    global frame_id
    global pub

    v = data.linear.x
    steering = convert_trans_rot_vel_to_steering_angle(v, data.angular.z, wheelbase)

    msg = AckermannDriveStamped()
    msg.header.stamp = rospy.Time.now()
    msg.header.frame_id = frame_id
    msg.drive.steering_angle = steering
    msg.drive.speed = v

    pub.publish(msg)
```



```
if __name__ == '__main__':
    try:

        rospy.init_node('cmd_vel_to_ackermann_drive')

        twist_cmd_topic = rospy.get_param('~twist_cmd_topic', '/cmd_vel')
        ackermann_cmd_topic = rospy.get_param('~ackermann_cmd_topic', '/ackermann_cmd')
        wheelbase = rospy.get_param('~wheelbase', 0.33)
        frame_id = rospy.get_param('~frame_id', 'odom')

        rospy.Subscriber(twist_cmd_topic, Twist, cmd_callback, queue_size=1)
        pub = rospy.Publisher(ackermann_cmd_topic, AckermannDriveStamped, queue_size=1)

        rospy.loginfo("Node 'cmd_vel_to_ackermann_drive' started.\nListening to %s, publishing
to %s. Frame id: %s, wheelbase: %f", "/cmd_vel", ackermann_cmd_topic, frame_id,
wheelbase)

        rospy.spin()

    except rospy.ROSInterruptException:
        pass
```

Alle Komponenten zur Navigation werden schließlich mit diesem Launch-File gestartet:

move_base.launch

```
<launch>

  <!-- Run the map server -->
  <node name="map_server" pkg="map_server" type="map_server" args="$(find
slam_gmapping)/map/office5.yaml"/>

  <!-- Run AMCL -->
```

```

<include file="$(find amcl)/examples/amcl_diff.launch" />

<node pkg="move_base" type="move_base" respawn="false" name="move_base"
output="screen">
  <rosparam file="$(find navigation_pkg)/costmap_common_params.yaml"
command="load" ns="global_costmap" />
  <rosparam file="$(find navigation_pkg)/costmap_common_params.yaml"
command="load" ns="local_costmap" />
  <rosparam file="$(find navigation_pkg)/local_costmap_params.yaml" command="load"
/>
  <rosparam file="$(find navigation_pkg)/global_costmap_params.yaml" command="load"
/>
  <rosparam file="$(find navigation_pkg)/base_local_planner_params.yaml"
command="load" />

  <param name="base_local_planner" value="teb_local_planner/TebLocalPlannerROS" />
  <param name="clearing_rotation_allowed" value="false" />
  <param name="controller_frequency" value="5.0" />
</node>

<node pkg="navigation_pkg" type="cmd_vel_to_ackermann_cmd.py"
name="cmd_vel_to_ackermann_drive" />
</launch>

```

Eingebundenes Launch-File für AMCL:

amcl_diff.launch

```

<launch>
<node pkg="amcl" type="amcl" name="amcl" output="screen">
  <!-- Publish scans from best pose at a max of 10 Hz -->
  <param name="odom_model_type" value="diff"/>
  <param name="odom_alpha5" value="0.1"/>
  <param name="transform_tolerance" value="0.2" />
  <param name="gui_publish_rate" value="10.0"/>
  <param name="laser_max_beams" value="30"/>
  <param name="laser_max_range" value="5.6" />
  <param name="min_particles" value="500"/>

```

```
<param name="max_particles" value="2000"/>
<param name="kld_err" value="0.05"/>
<param name="kld_z" value="0.99"/>
<param name="odom_alpha1" value="0.2"/>
<param name="odom_alpha2" value="0.2"/>
<!-- translation std dev, m -->
<param name="odom_alpha3" value="0.2"/>
<param name="odom_alpha4" value="0.2"/>
<param name="laser_z_hit" value="0.5"/>
<param name="laser_z_short" value="0.05"/>
<param name="laser_z_max" value="0.05"/>
<param name="laser_z_rand" value="0.5"/>
<param name="laser_sigma_hit" value="0.2"/>
<param name="laser_lambda_short" value="0.1"/>
<param name="laser_lambda_short" value="0.1"/>
<param name="laser_model_type" value="likelihood_field"/>
<!-- <param name="laser_model_type" value="beam"/> -->
<param name="laser_likelihood_max_dist" value="2.0"/>
<param name="update_min_d" value="0.25"/>
<param name="update_min_a" value="0.2"/>
<param name="odom_frame_id" value="odom"/>
<param name="base_frame_id" value="base_link"/>
<param name="global_frame_id" value="map"/>
<param name="resample_interval" value="1"/>
<param name="transform_tolerance" value="1.0"/>
<param name="recovery_alpha_slow" value="0.001"/>
<param name="recovery_alpha_fast" value="0.1"/>
</node>
</launch>
```

5. Programmierung eines GUI- Userinterfaces für die Routenerstellung

Die Python-GUI wird mit Qt erstellt. Die Generierung des Layouts erfolgt über den Qt-Designer. Das daraus generiert Layout-Python-Skript sieht folgendermaßen aus:

nav_GUI_layout.py

```
from PyQt5 import QtCore, QtGui, QtWidgets

class Ui_Robot_Navigation_GUI(object):
    def setupUi(self, Robot_Navigation_GUI):
        Robot_Navigation_GUI.setObjectName("Robot_Navigation_GUI")
        Robot_Navigation_GUI.resize(931, 854)
        self.centralwidget = QtWidgets.QWidget(Robot_Navigation_GUI)
        self.centralwidget.setObjectName("centralwidget")
        self.label_5 = QtWidgets.QLabel(self.centralwidget)
        self.label_5.setGeometry(QtCore.QRect(10, 360, 611, 371))
        self.label_5.setText("")
        self.label_5.setPixmap(QtGui.QPixmap("../Map_Landmarks.png"))
        self.label_5.setScaledContents(True)
        self.label_5.setObjectName("label_5")
        self.label_11 = QtWidgets.QLabel(self.centralwidget)
        self.label_11.setGeometry(QtCore.QRect(20, 330, 81, 21))
        font = QtGui.QFont()
        font.setPointSize(15)
        self.label_11.setFont(font)
        self.label_11.setObjectName("label_11")
        self.widget = QtWidgets.QWidget(self.centralwidget)
        self.widget.setGeometry(QtCore.QRect(14, 3, 562, 316))
        self.widget.setObjectName("widget")
        self.gridLayout = QtWidgets.QGridLayout(self.widget)
        self.gridLayout.setContentsMargins(0, 0, 0, 0)
        self.gridLayout.setObjectName("gridLayout")
        self.label = QtWidgets.QLabel(self.widget)
        font = QtGui.QFont()
        font.setPointSize(15)
        self.label.setFont(font)
```

```
self.label.setObjectName("label")
self.gridLayout.addWidget(self.label, 0, 0, 1, 2)
self.label_2 = QtWidgets.QLabel(self.widget)
font = QtGui.QFont()
font.setPointSize(15)
self.label_2.setFont(font)
self.label_2.setObjectName("label_2")
self.gridLayout.addWidget(self.label_2, 0, 5, 1, 2)
self.label_3 = QtWidgets.QLabel(self.widget)
self.label_3.setObjectName("label_3")
self.gridLayout.addWidget(self.label_3, 1, 0, 1, 1)
self.label_4 = QtWidgets.QLabel(self.widget)
self.label_4.setObjectName("label_4")
self.gridLayout.addWidget(self.label_4, 1, 2, 1, 1)
self.textBrowser_log = QtWidgets.QTextBrowser(self.widget)
self.textBrowser_log.setObjectName("textBrowser_log")
self.gridLayout.addWidget(self.textBrowser_log, 1, 6, 10, 1)
self.lineEdit_free_nav_start = QtWidgets.QLineEdit(self.widget)
self.lineEdit_free_nav_start.setObjectName("lineEdit_free_nav_start")
self.gridLayout.addWidget(self.lineEdit_free_nav_start, 2, 0, 1, 2)
self.lineEdit_free_nav_goal = QtWidgets.QLineEdit(self.widget)
self.lineEdit_free_nav_goal.setObjectName("lineEdit_free_nav_goal")
self.gridLayout.addWidget(self.lineEdit_free_nav_goal, 2, 2, 1, 3)
self.label_6 = QtWidgets.QLabel(self.widget)
font = QtGui.QFont()
font.setItalic(True)
self.label_6.setFont(font)
self.label_6.setObjectName("label_6")
self.gridLayout.addWidget(self.label_6, 3, 0, 1, 1)
spacerItem = QtWidgets.QSpacerItem(20, 13, QtWidgets.QSizePolicy.Minimum,
QtWidgets.QSizePolicy.Expanding)
self.gridLayout.addItem(spacerItem, 3, 4, 1, 1)
self.pushButton_free_nav = QtWidgets.QPushButton(self.widget)
self.pushButton_free_nav.setObjectName("pushButton_free_nav")
self.gridLayout.addWidget(self.pushButton_free_nav, 4, 1, 1, 2)
spacerItem1 = QtWidgets.QSpacerItem(20, 40, QtWidgets.QSizePolicy.Minimum,
QtWidgets.QSizePolicy.Expanding)
```

```
self.gridLayout.addItem(spacerItem1, 5, 1, 1, 1)
self.label_7 = QtWidgets.QLabel(self.widget)
font = QtGui.QFont()
font.setPointSize(15)
self.label_7.setFont(font)
self.label_7.setObjectName("label_7")
self.gridLayout.addWidget(self.label_7, 6, 0, 1, 3)
self.label_8 = QtWidgets.QLabel(self.widget)
self.label_8.setObjectName("label_8")
self.gridLayout.addWidget(self.label_8, 7, 0, 1, 1)
self.label_9 = QtWidgets.QLabel(self.widget)
self.label_9.setObjectName("label_9")
self.gridLayout.addWidget(self.label_9, 7, 2, 1, 1)
self.lineEdit_landmark_nav = QtWidgets.QLineEdit(self.widget)
self.lineEdit_landmark_nav.setObjectName("lineEdit_landmark_nav")
self.gridLayout.addWidget(self.lineEdit_landmark_nav, 8, 0, 1, 2)
self.lineEdit_landmark_nav_2 = QtWidgets.QLineEdit(self.widget)
self.lineEdit_landmark_nav_2.setObjectName("lineEdit_landmark_nav_2")
self.gridLayout.addWidget(self.lineEdit_landmark_nav_2, 8, 2, 1, 4)
self.label_10 = QtWidgets.QLabel(self.widget)
font = QtGui.QFont()
font.setItalic(True)
self.label_10.setFont(font)
self.label_10.setObjectName("label_10")
self.gridLayout.addWidget(self.label_10, 9, 0, 1, 2)
spacerItem2 = QtWidgets.QSpacerItem(20, 13, QtWidgets.QSizePolicy.Minimum,
QtWidgets.QSizePolicy.Expanding)
self.gridLayout.addItem(spacerItem2, 9, 3, 1, 1)
self.pushButton_landmark_nav = QtWidgets.QPushButton(self.widget)
self.pushButton_landmark_nav.setObjectName("pushButton_landmark_nav")
self.gridLayout.addWidget(self.pushButton_landmark_nav, 10, 1, 1, 2)
Robot_Navigation_GUI.setCentralWidget(self.centralwidget)
self.menubar = QtWidgets.QMenuBar(Robot_Navigation_GUI)
self.menubar.setGeometry(QtCore.QRect(0, 0, 931, 22))
self.menubar.setObjectName("menubar")
Robot_Navigation_GUI.setMenuBar(self.menubar)
self.statusbar = QtWidgets.QStatusBar(Robot_Navigation_GUI)
```

```

self.statusbar.setObjectName("statusbar")
Robot_Navigation_GUI.setStatusBar(self.statusbar)

self.retranslateUi(Robot_Navigation_GUI)
QtCore.QMetaObject.connectSlotsByName(Robot_Navigation_GUI)

def retranslateUi(self, Robot_Navigation_GUI):
    _translate = QtCore.QCoreApplication.translate
    Robot_Navigation_GUI.setWindowTitle(_translate("Robot_Navigation_GUI", "Robot
Navigation GUI"))
    self.label_11.setText(_translate("Robot_Navigation_GUI", "Map"))
    self.label.setText(_translate("Robot_Navigation_GUI", "Free Navigation"))
    self.label_2.setText(_translate("Robot_Navigation_GUI", "LOG"))
    self.label_3.setText(_translate("Robot_Navigation_GUI", "Start"))
    self.label_4.setText(_translate("Robot_Navigation_GUI", "Goal"))
    self.label_6.setText(_translate("Robot_Navigation_GUI", "Usage: x;y;z;w"))
    self.pushButton_free_nav.setText(_translate("Robot_Navigation_GUI", "Start Robot"))
    self.label_7.setText(_translate("Robot_Navigation_GUI", "Landmark Navigation"))
    self.label_8.setText(_translate("Robot_Navigation_GUI", "Start"))
    self.label_9.setText(_translate("Robot_Navigation_GUI", "Goal"))
    self.label_10.setText(_translate("Robot_Navigation_GUI", "Usage example: 1_W"))
    self.pushButton_landmark_nav.setText(_translate("Robot_Navigation_GUI", "Start
Robot"))

```

Das Layout-File wird dann in der Hauptanwendung importiert und die GUI-Funktionalität darauf aufgebaut.

nav_GUI.py

```

from PyQt5 import QtCore, QtGui, QtWidgets
from PyQt5.QtWidgets import QApplication
import sys
import time
import threading
import nav_GUI_layout
import rospy
import actionlib

```

```
from move_base_msgs.msg import MoveBaseAction, MoveBaseGoal
from math import radians, degrees
from actionlib_msgs.msg import *
from geometry_msgs.msg import Point, PoseWithCovarianceStamped

class Robot_GUI(QtWidgets.QMainWindow, nav_GUI_layout.Ui_Robot_Navigation_GUI):
    def __init__(self, parent=None):
        super(Robot_GUI, self).__init__(parent)
        self.setupUi(self)

        self.landmarks = {
            "1_O": [0.287551669973, 0.166076723376, 0.0, -0.0212901784965,
0.999773338462],
            "1_W": [0.287551669973, 0.166076723376, 0.0, -0.999019496747,
0.0212901784965],
            "2_O": [0.5831331515, -4.14627704066, 0.0, -0.00571982419211,
0.999983641672],
            "2_W": [0.5831331515, -4.14627704066, 0.0, -0.999983641672,
0.0247981156306],
            "3_N": [7.84857570162,-3.99124300721, 0.0, 0.733688684782,
0.679485771612],
            "3_W": [7.84857570162,-3.99124300721, 0.0, -0.999930475474,
0.0117917012305],
            "3_S": [7.84857570162,-3.99124300721, 0.0, -0.702700050933,
0.711486218011],
            "4_O": [0.235598277095, 5.15163679907, 0.0, 0.0294854278791,
0.99956521025],
            "4_W": [0.235598277095, 5.15163679907, 0.0, -0.999511983046,
0.0312377295465],
            "5_O": [6.26796687525, 4.7685984255, 0.0, 0.0318287091979,
0.999493338282],
            "5_W": [6.26796687525, 4.7685984255, 0.0, -0.999998503618,
0.00172996026245],
            "6_N": [15.5448061462, 7.1165123077, 0.0, 0.6981787861,
0.715923447472],
            "6_S": [15.5448061462, 7.1165123077, 0.0, -0.705759690111,
0.708451310828],
```



```

        "7_O": [27.0953896995, 3.42688735034, 0.0, 0.0123469380298,
0.999923773655],
        "7_W": [27.0953896995, 3.42688735034, 0.0, -0.999988604097,
0.00477406284339]
    }

    #button callbacks
    self.pushButton_free_nav.clicked.connect(self.callback_free_nav)
    self.pushButton_landmark_nav.clicked.connect(self.callback_landmark_nav)

def callback_free_nav(self):
    init_pose_set = self.set_initial_pose(field_ptr=self.lineEdit_free_nav_start)

    if init_pose_set:
        goal_pose_str = self.lineEdit_free_nav_goal.text() #pattern: x;y;z;w
        goal_pose_list = goal_pose_str.split(";")
        x = float(goal_pose_list[0])
        y = float(goal_pose_list[1])
        z = float(goal_pose_list[2])
        w = float(goal_pose_list[3])
        self.moveToGoal(x,y,z,w)
    else:
        self.set_log_message("Goal cannot be executed. No initial pose given.")

def callback_landmark_nav(self):
    init_pose_set = self.set_initial_pose(landmark = self.lineEdit_landmark_nav.text())

    if init_pose_set:
        goal_pose = self.lineEdit_landmark_nav_2.text()
        goal_pose_list = self.landmarks[goal_pose]
        x = goal_pose_list[0]
        y = goal_pose_list[1]
        z = goal_pose_list[3]
        w = goal_pose_list[4]
        self.moveToGoal(x,y,z,w)

def set_initial_pose(self, field_ptr=None, landmark=None):

```

```
pub = rospy.Publisher('initialpose', PoseWithCovarianceStamped, queue_size=10 )
checks_passed = False

#read initial pose
if field_ptr != None:
    start_pose_str=field_ptr.text() #pattern: x;y;z;w (z und w sind orientation)
    start_pose_list = start_pose_str.split(";")
    checks_passed = True
elif landmark!= None:
    #use landmark
    if landmark in self.landmarks.keys():
        start_pose_list = self.landmarks[landmark]
        checks_passed = True
    else:
        self.set_log_message("Landmark is not correct!")
else:
    self.set_log_message("Landmark or Coordinates must be given!")

if checks_passed:
    pose = PoseWithCovarianceStamped()
    pose.header.frame_id="map"
    pose.header.stamp = rospy.Time.now()
    pose.pose.pose.position.x = float(start_pose_list[0])
    pose.pose.pose.position.y = float(start_pose_list[1])
    pose.pose.pose.position.z = 0
    pose.pose.pose.orientation.x = 0
    pose.pose.pose.orientation.y = 0
    pose.pose.pose.orientation.z = float(start_pose_list[2])
    pose.pose.pose.orientation.w = float(start_pose_list[3])
    pose.pose.covariance= [0.25, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.25, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.06853892326654787]

    pub.publish(pose)
    time.sleep(1)
    pub.publish(pose)
```

```
time.sleep(1)
pub.publish(pose)

self.set_log_message("Initial Pose set")
return True
else:
    return False

def moveToGoal(self,xGoal,yGoal,zGoal, wGoal):

    #define a client for to send goal requests to the move_base server through a
SimpleActionClient
    ac = actionlib.SimpleActionClient("move_base", MoveBaseAction)

    #wait for the action server to come up
    while(not ac.wait_for_server(rospy.Duration.from_sec(5.0))):
        rospy.loginfo("Waiting for the move_base action server to come up")

    goal = MoveBaseGoal()

    #set up the frame parameters
    goal.target_pose.header.frame_id = "map"
    goal.target_pose.header.stamp = rospy.Time.now()

    # moving towards the goal*/

    goal.target_pose.pose.position = Point(xGoal,yGoal,0)
    goal.target_pose.pose.orientation.x = 0.0
    goal.target_pose.pose.orientation.y = 0.0
    goal.target_pose.pose.orientation.z = zGoal
    goal.target_pose.pose.orientation.w = wGoal

    rospy.loginfo("Sending goal location ...")
    self.set_log_message("Sending goal location ...")
    ac.send_goal(goal)
```

```
ac.wait_for_result(rospy.Duration(60))

if(ac.get_state() == GoalStatus.SUCCEEDED):
    rospy.loginfo("You have reached the destination")
    self.set_log_message("You have reached the destination")
    return True

else:
    rospy.loginfo("The robot failed to reach the destination")
    self.set_log_message("The robot failed to reach the destination")
    return False

def set_log_message(self, mes):
    log = "[{}].format(time.strftime("%H:%M:%S", time.localtime())) + mes
    self.textBrowser_log.append(log)

def main():
    rospy.init_node('navigation_GUI', anonymous=True)
    app = QApplication(sys.argv)
    form = Robot_GUI()
    form.show()
    app.exec_()

if __name__ == '__main__':
    main()
```